

ORACLE

CloudNativeなMCPサーバー のための基礎知識

Yuki Sogawa
Oracle Japan



自己紹介



曾川 有輝

日本オラクル CloudNative/AI,ML ク
ラウドエンジニア

バックグラウンド

大阪大学でデータセンターの省電力化の研究をしていました

趣味

ギター、バイク、ゲームなど

Agenda

- 1 MCPとは
- 2 MCPサーバーのスケーラビリティ
- 3 MCPでの認可
- 4 MCPサーバーのObservability
- 5 デモ
- 6 まとめ

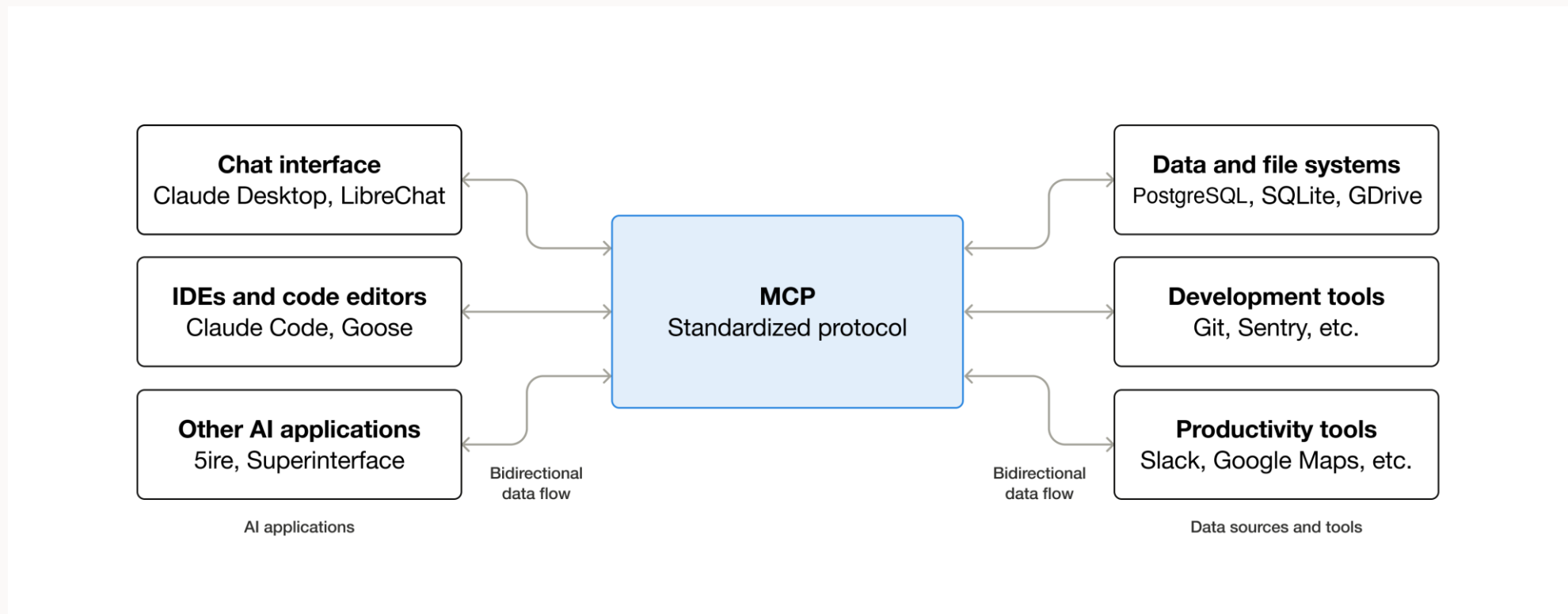
MCPとは



MCPとは？

AIアプリケーションを外部システムに接続するための標準規格

Anthropicが提唱した、AIアプリケーションを外部システムと連携させるためのプロトコル

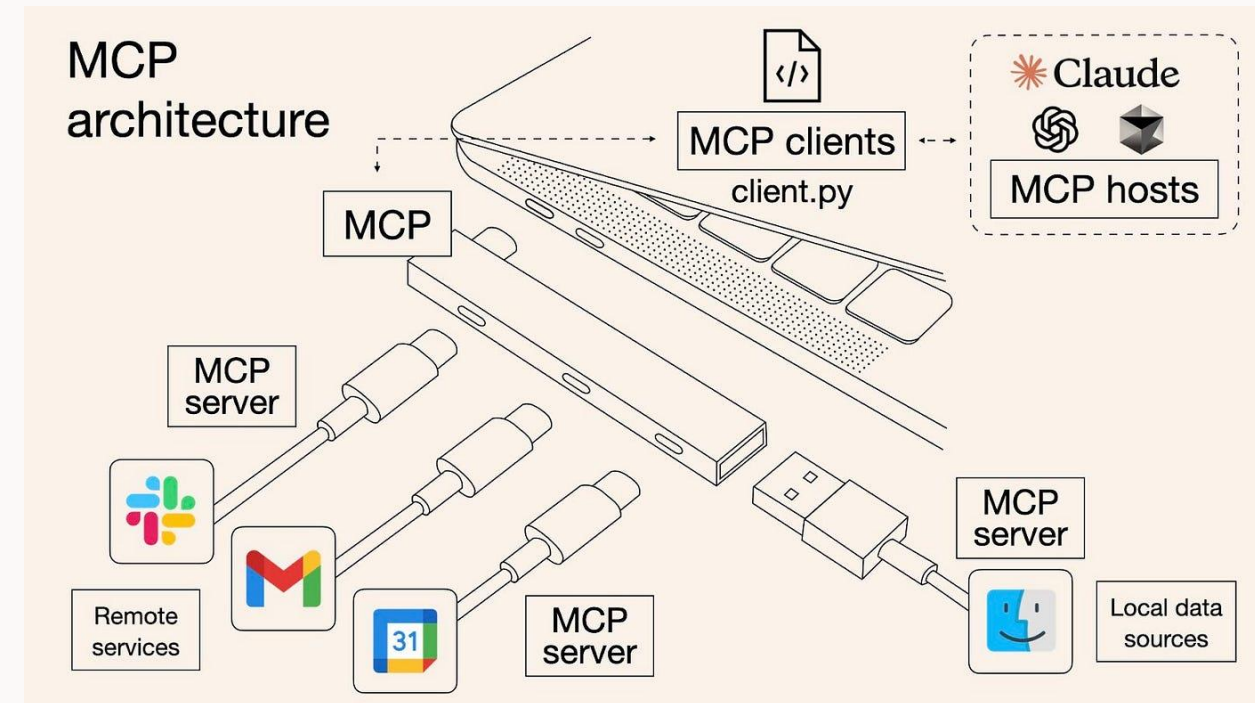


MCPを使うと何ができる？

LLMに、アクションを実行するための手足を与えることができる

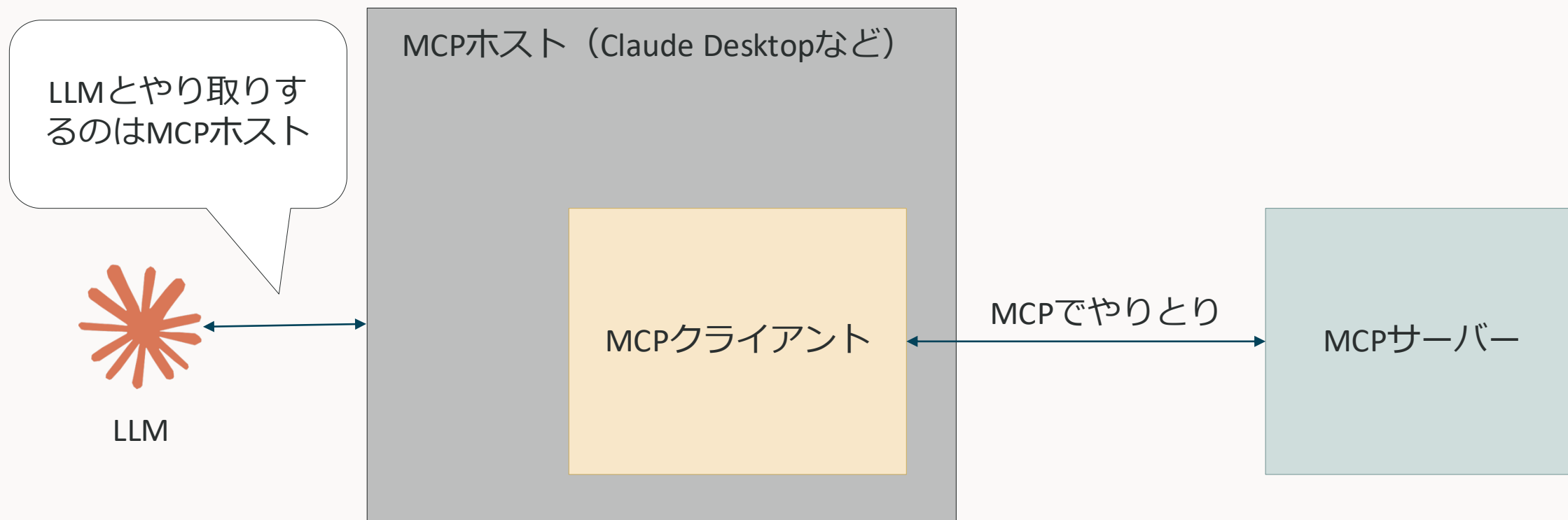
MCPを使うと自然言語で色々なアクションが実行できるようになる

- データベースを操作する
- ファイル操作を行う
- 天気APIから天気の情報を取得する
- メールを送信する
- コードを実行する
- 他のシステム（Kubernetesなど）を操作する
- Etc...



MCPのアーキテクチャ

ホスト・クライアント・サーバーの3つの登場人物がいる



MCPのアーキテクチャ（レイヤー）

データ層とトランスポート層の2層構造

データ層

- JSON-RPCでメッセージのやり取りを行う
- MCPサーバーの機能
 - Tools
 - Resources
 - Prompts
- MCPクライアントの機能
 - Roots
 - Sampling
 - Elicitation

などを定義している（詳細は後述します）

トランスポート層

- JSON-RPCのメッセージをどのようにやり取りするかを定義する
- 標準入出力でやり取り
- Server Sent Eventsでやりとり（2024-11-05まで）
- Streamable HTTPでやり取り（以降）

などを定義している（詳細は後述します）

MCPのデータ層

クライアント・サーバー間のメッセージのフォーマット、JSON-RPC 2.0

JSON-RPCは、Remote Procedure Callを実現するプロトコルの一つ。

Server Sent Events
Streamable HTTP

REST APIの方が聞き馴染みがあるが、JSON-RPCだと非対称な通信路でも使える

- REST API : リクエストとレスポンスは同一コネクション
- RPC: idでリクエストとレスポンスを紐づけるので、同一コネクションである必要はない

```
ツール呼び出しリクエスト ツールコールレスポンス

{
  "jsonrpc": "2.0",
  "id": 3,
  "method": "tools/call",
  "params": {
    "name": "weather_current",
    "arguments": {
      "location": "San Francisco",
      "units": "imperial"
    }
  }
}
```

```
ツール呼び出しリクエスト ツールコールレスポンス

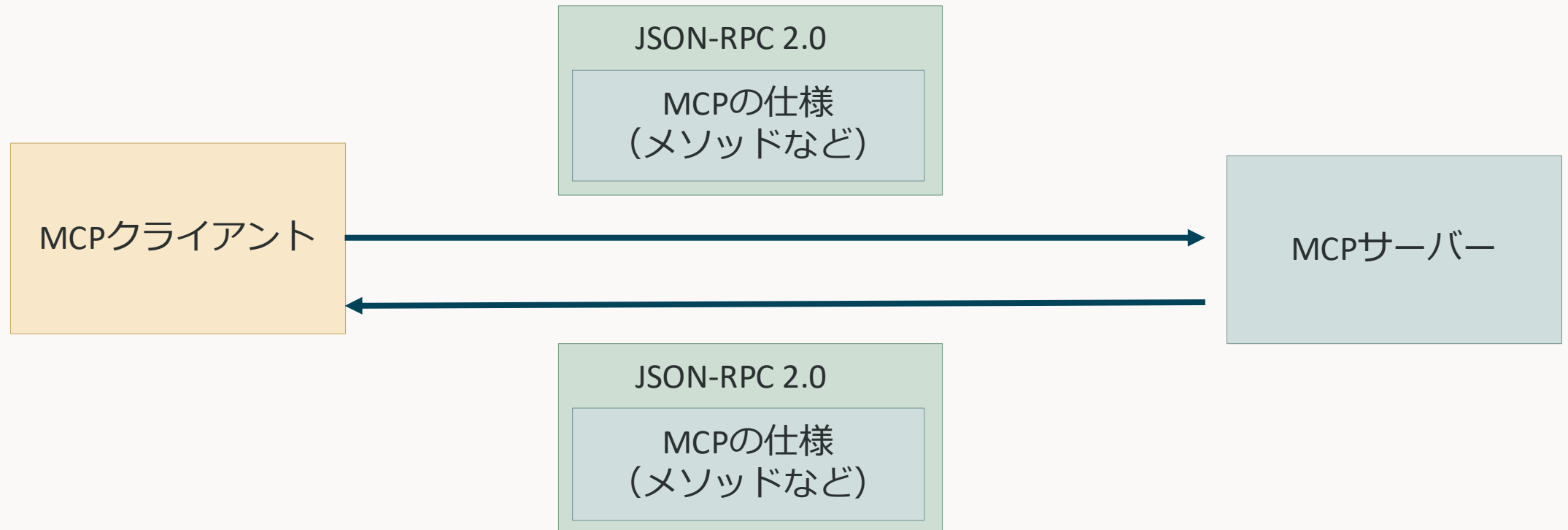
{
  "jsonrpc": "2.0",
  "id": 3,
  "result": {
    "content": [
      {
        "type": "text",
        "text": "Current weather in San Francisco: 68°F, partly cloudy with li"
      }
    ]
  }
}
```

MCPのデータ層

クライアント・サーバー間のメッセージのフォーマット、JSON-RPC 2.0

*JSON-RPC 2.0では、jsonrpc, id, method, paramsなどのフィールドが定義されているが、その中に何がくるのかまでは定義されていない

MCPは、その中（例えばmethodの中）にどういう名前が入るかななどを定義している



MCPのトランスポート層

標準入出力

- MCPクライアントとMCPサーバーを同じマシン上で実行する場合に使用する
- 基本的に認可などが不要なので非常に手軽に利用できる
- MCPクライアントのサブプロセスとしてMCPサーバーを起動する



Streamable HTTP

- リモートのMCPサーバーを利用する場合に使用する
- MCPは、サーバー起点でクライアントに通信を開始する場合がある
- しかし通常のHTTPでは、常にクライアント起点の通信しかできない
- HTTPの拡張仕様で、サーバー起点の通信もできるようにしたものがStreamable HTTP

MCPサーバーのプリミティブ

3つの機能がある

- Tools
 - LLMがアクティブに呼び出せる関数
 - データベースへの書き込み、外部APIの呼び出し、ファイル変更など
- Resources
 - LLMに追加のコンテキストを与える機能（RAGのためのデータ供給源を提供）
 - サーバーにあるファイルの内容
 - データベースのデータ
 - ドキュメント など
- Prompts
 - LLMがToolsやResourcesを上手く扱えるようにするためのプロンプトを提供する機能
 - ToolsやResourcesの呼び出しをユースケース単位でテンプレート化しておく
 - 正しくツール呼び出しが行われるようなプロンプトをあらかじめ用意しておく
 - Issueを作成してからPR作成して など

Tools

LLMに関数を実行させる

二つのメソッドがある

- tools/list
 - そのMCPサーバーで利用できるツールのリストを提供する
 - ツールがどういったものを事前にLLMに教える
- tools/call
 - ツールを実行する

Tools

利用の流れ1

まずはLLMにどんなツールがあるかを教えてあげる



LLM



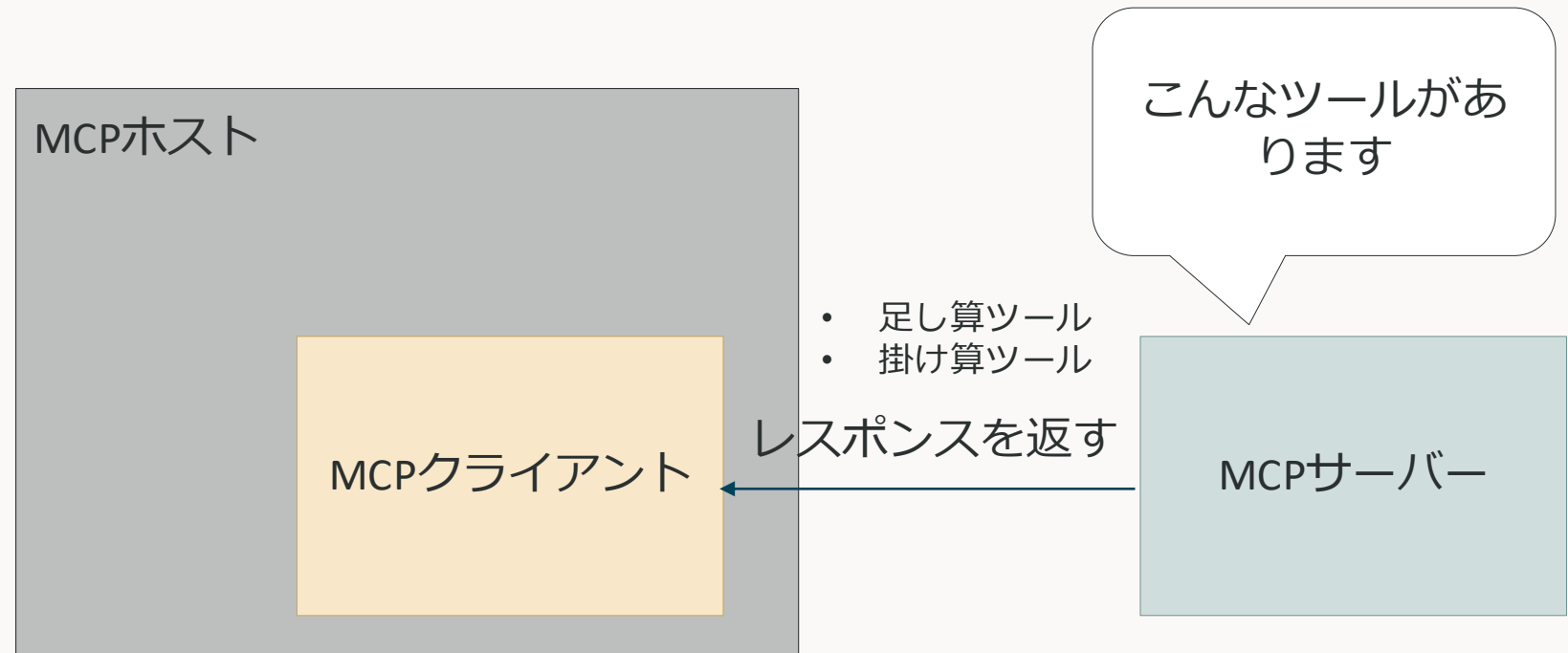
Tools

利用の流れ1

まずはLLMにどんなツールがあるかを教えてあげる



LLM



Tools

利用の流れ1

まずはLLMにどんなツールがあるかを教えてあげる

- 足し算ツール
 - 引数 1 と引数 2 を足します
- 掛け算ツール
 - 引数 1 と引数 2 をかけます

プロンプトにツール
リストを入れる



LLM

MCPホスト

このMCPサーバー
にはこんなツール
があるみたいです

MCPクライアント

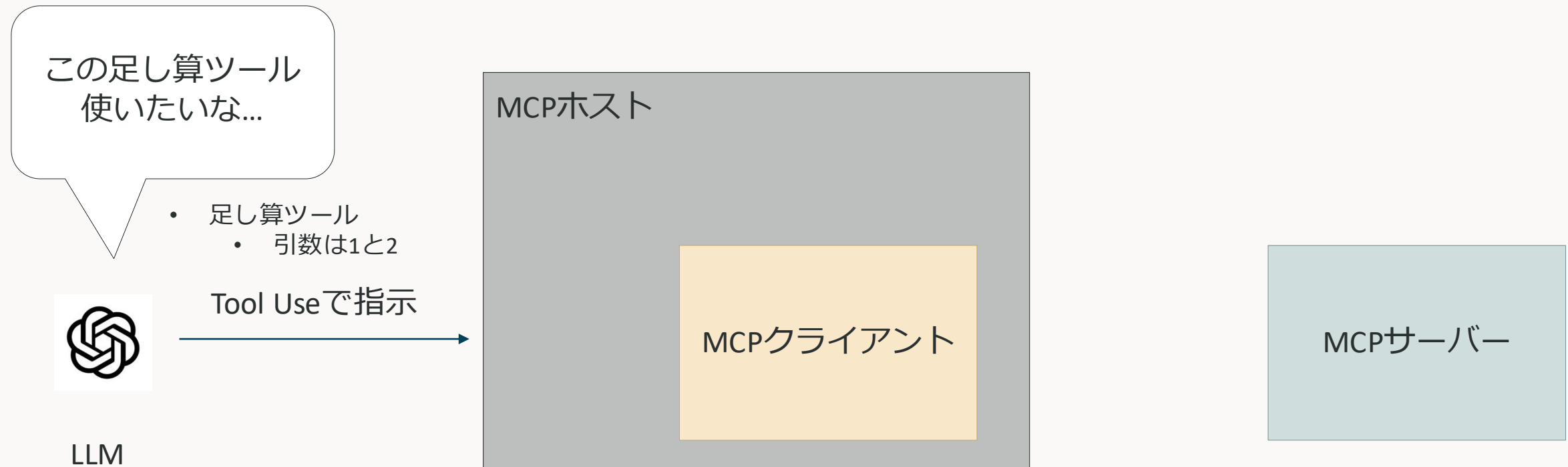
MCPサーバー

※LLMと直接やり取りするのはMCPホスト

Tools

利用の流れ2

LLMがツール実行を行う

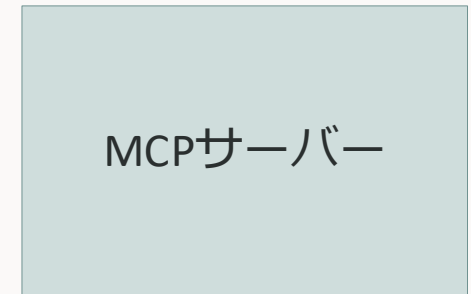


Tools 利用の流れ2

LLMがツール実行を行う



LLM



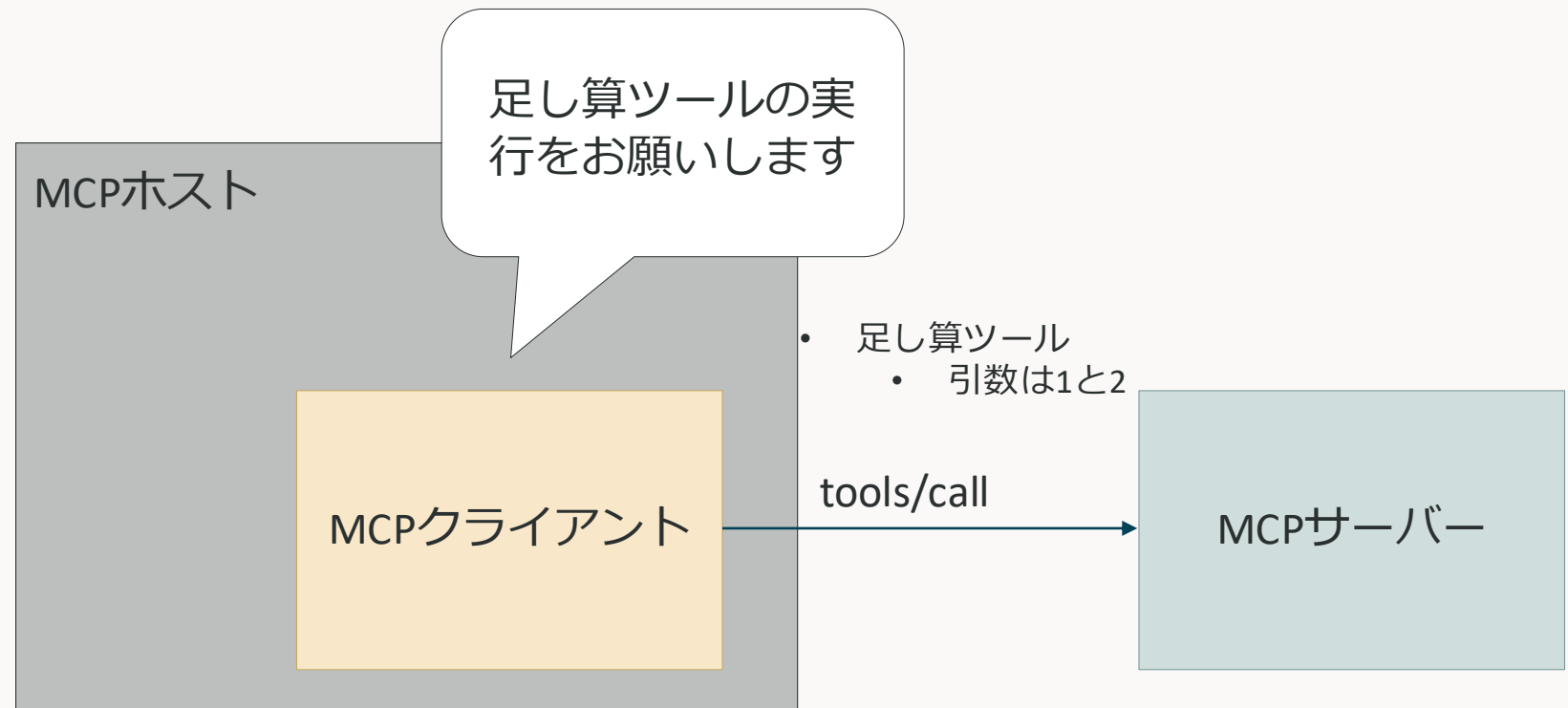
LLMからの要求をみてMCPクライアントを呼び出す

Tools 利用の流れ2

LLMがツール実行を行う



LLM

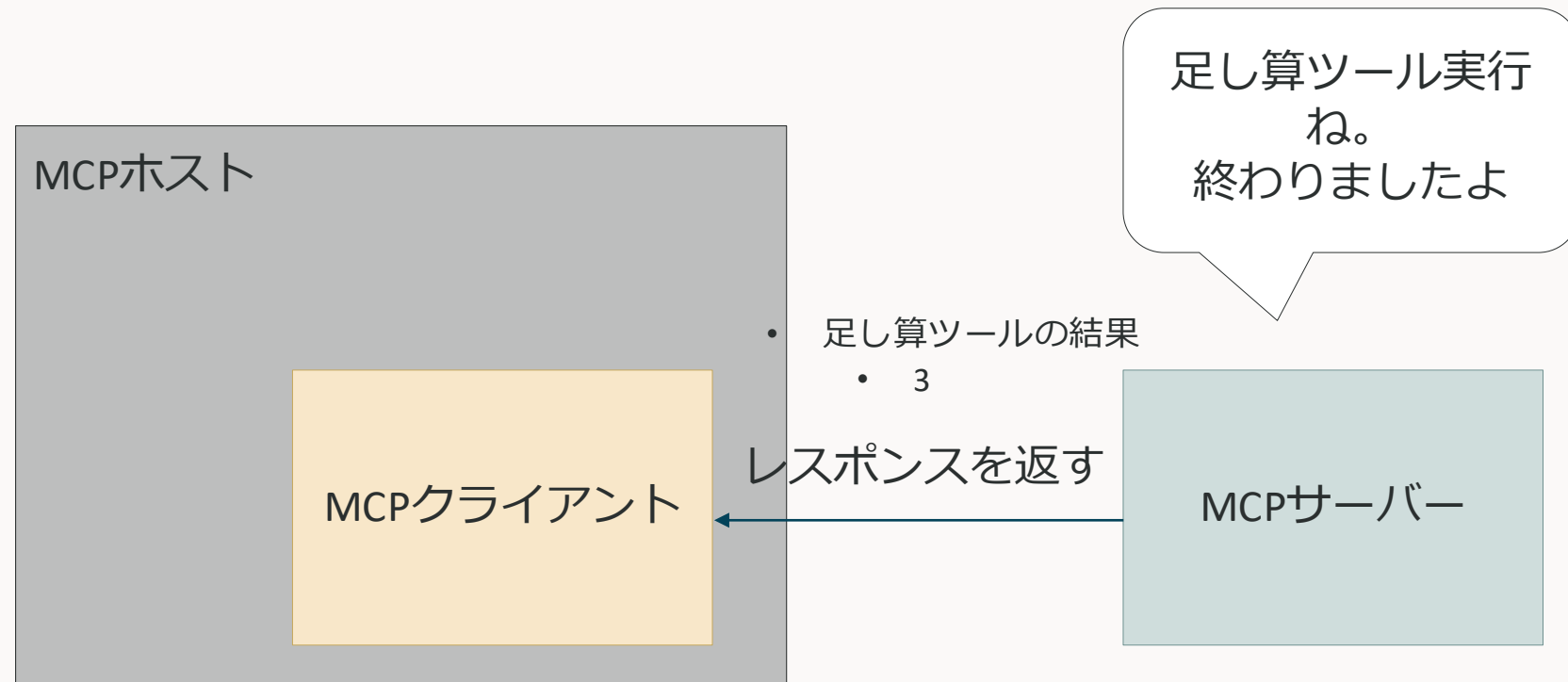


Tools 利用の流れ2

LLMがツール実行を行う

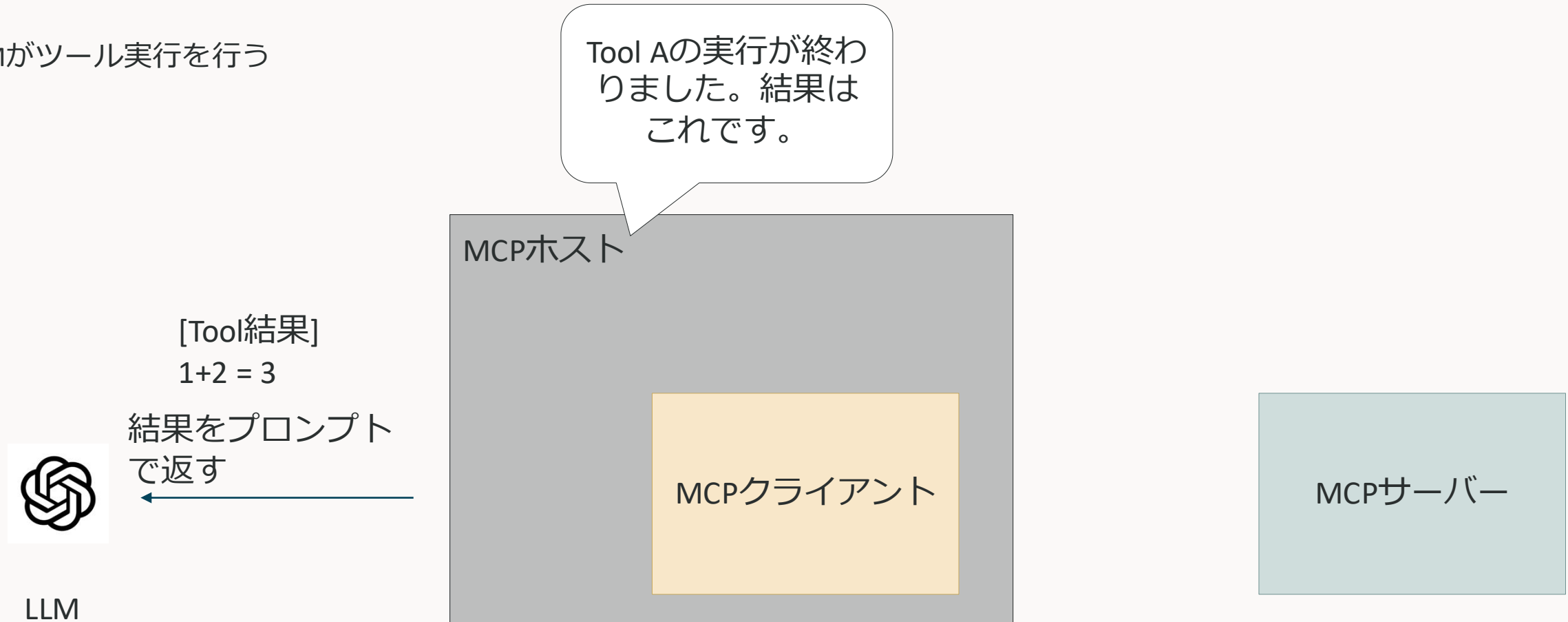


LLM



Tools 利用の流れ2

LLMがツール実行を行う



Tools

注意点

- ツールリストは、MCPホスト起点でLLMに教えてあげる必要がある
 - ツールリストを与えないと、そもそもLLMはMCPサーバーの存在を知らない
- ツール実行の結果をどのようにLLMに伝達するかは、MCPホストの実装次第
 - LLMはあくまで入力された自然言語に対して、尤もらしい自然言語を出力するだけ
- LLMは無邪気にTool実行を要求するため、危険度の高いToolの扱いには注意が必要
 - シェルが実行できるToolを提供するなら、操作できる範囲はクライアントかサーバー側で定めておく
 - データベース操作ができるToolを提供するなら、クエリの安全性はクライアントかサーバー側で定めておく
 - そもそもサンドボックス環境（VM, コンテナ）でしか利用しない など

Tools

やり取りされるJSONスキーマ (tools/list)

リクエスト

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/list",
  "params": {
    "cursor": "optional-cursor-value"
  }
}
```

レスポンス

この結果をLLMにどう伝えるかはクライアント次第

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "tools": [
      {
        "name": "get_weather",
        "title": "Weather Information Provider",
        "description": "Get current weather information for a location",
        "inputSchema": {
          "type": "object",
          "properties": {
            "location": {
              "type": "string",
              "description": "City name or zip code"
            }
          },
          "required": ["location"]
        },
        "icons": [
```

Tools

やり取りされるJSONスキーマ (tools/call)

リクエスト

```
{
  "jsonrpc": "2.0",
  "id": 2,
  "method": "tools/call",
  "params": {
    "name": "get_weather",
    "arguments": {
      "location": "New York"
    }
  }
}
```

レスポンス

```
{
  "jsonrpc": "2.0",
  "id": 2,
  "result": {
    "content": [
      {
        "type": "text",
        "text": "Current weather in New York:\nTemperature: 72°F"
      }
    ],
    "isError": false
  }
}
```

この結果をLLMにどう伝えるかはクライアント次第

Resources

LLMに追加のコンテキストを与える

四つのメソッドがある

- resources/list
 - リソース（URI）のリストを提供する
- resources/templates/list
 - リソーステンプレート（こういうパターンのURIはいけますよ）のリストを提供する
- resources/read
 - リソースの内容を提供する
- resources/subscribe
 - リソースの変更を監視する機能を提供する

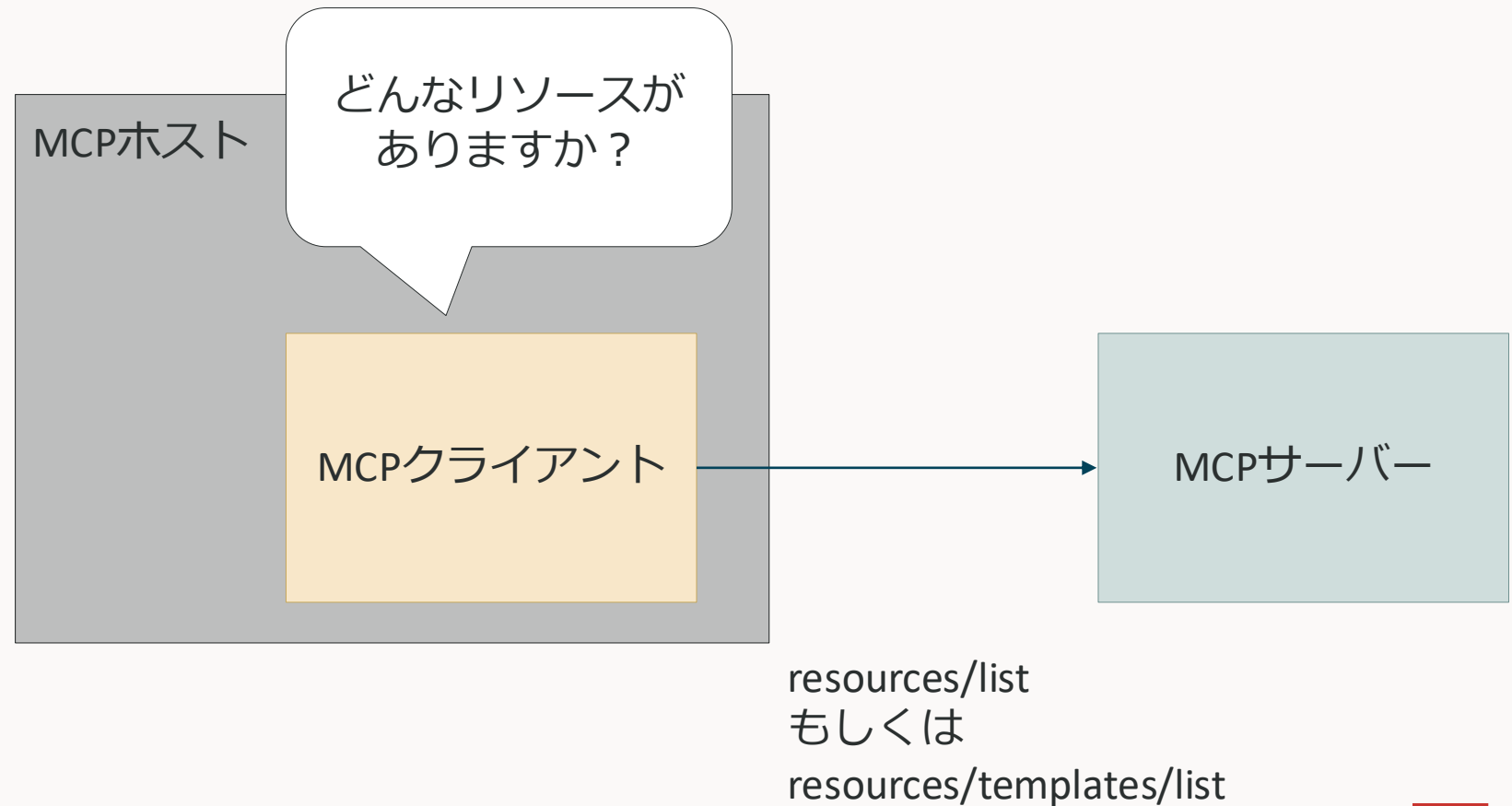
Tools

利用の流れ2

どういふリソースがあるかを把握する



LLM



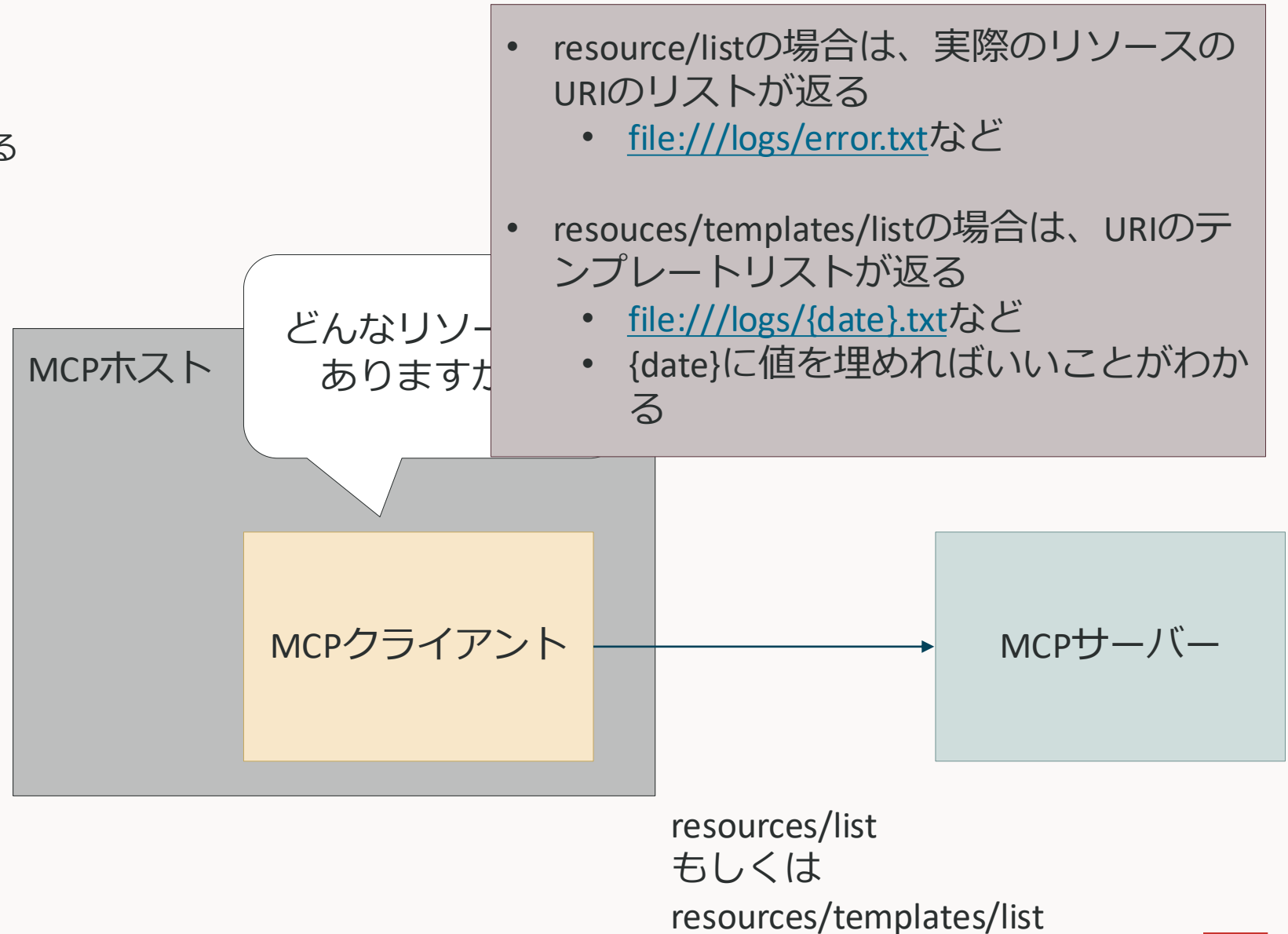
Tools

利用の流れ2

どういうリソースがあるかを把握する



LLM



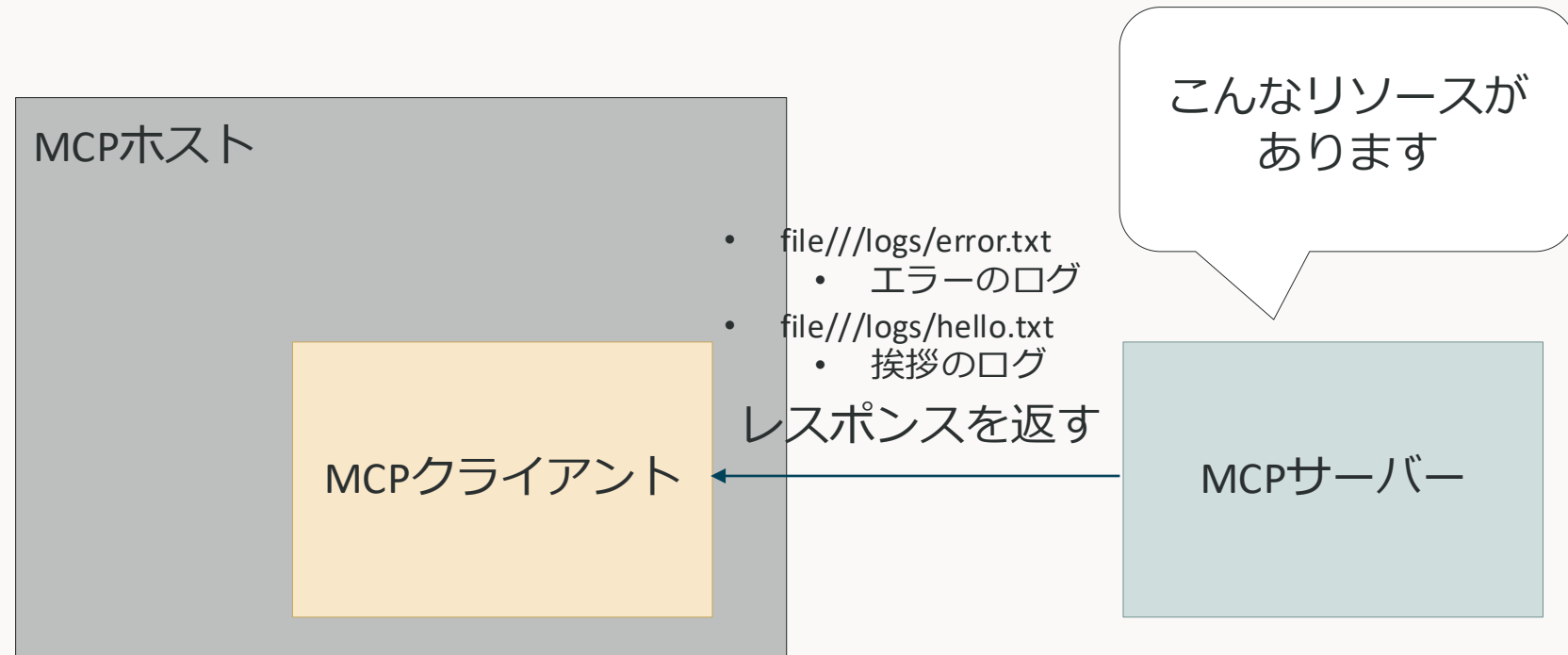
Tools

利用の流れ2

どういふリソースがあるかを把握する



LLM



Tools

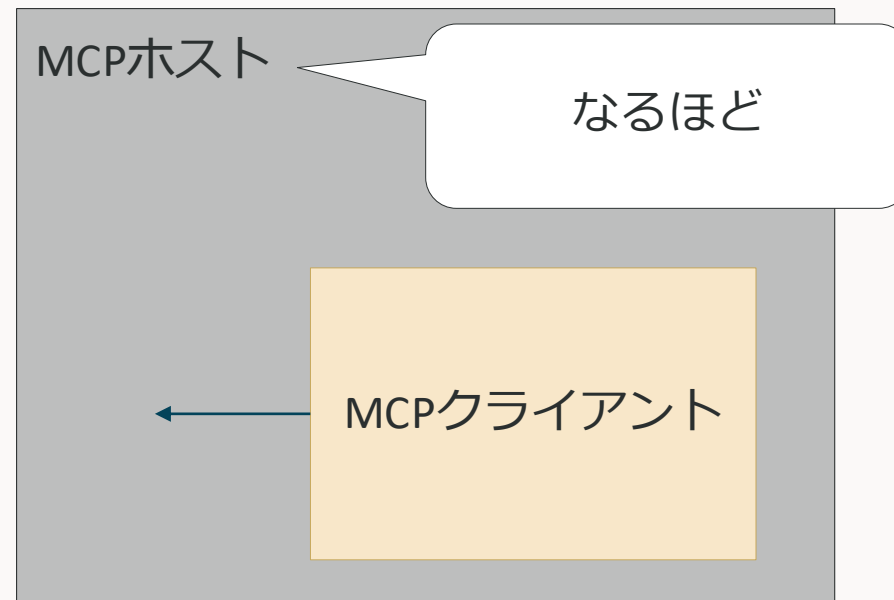
利用の流れ2

どういうリソースがあるかを把握する

※ リソースはクライアント（ユーザー）が利用開始するので、LLMにリソースのリストを渡す必要はない（後述する Promptsを使うとサーバーが利用開始することも可能）



LLM



MCPクライアントから受け取った結果を見てUI上に表示

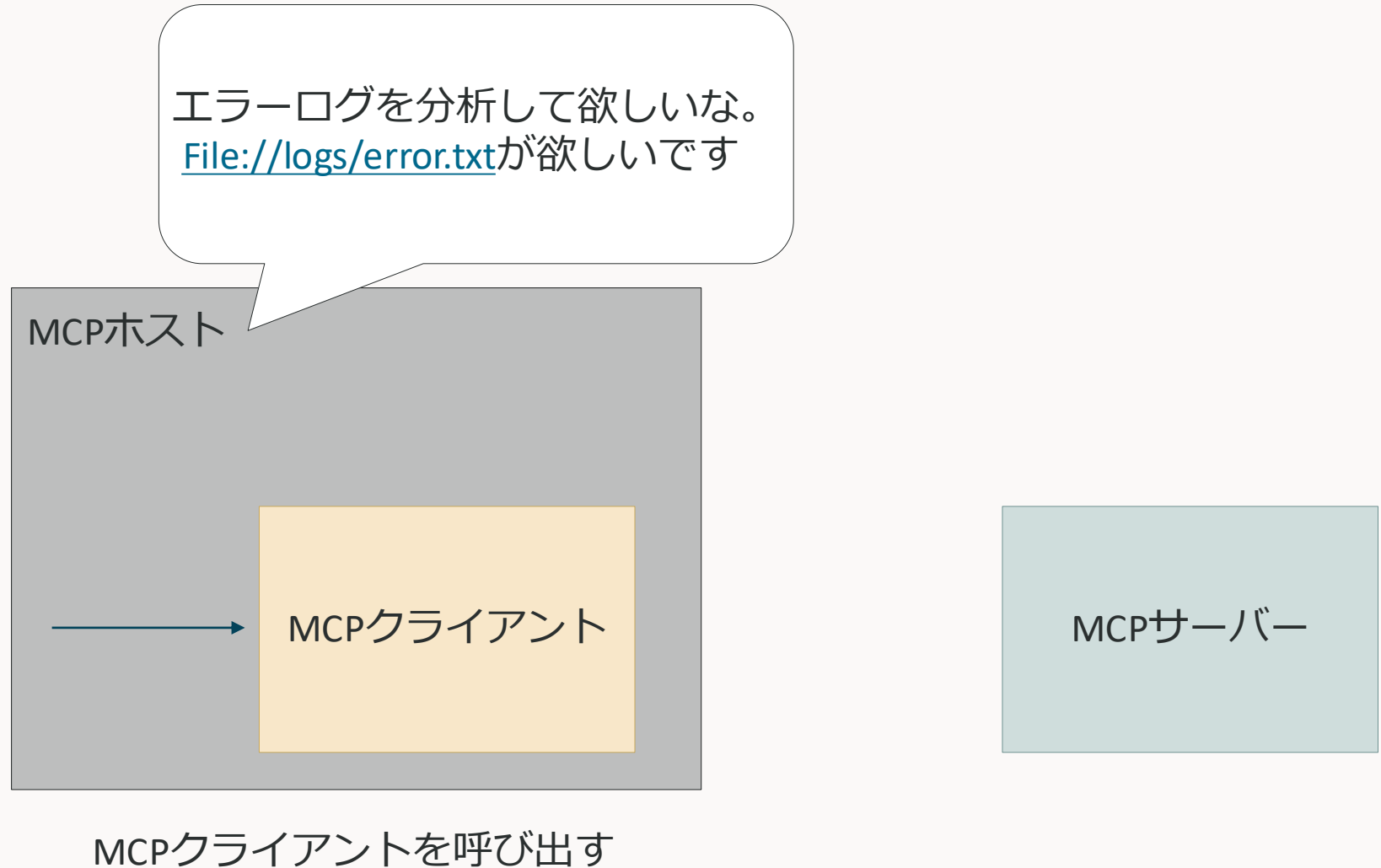
Resources

利用の流れ2

ユーザーがリソースを要求する



LLM



Resources 利用の流れ2

ユーザーがリソースを要求する



LLM



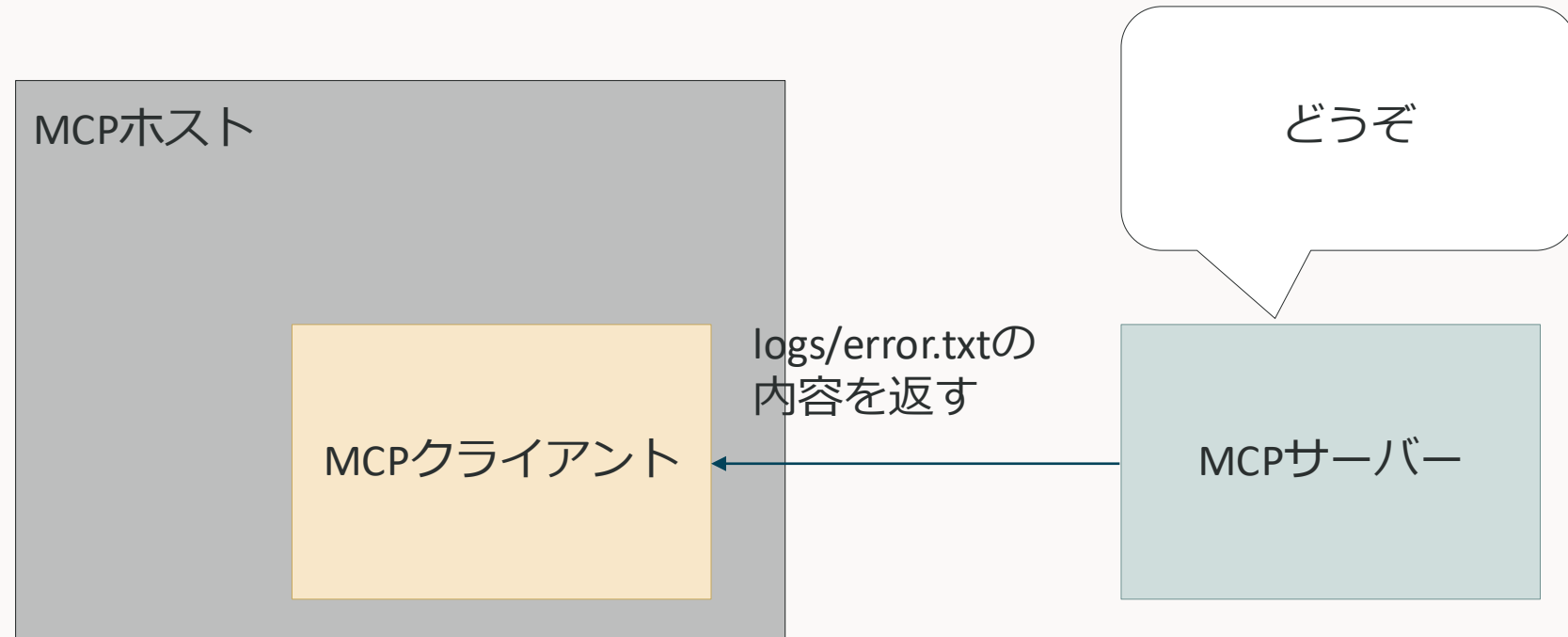
Resources

利用の流れ2

ユーザーがリソースを要求する

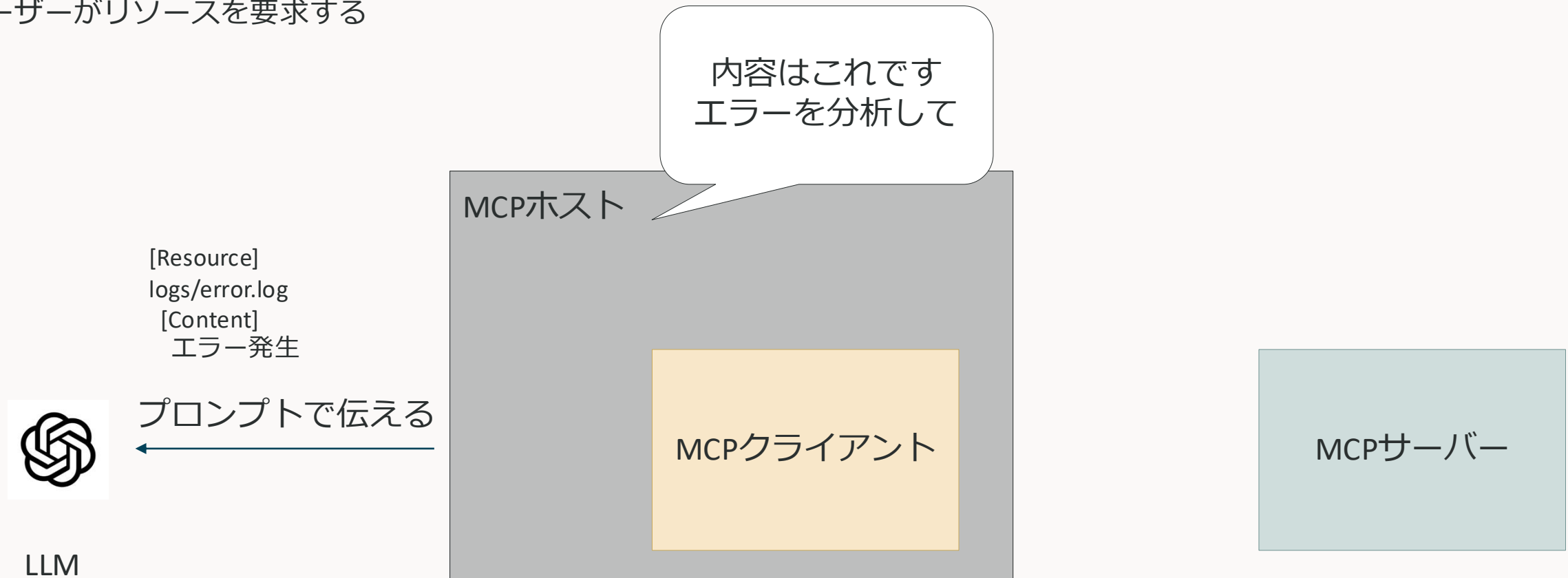


LLM



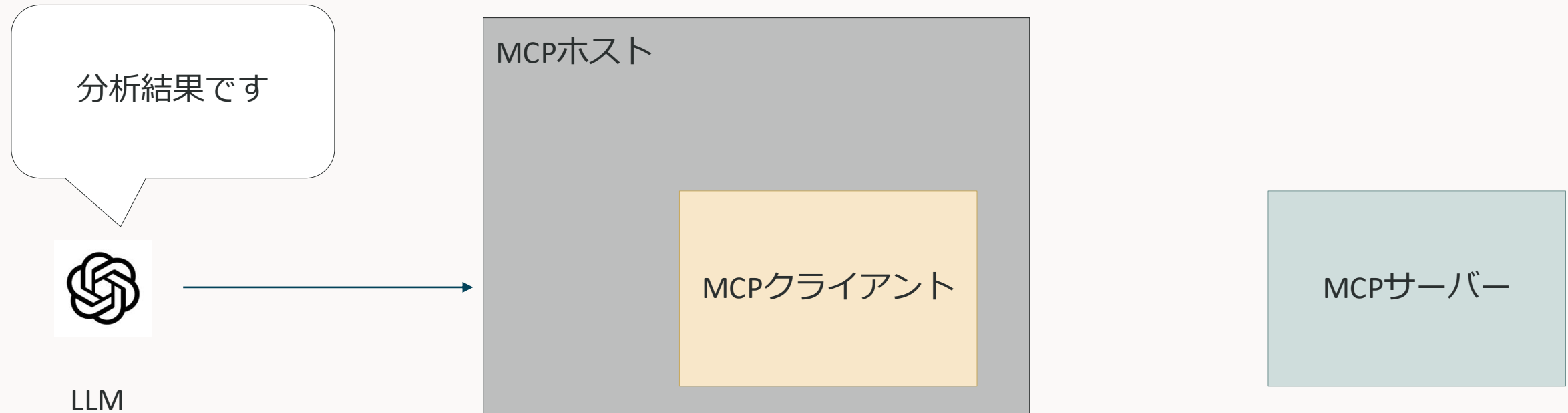
Resources 利用の流れ2

ユーザーがリソースを要求する



Resources 利用の流れ2

ユーザーがリソースを要求する



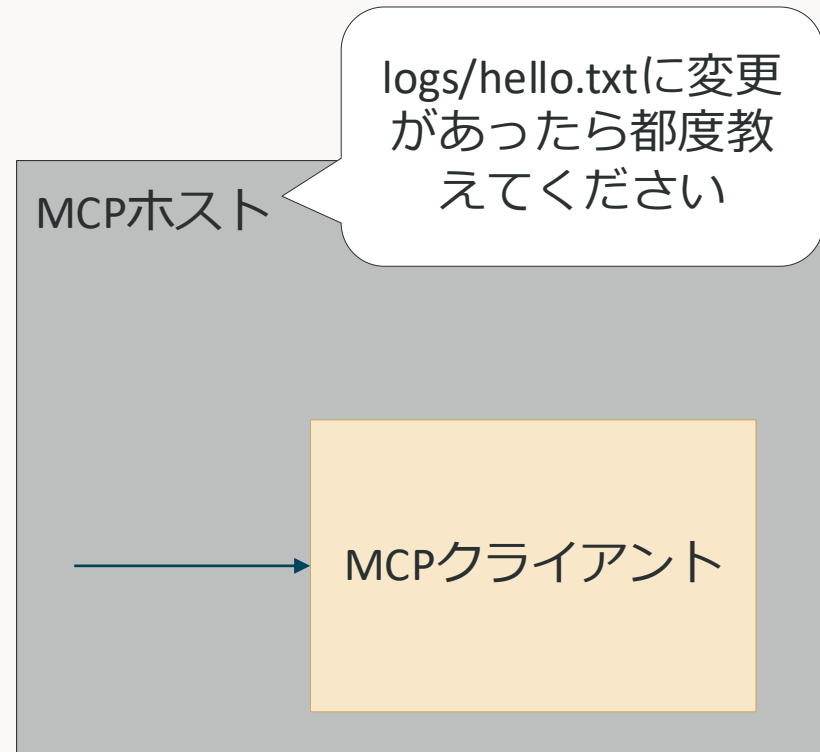
Resources

利用の流れ3

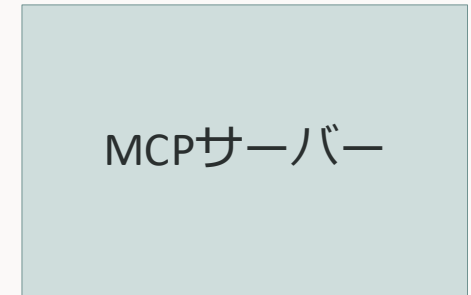
ユーザーもしくはMCPホストがリソースを要求する（自動で要求するかは実装による）



LLM



MCPクライアントを呼び出す



Resources 利用の流れ3

ユーザーもしくはMCPホストがリソースを要求する（自動で要求するかは実装による）



LLM



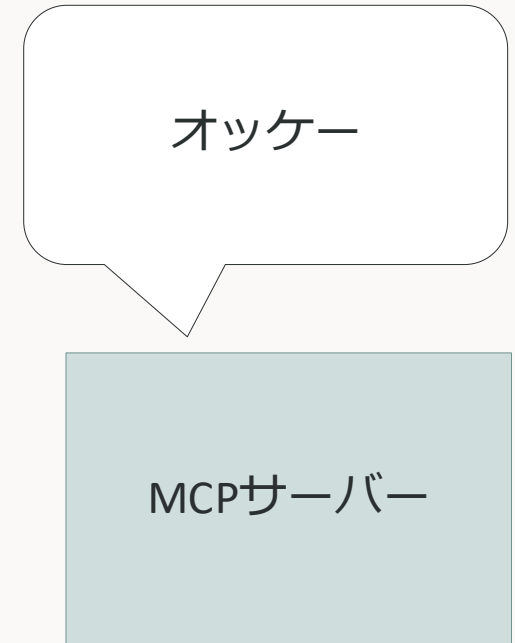
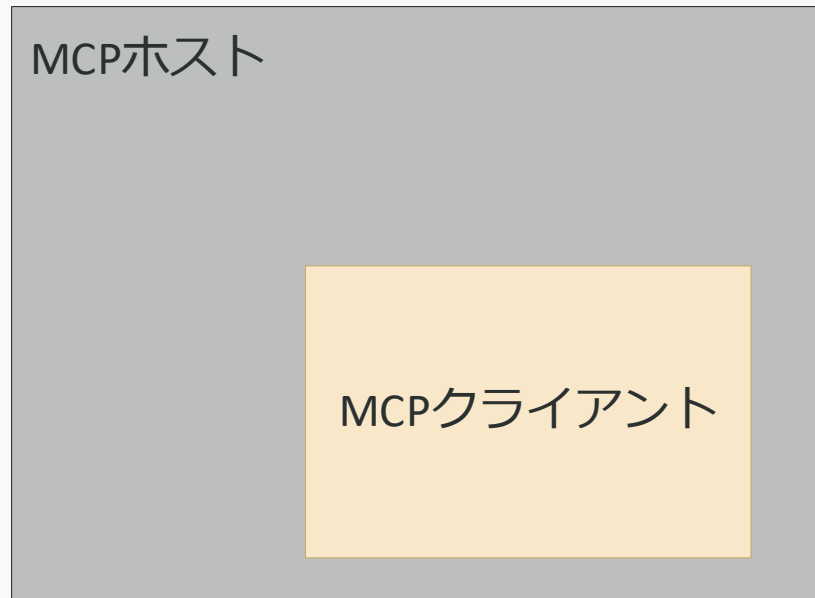
Resources

利用の流れ3

ユーザーもしくはMCPホストがリソースを要求する（自動で要求するかは実装による）



LLM

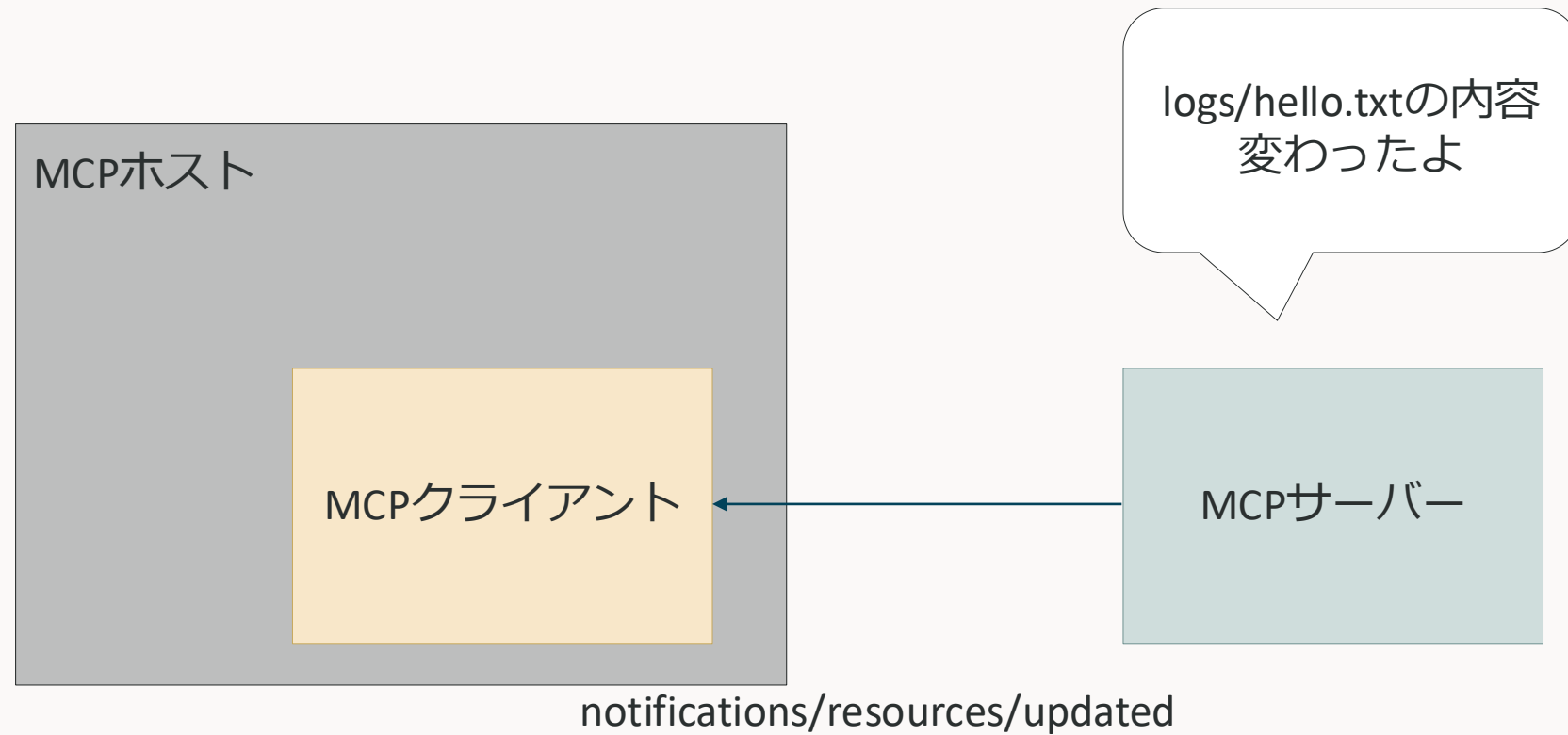


Resources 利用の流れ3

サブスクライブしたリソースに変更があった時



LLM



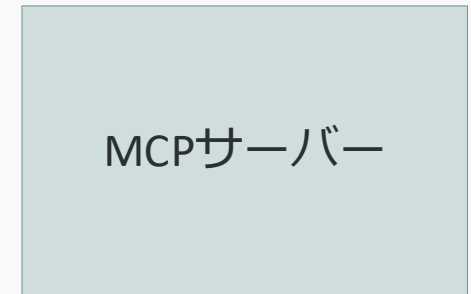
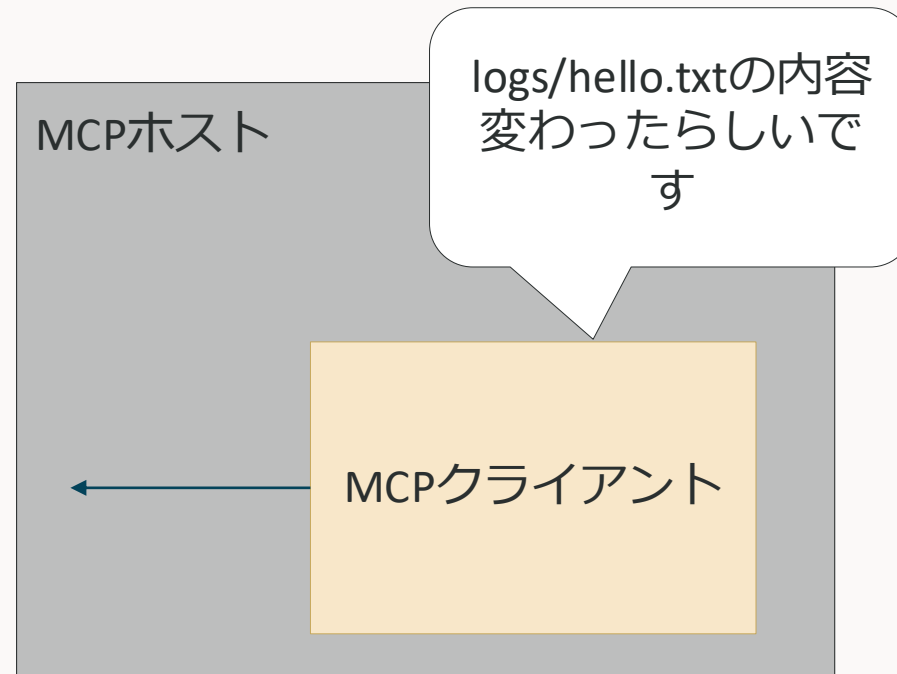
Resources

利用の流れ3

サブスクライブしたリソースに変更があった時



LLM



コールバック関数でMCPクライアントから通知を受け取る

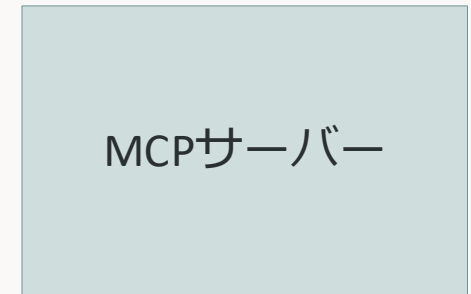
Resources

利用の流れ3

サブスクライブしたリソースに変更があった時



LLM



MCPクライアントを呼び出してリソースの内容を確認する

Resources 利用の流れ3

サブスクライブしたリソースに変更があった時



LLM



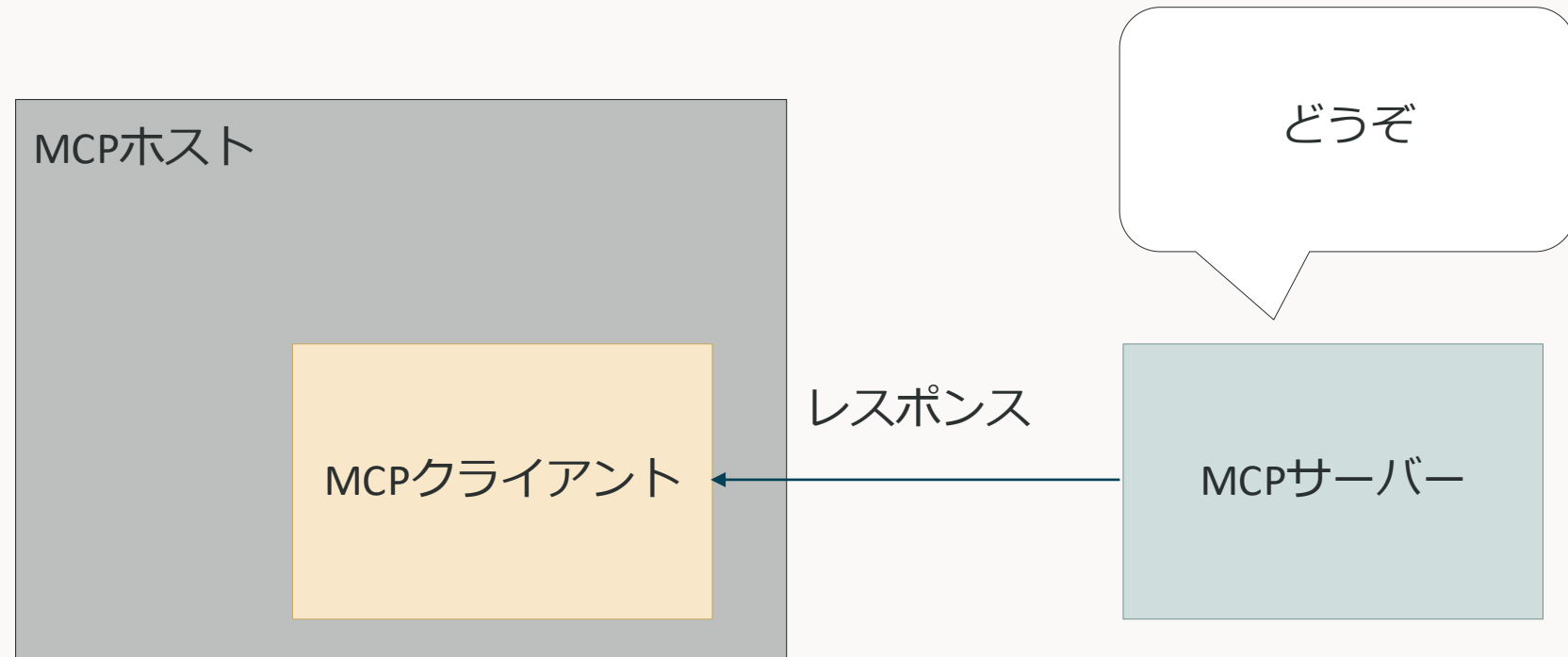
Resources

利用の流れ3

サブスクライブしたリソースに変更があった時



LLM



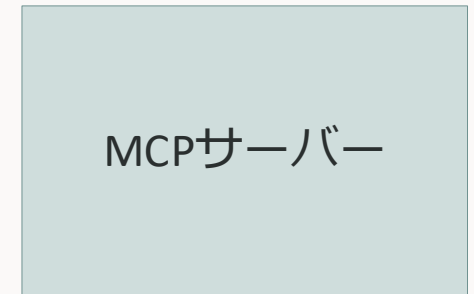
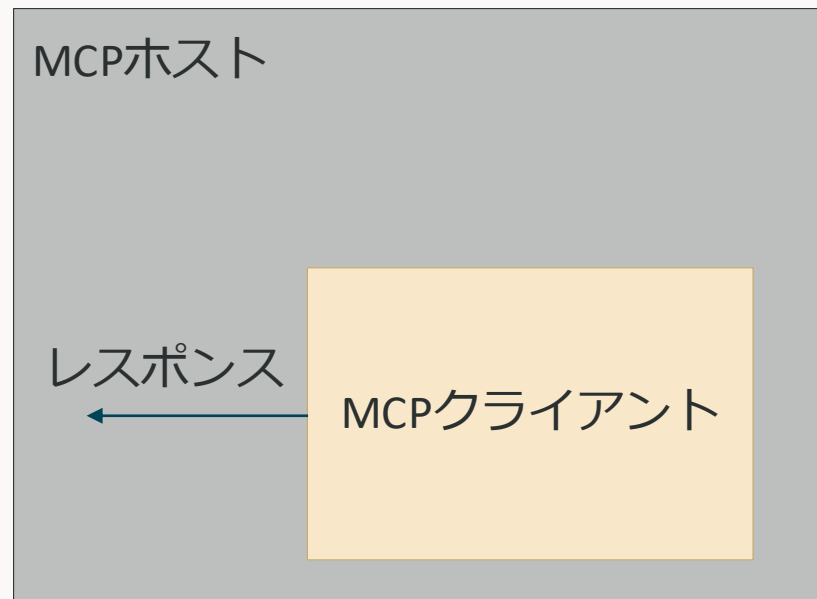
Resources

利用の流れ3

サブスクライブしたリソースに変更があった時
基本的には裏側で勝手に行われるので、ユーザーが気づくことはない



LLM



Resources

注意点

- サーバー起点の通信が発生する（これが純粋なHTTPではMCPを使えない理由）
- Notificationという通信（メソッド）は返信する必要がない（返信しようがない）
 - JSON-RPCではリクエストIDを見て適切なレスポンスを返すが、NotificationにはリクエストIDがないため
 - 一方的に送りつけるだけで良い情報に使われる
 - 今回のリソースのサブスクライブも返信が不要（返信する意味がない）
- サブスクライブしていても、変更の通知と同時に内容が送られてくるわけではない
 - 内容が知りたければresources/readを呼び出す

Prompts

LLMにTools, Resourcesの使い方を教える

二つのメソッドがある

- prompts/list
 - プロンプトのリストを提供する
- prompts/get
 - プロンプトの詳細を提供する

Prompts

なんのための機能か

有用なToolsやResourcesを実装しても、以下のような問題がある

- MCPサーバーに接続しても、何をどう聞けば利用できるかがよくわからない
- うまくToolやResourceを使えるようなプロンプトを毎回書くのは大変
- ベストな使い方はサーバー開発者がよく知っているはずだが、利用者（クライアント）はそれを知らない



そのMCPサーバーの機能をうまく利用できるプロンプトをサーバー開発者側が用意して、提供する仕組み

必要な引数を入れるだけ

プルリクエストの説明をして欲しいな...



/git

gh-pr-description

スラッシュコマンドでプロンプトを呼び出し

Prompts

利用の流れ1

MCPサーバーにプロンプトのリストを要求する



LLM



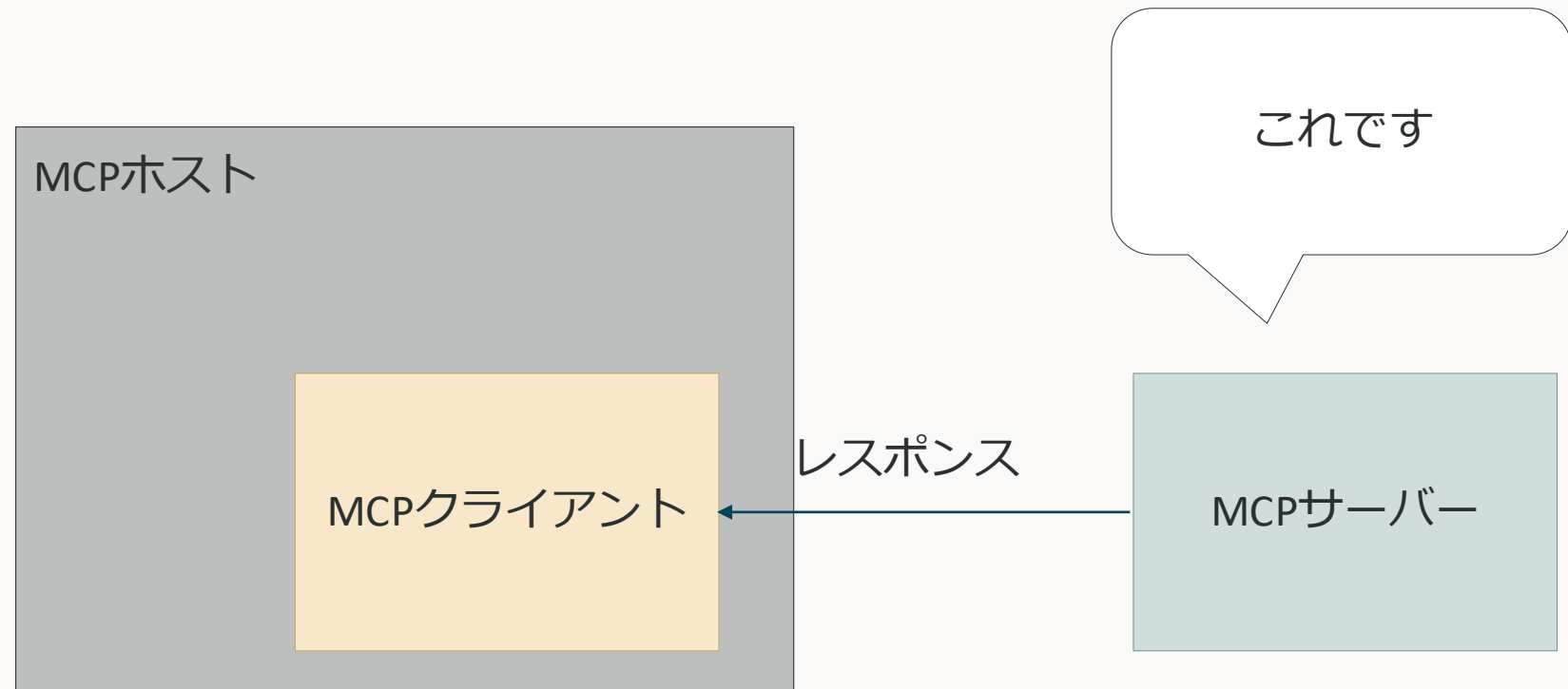
Prompts

利用の流れ1

MCPサーバーにプロンプトのリストを要求する



LLM



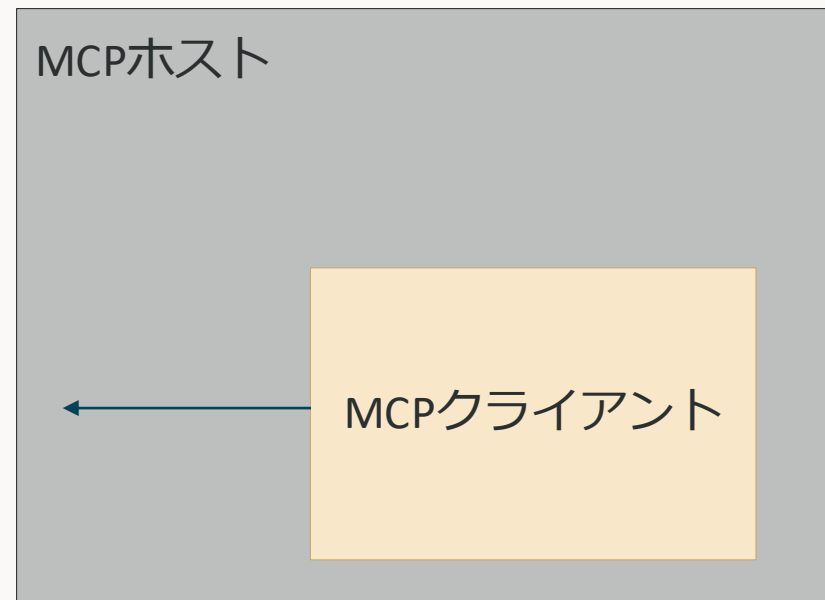
Prompts

利用の流れ1

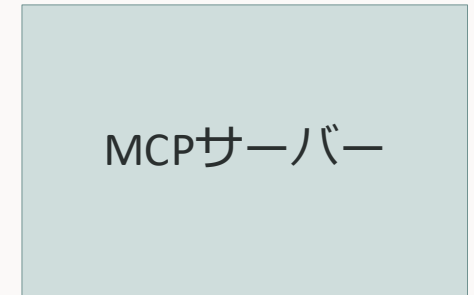
MCPサーバーにプロンプトのリストを要求する



LLM



MCPホストにプロンプトリストを渡す



MCPサーバー

Prompts

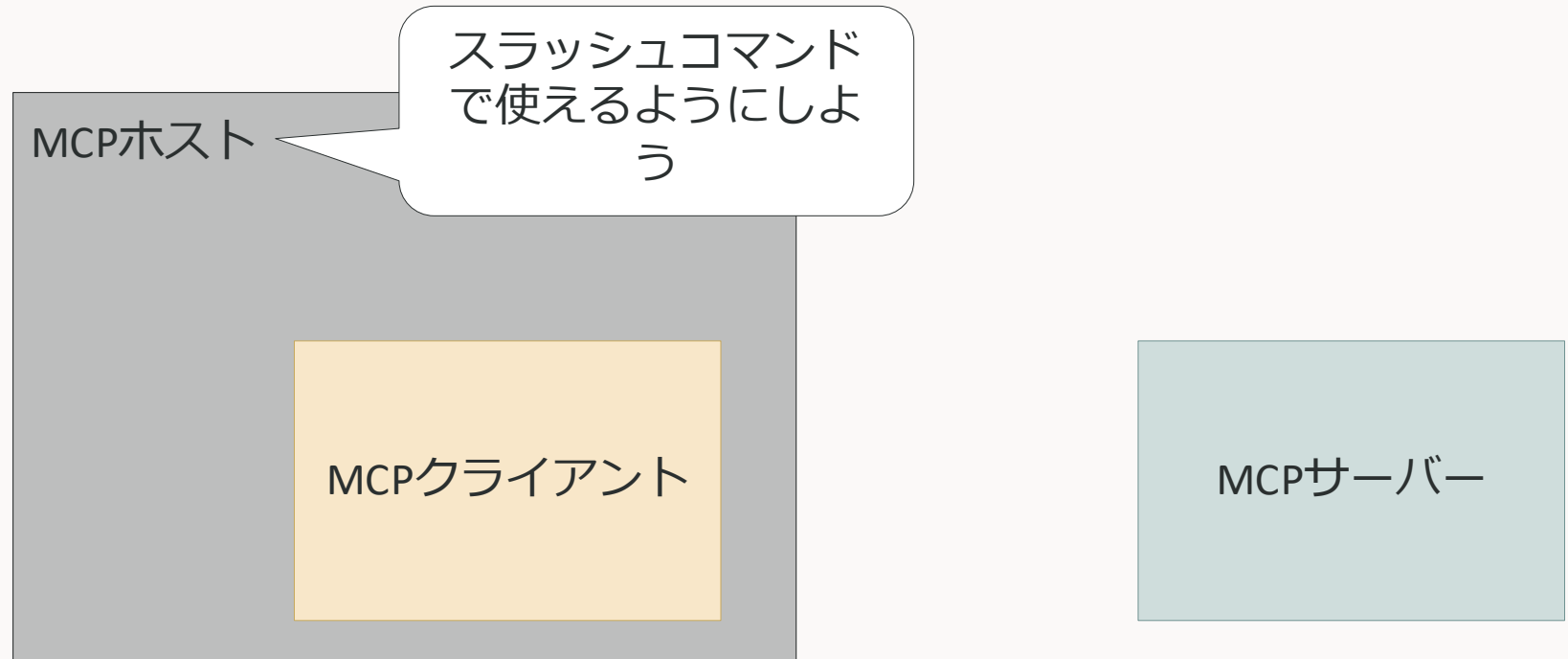
利用の流れ1

MCPサーバーにプロンプトのリストを要求する

※ プロンプトはクライアント（ユーザー）が利用開始するので、LLMにプロンプトのリストを渡す必要はない



LLM

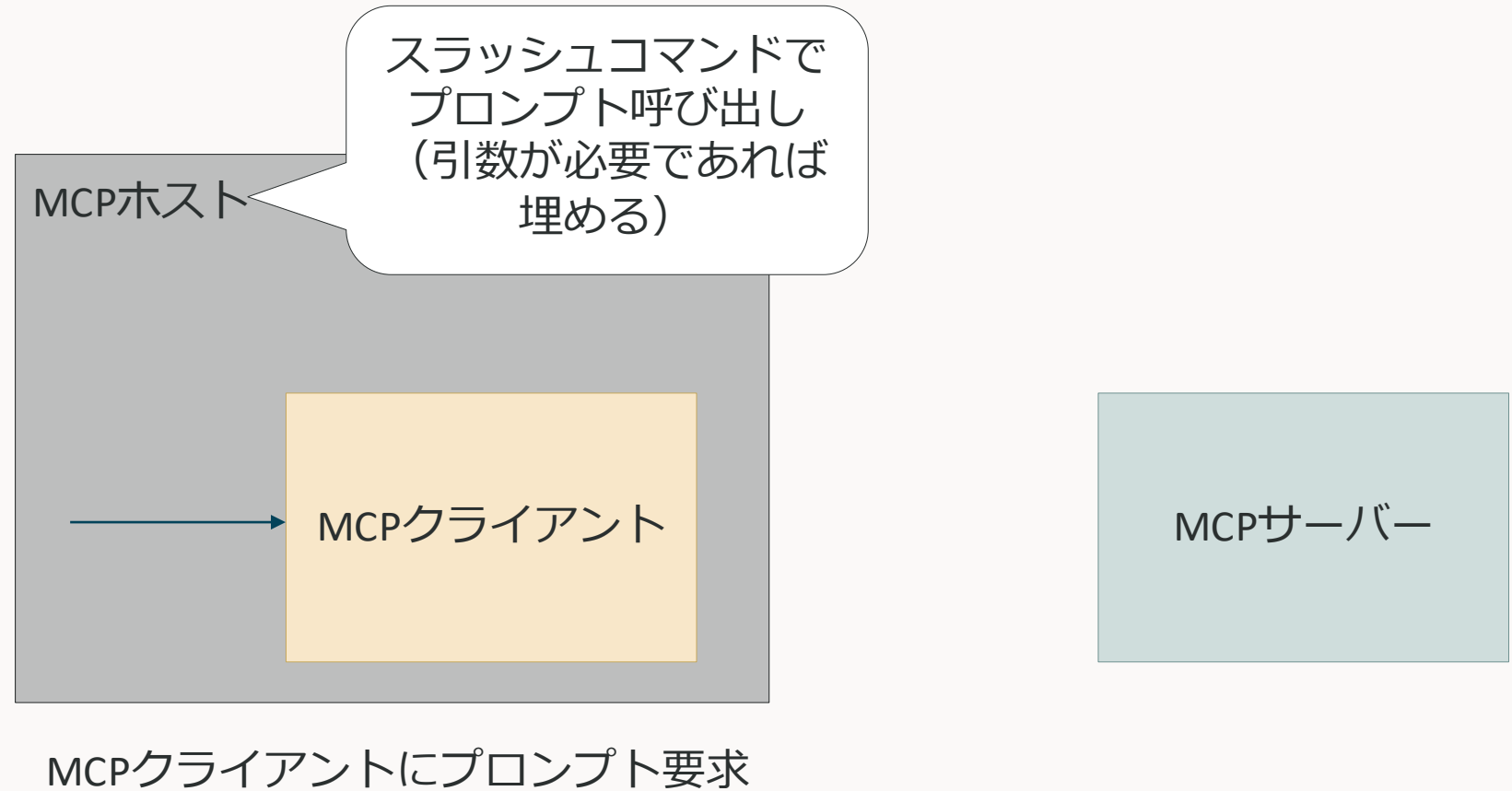


Prompts 利用の流れ2

ユーザーがスラッシュコマンドなどでプロンプトを利用する



LLM

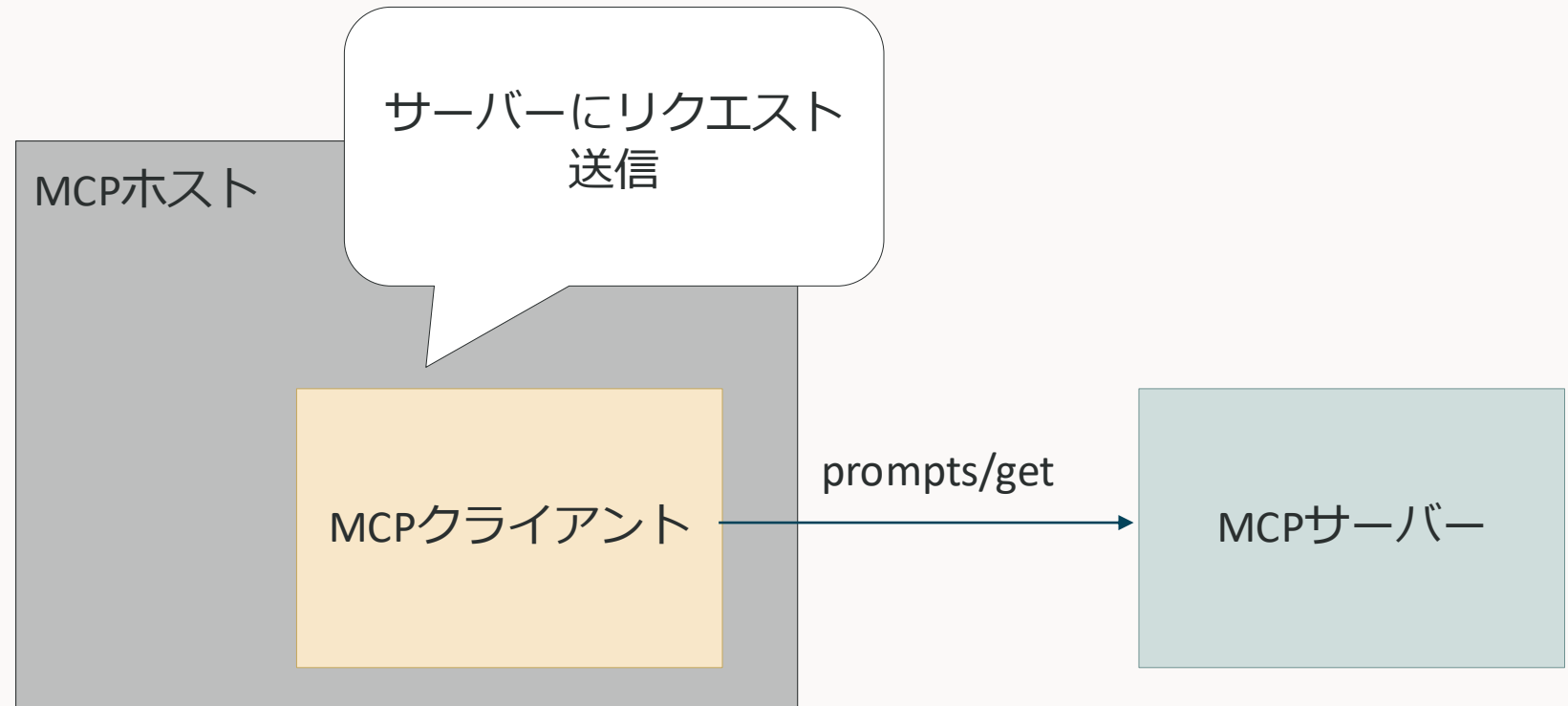


Prompts 利用の流れ2

ユーザーがスラッシュコマンドなどでプロンプトを利用する



LLM



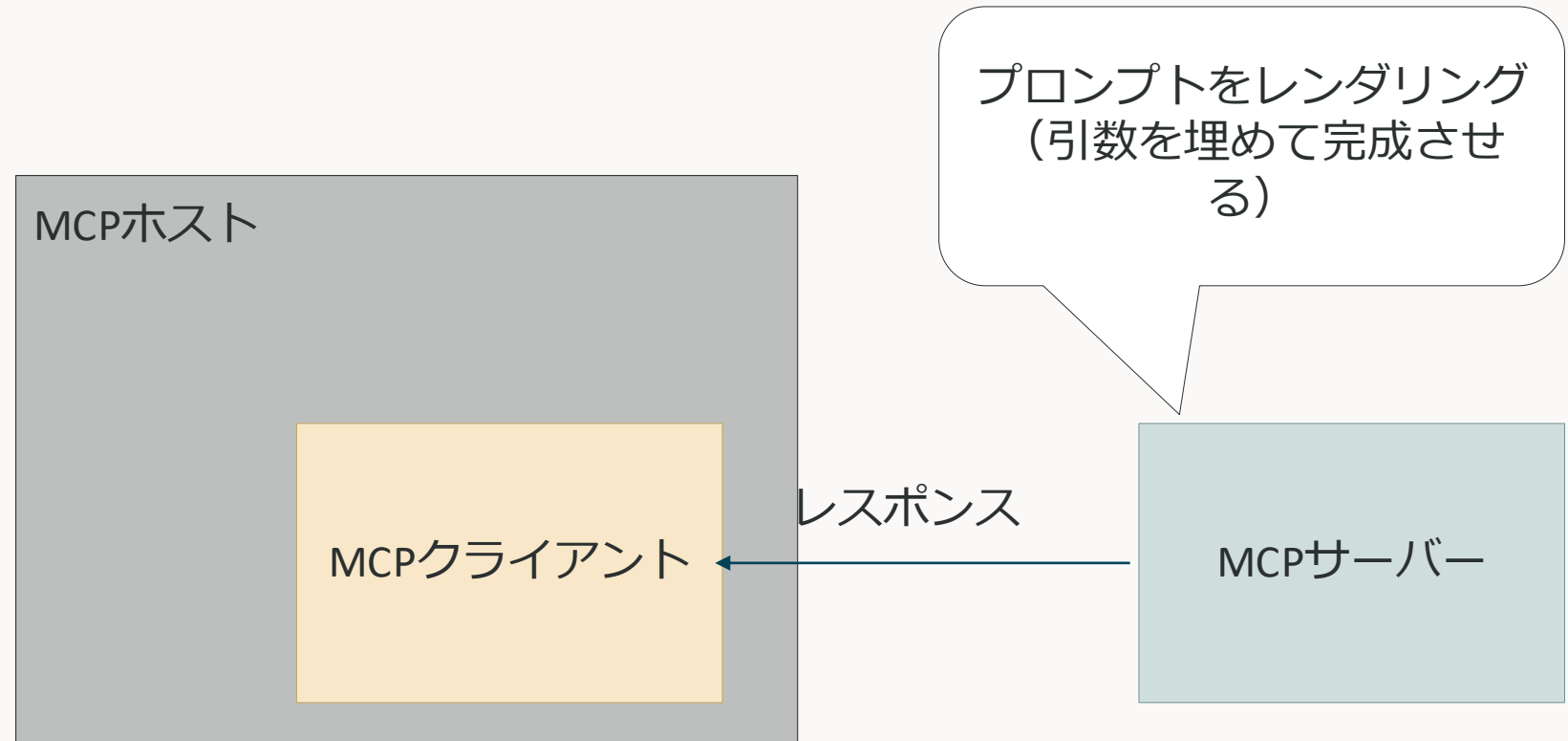
Prompts

利用の流れ2

ユーザーがスラッシュコマンドなどでプロンプトを利用する

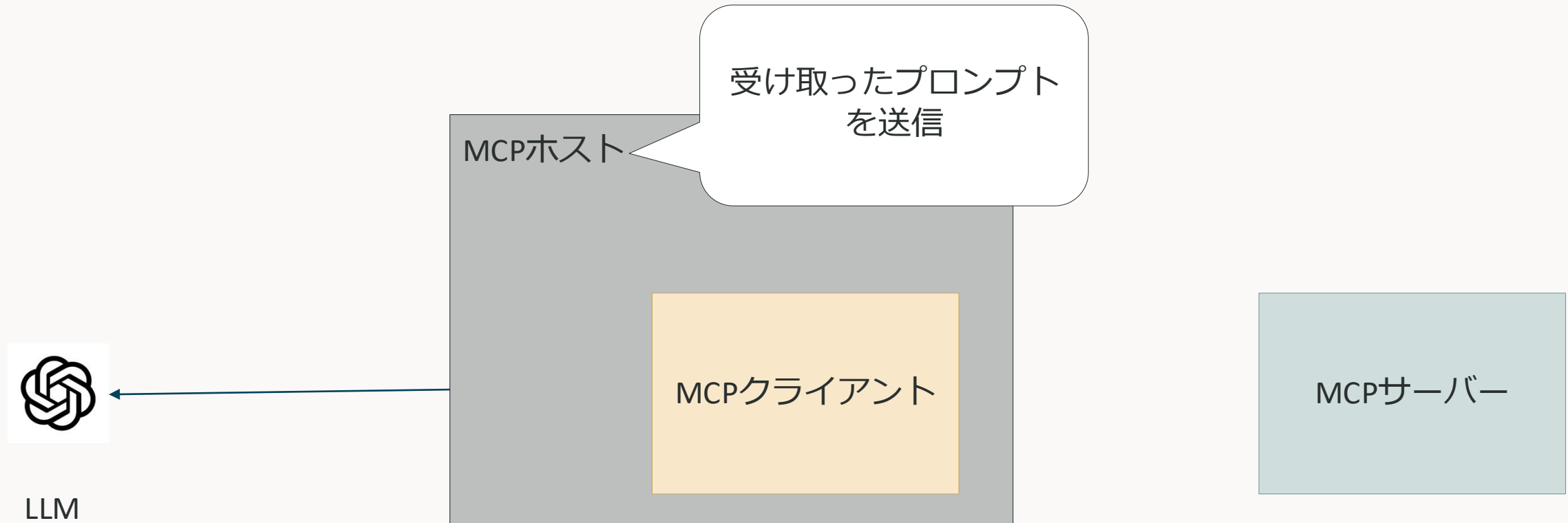


LLM



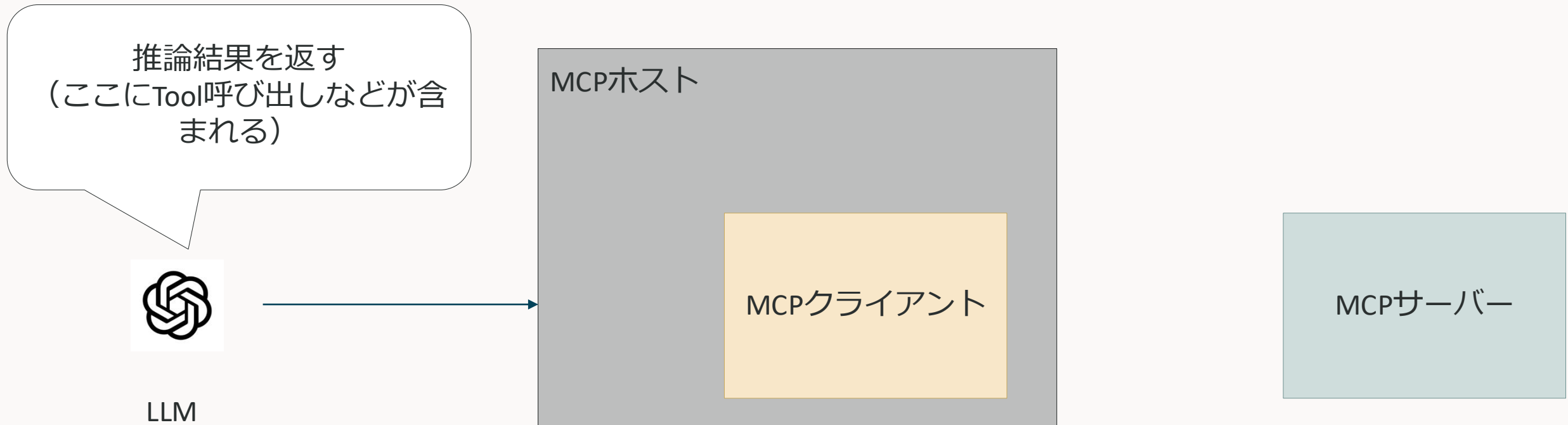
Prompts 利用の流れ2

ユーザーがスラッシュコマンドなどでプロンプトを利用する



Prompts 利用の流れ2

ユーザーがスラッシュコマンドなどでプロンプトを利用する

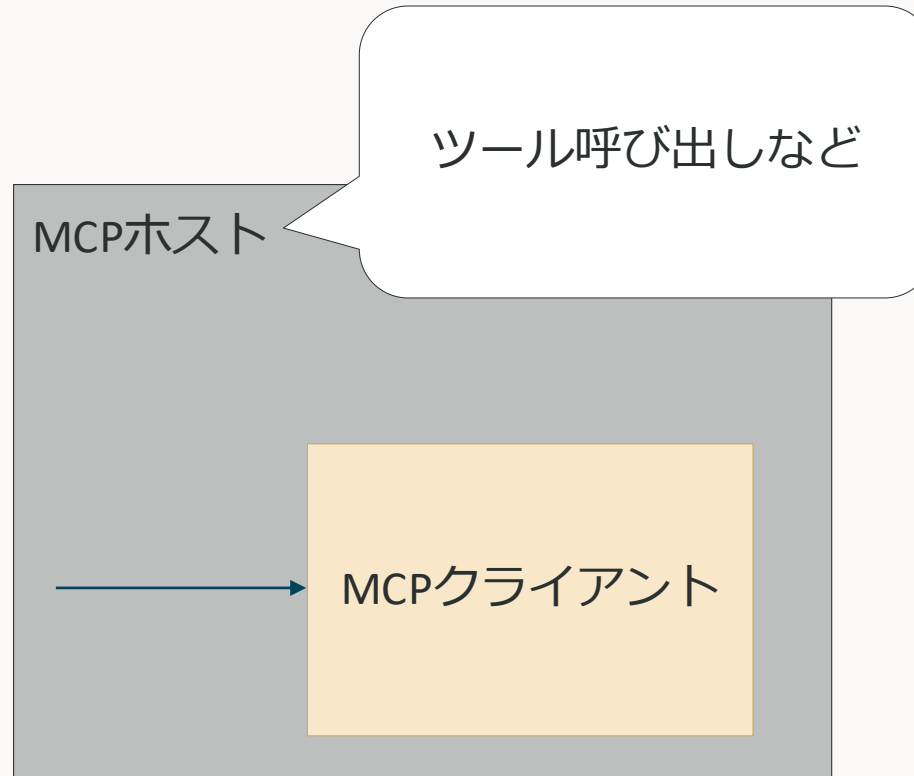


Prompts 利用の流れ2

ユーザーがスラッシュコマンドなどでプロンプトを利用する



LLM



ここまでのまとめ

- Tools
 - LLMがアクティブに呼び出せる関数
 - データベースへの書き込み、外部APIの呼び出し、ファイル変更など
- Resources
 - LLMに追加のコンテキストを与える機能（RAGのためのデータ供給源を提供）
 - サーバーにあるファイルの内容
 - データベースのデータ
 - ドキュメント など
- Prompts
 - LLMがToolsやResourcesを上手く扱えるようにするためのプロンプトを提供する機能
 - ToolsやResourcesの呼び出しをユースケース単位でテンプレート化しておく
 - Resources Aの内容を読んで、Tool A, Tool Bを呼び出す など

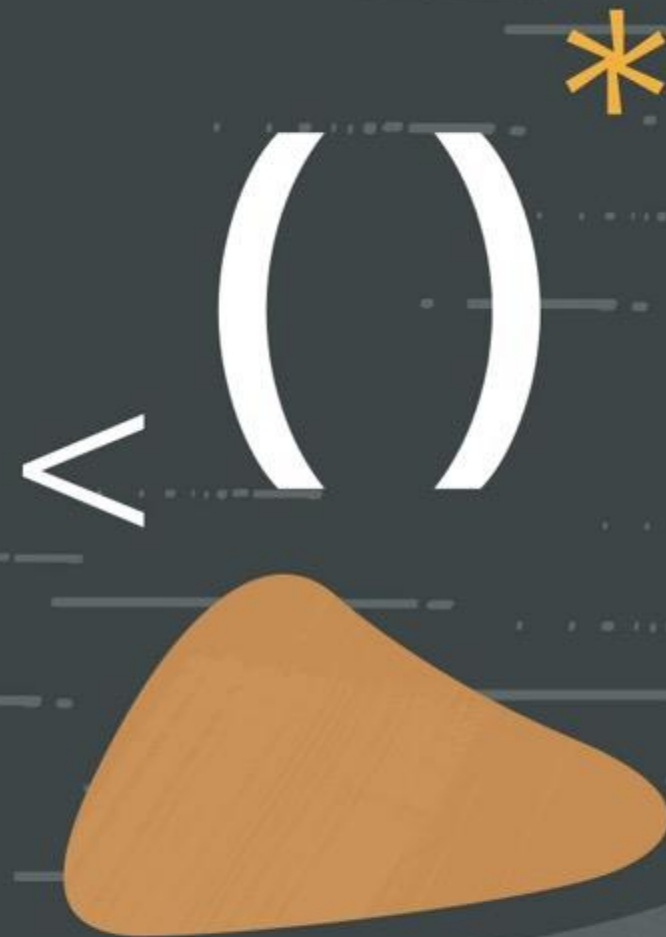
リモートMCPサーバーの場合は考えることが増える

ローカルMCPサーバーの場合はMCPクライアントのサブプロセスとしてMCPサーバーを起動し、標準入出力でやり取りするため非常にシンプルに扱うことができる

リモートMCPサーバーの場合はクライアント数が増えたり、標準入出力が使えないために以下のようなことを考える必要が出てくる

- スケーラビリティ
- MCPサーバーの認可
- Observability

MCPサーバーの スケーラビリティ



MCPのトランスポート層

標準入出力

- MCPクライアントとMCPサーバーを同じマシン上で実行する場合に使用する
- 基本的に認可などが不要なので非常に手軽に利用できる
- MCPクライアントのサブプロセスとしてMCPサーバーを起動する



SSE or Streamable HTTP

- リモートのMCPサーバーを利用する場合に使用する
- MCPは、サーバー起点でクライアントに通信を開始する場合がある
- しかし通常のHTTPでは、常にクライアント起点の通信しかできない
- HTTPの拡張仕様で、サーバー起点の通信もできるようにしたものがStreamable HTTP



リモートMCPサーバーの場合はこっち

Server Sent Events (SSE)

サーバーからクライアントへ一方向のPush通信を行うための仕組み

MCPのバージョン2024-11-05までで、リモートMCPサーバーのトランスポート層で使われていた

- HTTPでは1 リクエストに対して 1 レスポンスが原則で、都度コネクションを閉じる
- SSEでは、レスポンスを分割することでコネクションを閉じないようにする
- コネクションが閉じないので、サーバーからPush通信が行える
- クライアントのリクエストは完了しているので、クライアントからは新たにコネクションを作成しないと送れない

MCPでの使われ方

- サーバーは二つのエンドポイントを用意
 - SSE用のエンドポイント（このコネクションでクライアントに返答する）
 - クライアントのメッセージ受け取りエンドポイント
- クライアントは以下の手順
 - 最初にSSE用のエンドポイントに接続し、コネクションを張る
 - リクエストはメッセージ受け取りエンドポイントに送る
 - レスポンスはSSEコネクションから受け取る

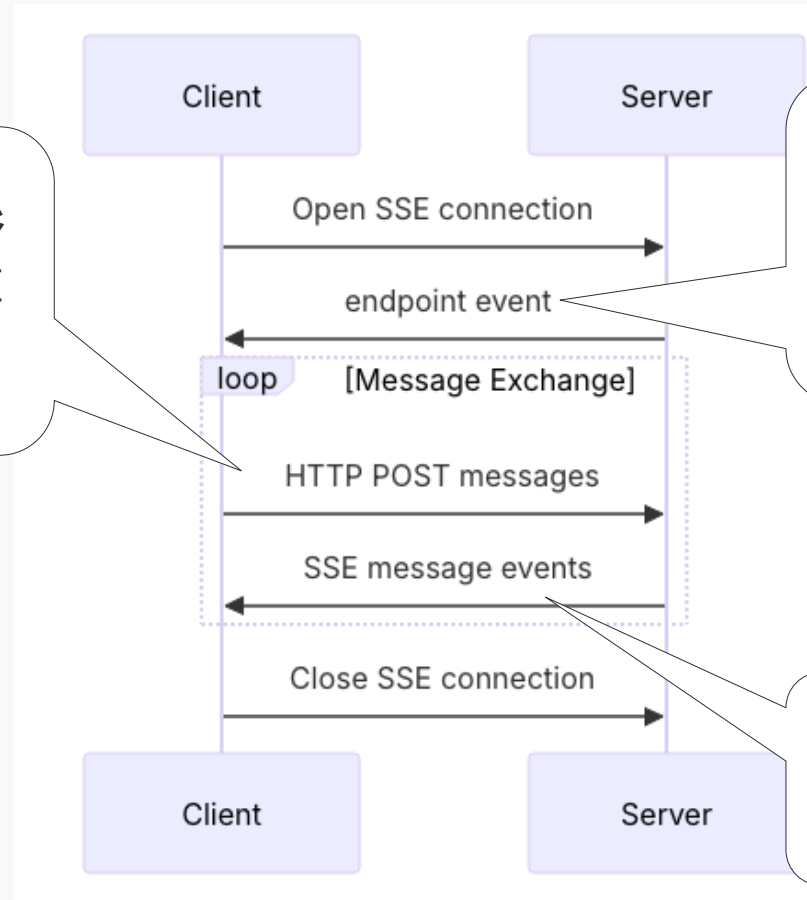
Server Sent Event のMCPでの使われ方

教えてもらったメッセージ
受け取りエンドポイントに
メッセージを送信

メッセージ受け取りエンド
ポイントを教える
(HTTP的にはこのレスポ
ンスは一部)

最初に確立したSSEコネク
ションを使ってレスポンス

※リクエストとレスポンスでコネ
クションが異なるため、JSON RPC
メッセージのIDで結びつける



Server Sent Eventsの問題点

- 接続が切れると状態が失われる
 - SSEでは、最初にかいたコネクションで通信を判断する
 - 接続が切れて新たにコネクションが張られると、以前のコネクションとの紐付けができず、状態（リソースのサブスクライブなど）が失われる
- スケーリングに弱い
 - 接続してきたクライアント数分のSSE接続を常時確立する必要がある
 - 接続数が増えるほど当然パフォーマンスは低下する
 - FaaSなどでは使いにくい（SSEコネクションを維持できないため）

Server Sent Eventsの問題点

- 接続が切れると状態が失われる
 - SSEでは、最初にかいたコネクションで通信を判断する
 - 接続が切れて新たにコネクションが張られると、以前のコネクションとの紐付けができず、状態（リソースのサブスクライブなど）が失われる
- スケーリングに弱い
 - 接続してきたクライアント数分のSSE接続を常時確立する必要がある
 - 接続数が増えるほど当然パフォーマンスは低下する
 - FaaSなどでは使いにくい（SSEコネクションを維持できないため）

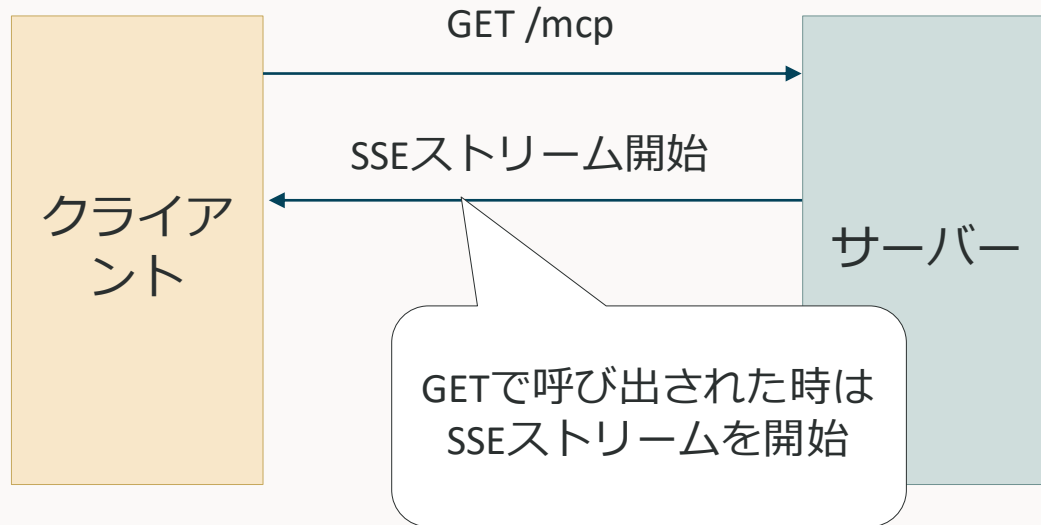


新旧コネクションの紐付けを行って、必要な時だけSSEを使えばいいのでは...?

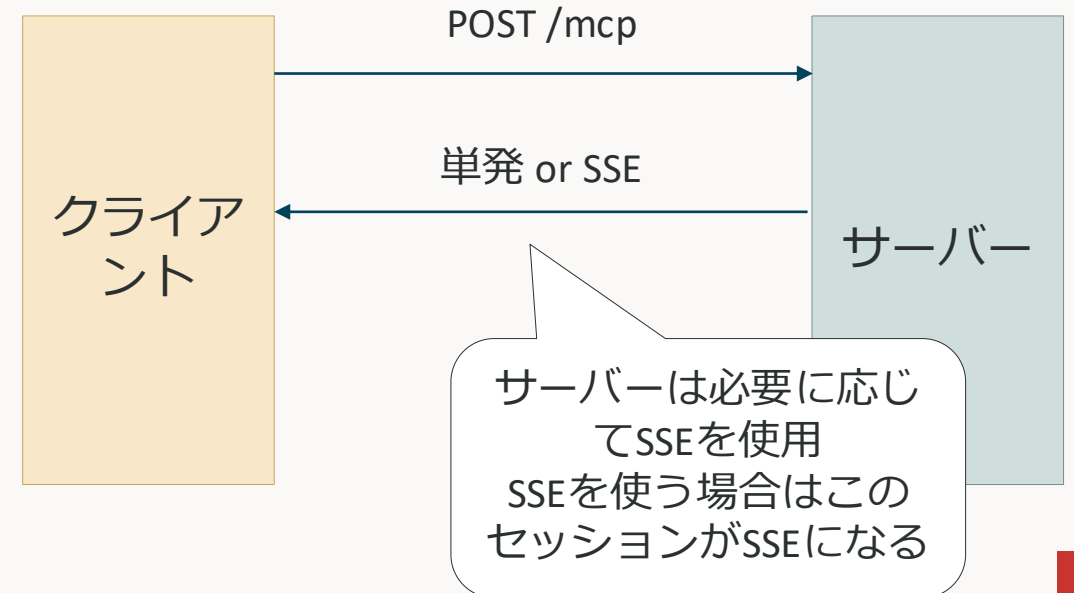
Streamable HTTP

- セッションIDを使ってセッションを管理
- 必要な時だけSSE接続を確立する（単一のHTTPエンドポイントで完結する）
- エンドポイントは一つ（/mcp）
 - GETするとSSEコネクションを作成
 - POSTでメッセージの送信

GET



POST

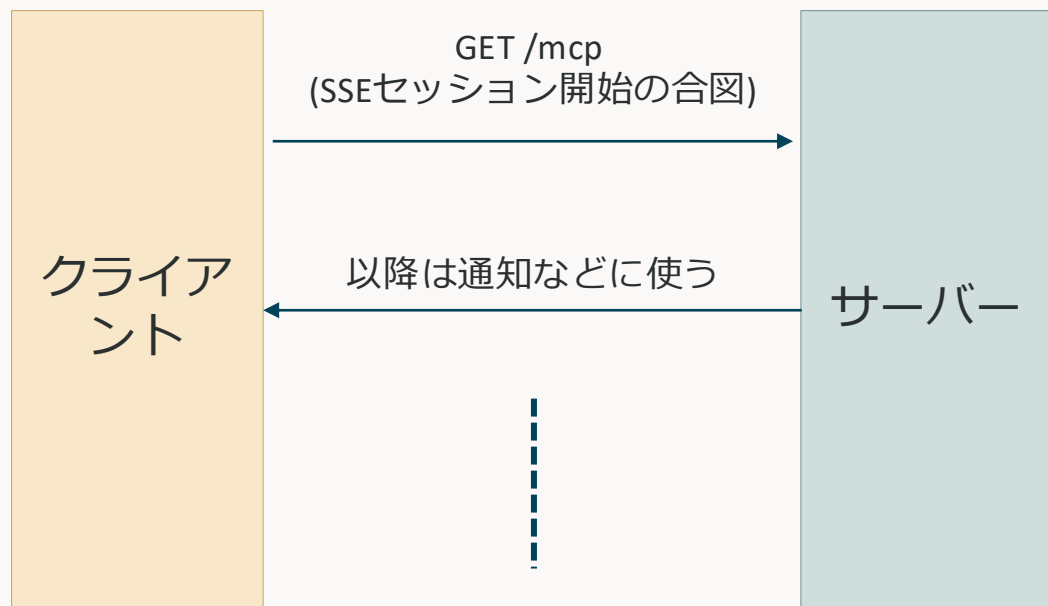


Streamable HTTPにおけるGETとPOSTの違い

特定のリクエストに紐づくか否かで異なる

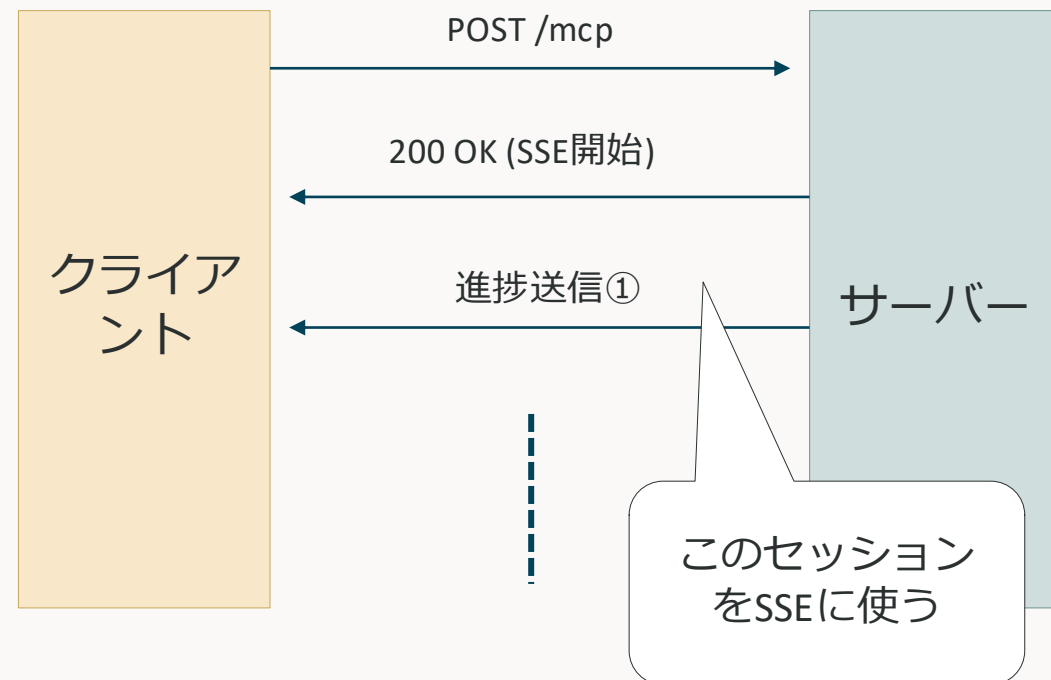
特定のリクエストに紐付かない場合（GET）

- リソースのサブスクライブなど
- あらかじめSSEセッションを開く必要があるため、GETであらかじめ開いておく



特定のリクエストに紐づく場合（POST）

- 長時間かかるツールの進捗など
- 紐づいているリクエストをSSEで返す
- そのセッションをSSEにする

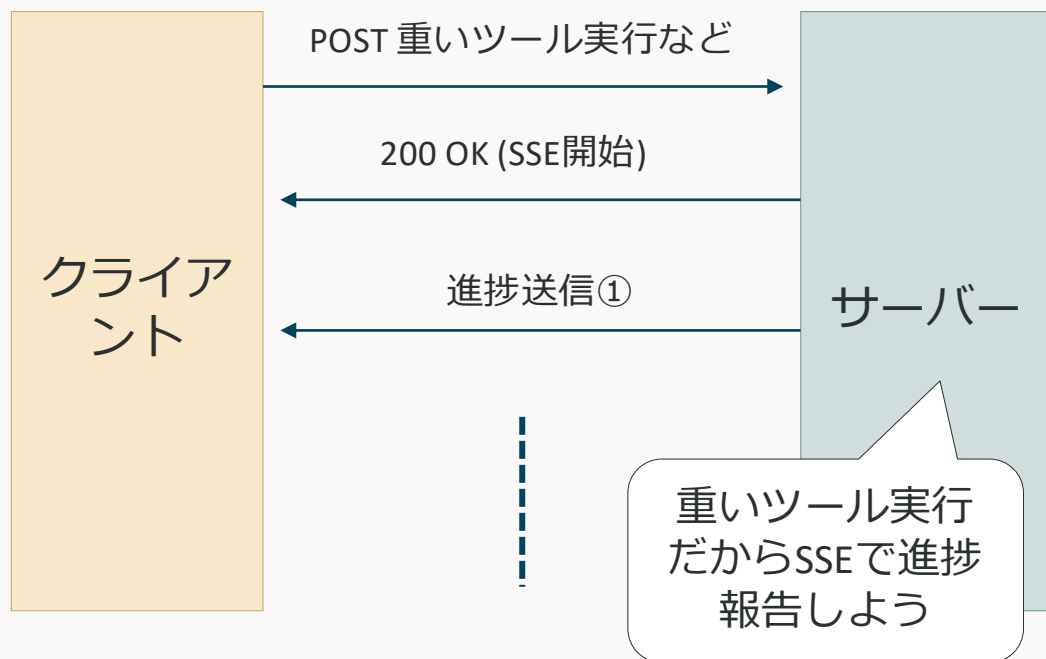


Streamable HTTPでのSSE接続の使い分け

サーバー起点のリクエストや通知が必要か否か

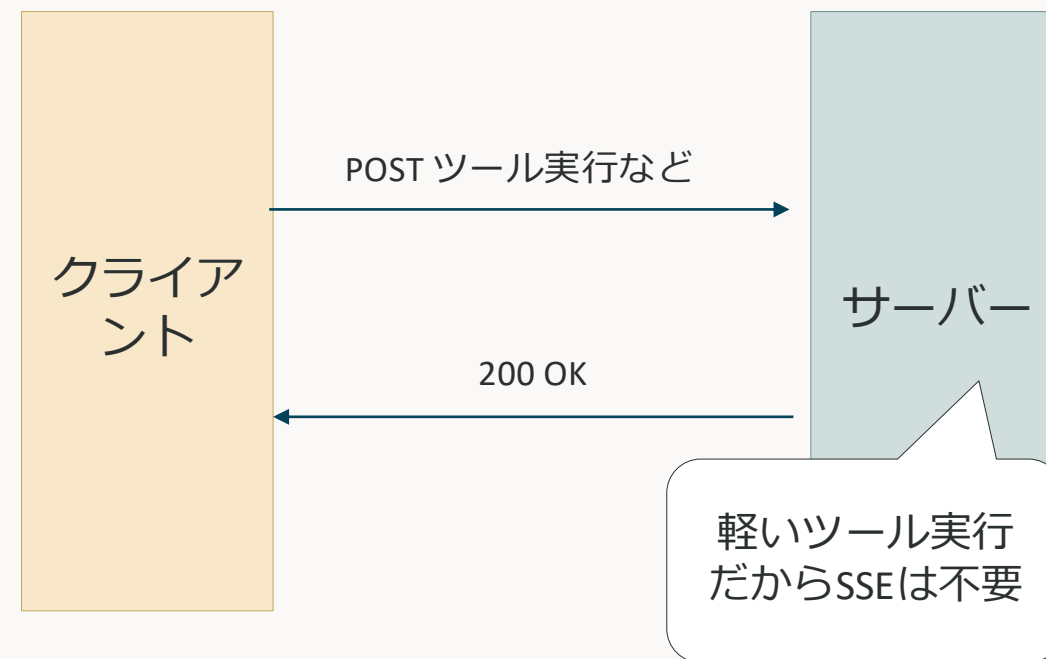
SSEが必要な場合

- 長時間かかるツールの進捗
- リソースのサブスクライブなど
- サーバー起点のリクエストや通知が必要な場合



通常のHTTPで十分な場合

- すぐ終わるツール実行
- プロンプトのレンダリングなど
- サーバー起点のリクエストや通知が不要な場合



Streamable HTTPのセッション管理

以下のような情報をステートとして管理する必要がある

- セッションID
- MCPの初期化フェーズでやり取りした情報
 - プロトコルバージョン
 - Capabilities
- MCPの動作フェーズでやり取りした情報
 - サブスクライブ中のリソース
 - サーバーからクライアントへの通知用SSE接続
 - 進行中のリクエストの対応関係

MCPサーバーを水平スケールする際に発生する問題

MCPサーバーを水平スケールすると、リクエストがどのPodに割り振られるかはロードバランサに委ねられる
この場合、以下の解決策が思いつく

- スティックセッションを使って同じPodにルーティングする
- ステートをすべて外部に外だしして、どのPodにルーティングされても問題ないようにする

しかしこれらの解決策にはそれぞれ問題がある

- スティックセッション
 - MCPではセッションをMcp-Session-Idヘッダーで管理するが、LBがこのヘッダーに応じてルーティングする必要がある
 - 多くのMCPクライアントではCookieが利用できないので、Cookieベースのルーティングができない
- ステートを全て外部に外だし
 - 現状、セッション情報を外部に外だしする機能があるフレームワークは確認できなかった
 - 全て自前で実装する必要がある



現状、水平スケールするMCPサーバーは
ステートレスに作るのが最も簡単

MCPサーバーをステートレスに構成するとできること・できないこと

基本的にはMCPはステートレスのため、大体の機能は利用できる

利用できる機能

- ツール呼び出し
- リソース要求
- プロンプト要求

利用できない機能

- Sampling（MCPサーバーがクライアント背後のLLMを呼び出す機能）
- Elicitation（MCPサーバーがユーザーに追加で情報を求める機能）
- リソースのサブスクライブ（リソースに変更があった際に、クライアントに通知する機能）

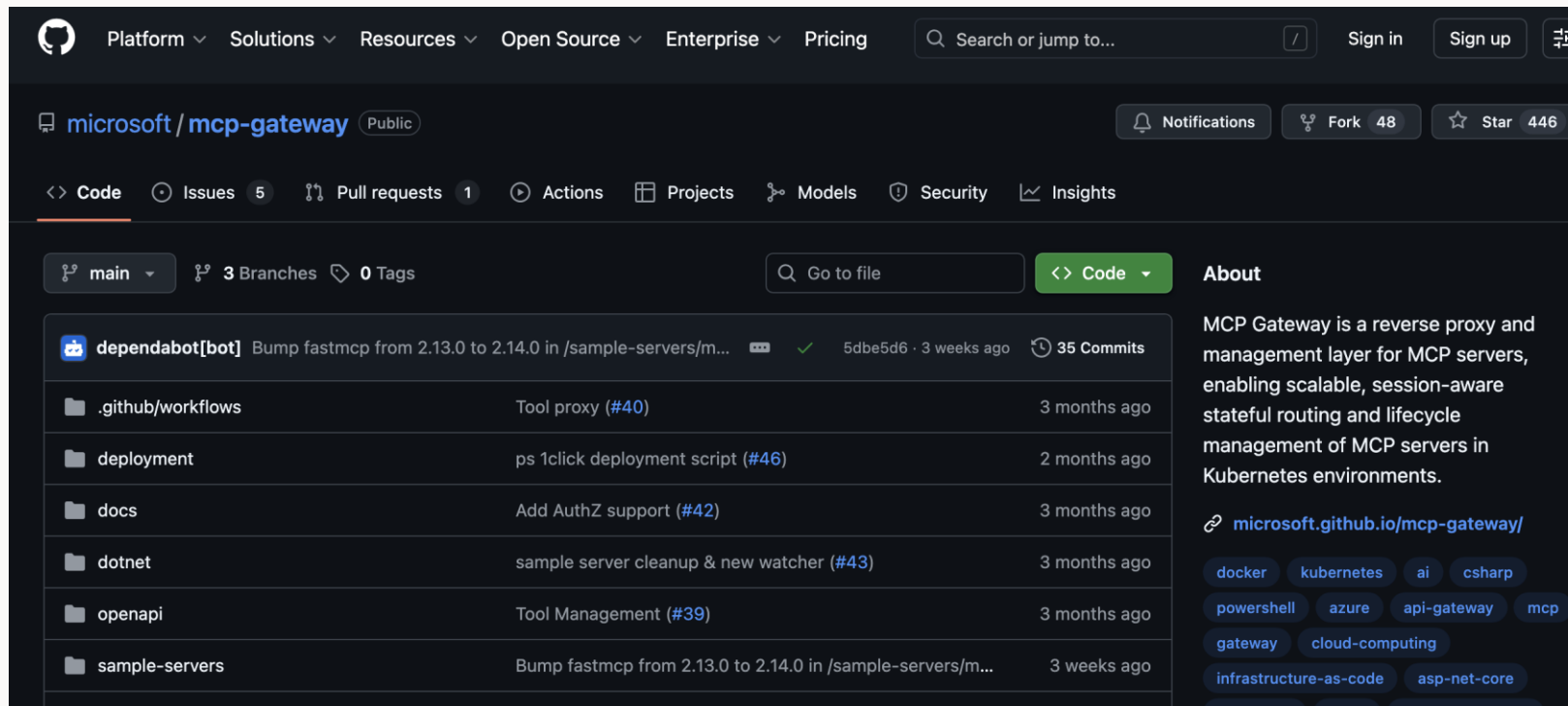
これらの機能はサーバーからクライアントにリクエストする必要があるため、ステートレスでは利用できない

MCPゲートウェイという選択肢もある

MCPリクエストを適切なPodにルーティングしてくれる

Kubernetes上にMcp-Session-Idヘッダーを解釈できるLBを立てるイメージ

他にもいろいろできそう（MCPサーバーのライフサイクル管理、アクセス制御、Observabilityなど）



<https://github.com/microsoft/mcp-gateway>

まとめ

- リモートMCPサーバーの場合、MCPバージョンによって二つの選択肢がある
 - Server Sent Events
 - Streamable HTTP
- Server Sent Events
 - 通信の最初にSSE用セッションを用意し、サーバーのレスポンスはここから受け取る
 - クライアントのリクエストは都度通常のHTTPで行う
 - 通信の切断に弱い
- Streamable HTTP
 - 普段は通常のHTTP、必要な時だけSSEを使う方式
 - セッションをIDで管理するため、切断時もそのIDで新旧セッションを紐付けられる
- MCPサーバーの水平スケールについて
 - ステートレスに構成すれば水平スケールできる
 - ステートレスに構成した場合、MCPの一部機能が利用できなくなる
 - ゲートウェイ構成にするのもあり

MCPでの認可



MCPの認可仕様

バージョンごとに異なる

MCPでは、バージョン2025-03-26から認可についての仕様を定めた

- 2025-03-26
 - OAuth 2.1 IETF ドラフト
 - OAuth 2.0 認可サーバーメタデータ(RFC8414)
 - OAuth 2.0 動的クライアント登録プロトコル(RFC7591)
- 2025-06-18
 - 上の三つはそのまま
 - OAuth 2.0 保護リソースメタデータ(RFC9728)
- 2025-11-25 (Latest)
 - 上の四つはそのまま
 - OAuth クライアントIDメタデータドキュメント (IETFドラフト)



ドラフトが多すぎ
ない？ ...

すべてに共通する、OAuth 2.1 IETFドラフトとは

OAuth 2.0との主な違い

- Authorization Code GrantでPKCEが必須に
- リダイレクトURIを完全に文字列一致させないといけなくなった
- URIクエリパラメータでのBearerトークン使用の禁止
- パブリッククライアントのリフレッシュトークンに要件が追加
- Implicit Grant、Resource Owner Password Credentials Grantを削除
- など...

OAuth 2.0 認可サーバーメタデータ(RFC8414)

認可サーバーの情報を渡すメタデータのフォーマットを定義

目的

認可サーバーのメタデータを標準化された形式で公開することで、OAuth2.0クライアントが以下の情報を自動的に取得できるようになる

- 各エンドポイント（トークンエンドポイントなど）のURL
- サーバーがサポートする機能（サポートするクライアント認証方式など）



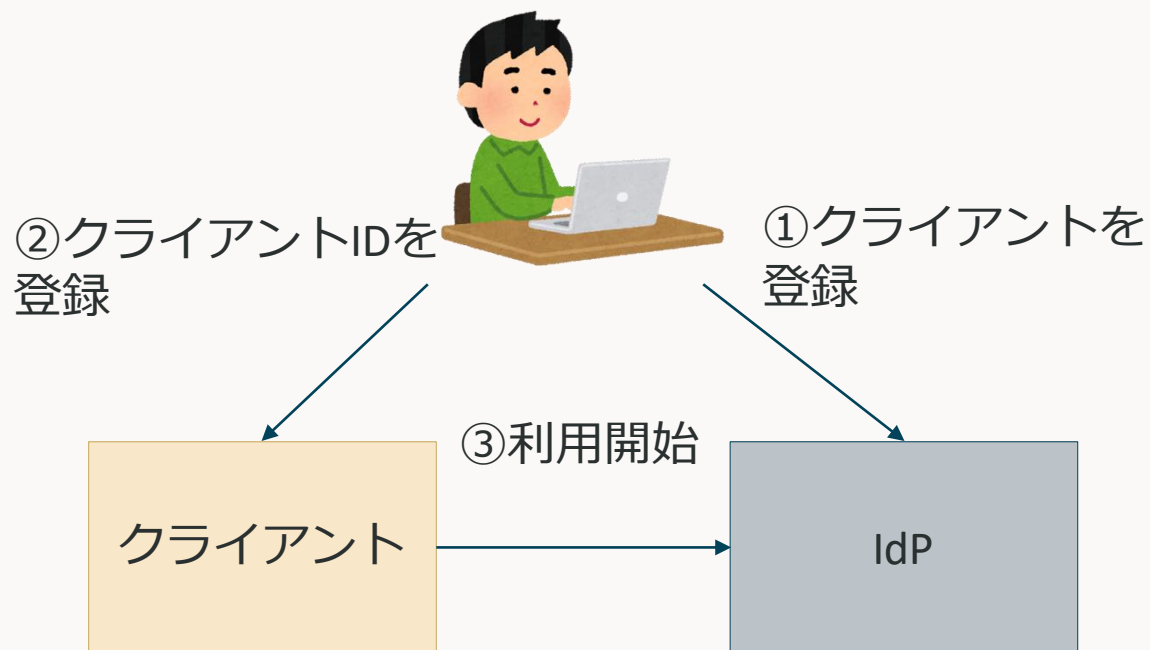
OAuth 2.0 動的クライアント登録プロトコル(RFC7591)

OAuthクライアントを認可サーバーに動的に登録する

目的

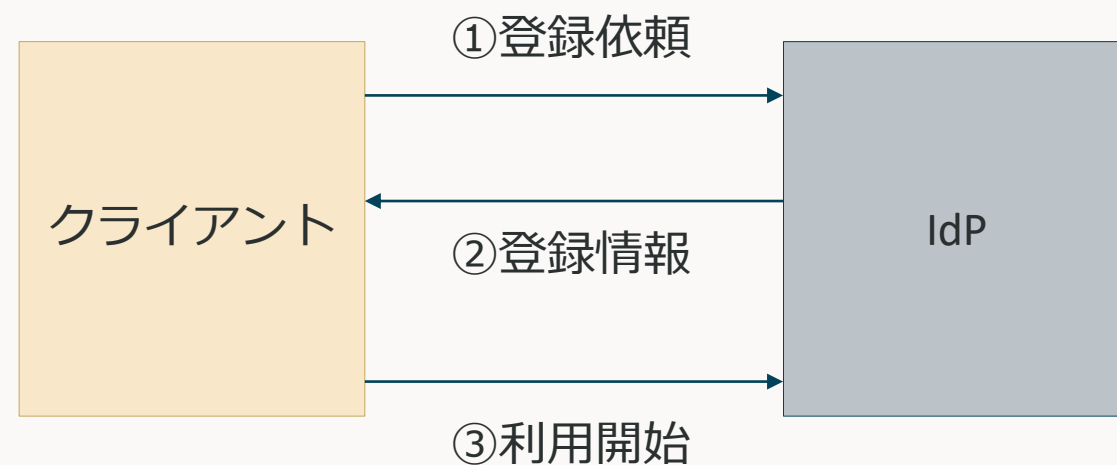
従来は、IdPでクライアントを登録してからでないとクライアントが認可サーバーを利用できなかった
IdPで自動でクライアント登録するための仕組み

従来



動的クライアント登録プロトコル

手動での登録が不要



OAuth 2.0 動的クライアント登録プロトコル(RFC7591)

MCPでなぜ採用されたのか

- 新しいMCPサーバーにシームレスに接続させたい
- クライアントはすべてのMCPサーバーを事前に知ることはできない



その都度クライアント登録が行える、RFC7591は非常に相性が良かった

<https://modelcontextprotocol.io/specification/2025-03-26/basic/authorization#dynamic-client-registration>

OAuth 2.0 動的クライアント登録プロトコル(RFC7591)

問題点

多くのIdPでは以下のような理由でサポートされていない

- オープン登録は誰でも登録できてしまうため、セキュリティ上よくない
- ユースケースがそもそもあんまりない
- 運用が大変
 - 大量のクライアントが登録される可能性
 - 不正利用への対応
 - 登録されたクライアントのライフサイクル管理

など

OAuth 2.0 保護リソースメタデータ(RFC9728)

保護リソース（リソースサーバー）のメタデータに関する仕様

RFC8414と同様、well-known URIでリソースサーバーが公開する

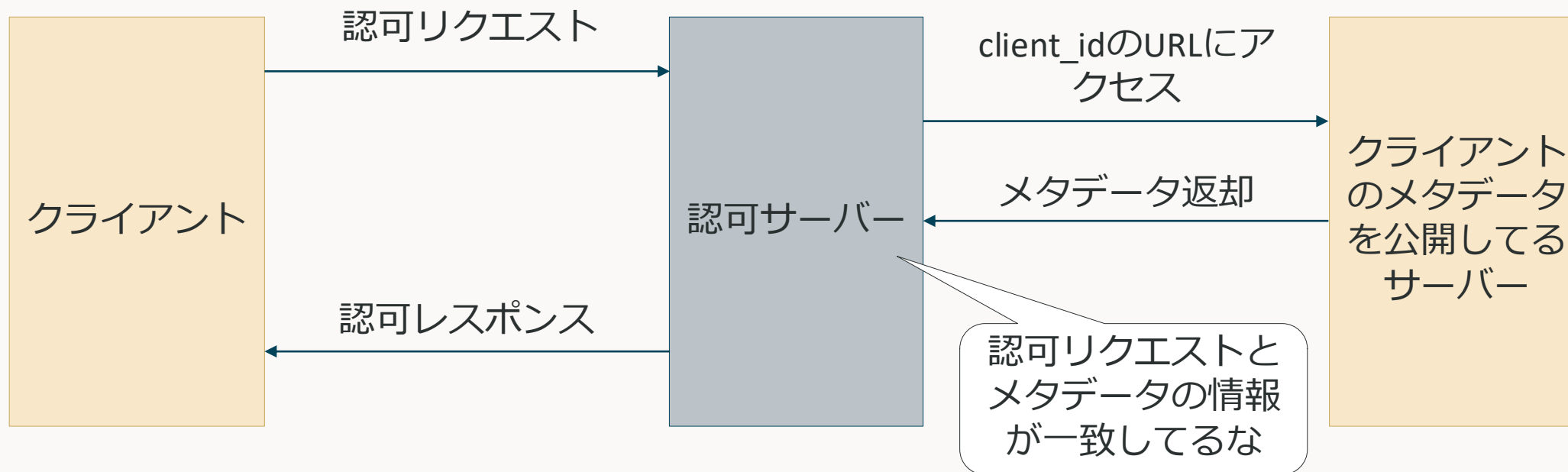
- リソースサーバーを使うのに必要な認可サーバー
 - 必要なスコープの情報
 - トークンの送り方
- などが取得できる



OAuth クライアントIDメタデータドキュメント (IETFドラフト)

クライアントIDに、クライアント情報を公開しているURLを指定する

- クライアントIDに、クライアント情報（メタデータドキュメント）を公開しているURLを指定
- 認可サーバーはそのURLにアクセスし、クライアント情報を確認する
 - 確認するのはclient_idとredirect_uri



OAuth クライアントIDメタデータドキュメント (IETFドラフト)

クライアントIDに、クライアント情報を公開しているURLを指定する

動的クライアント登録では、

- 認可サーバー側でクライアント情報を管理する必要がある
- オープン登録の場合は誰でも登録できてしまう

という問題があった

この方法では、

- クライアントIDに、クライアント情報（メタデータドキュメント）を公開しているURLを指定
- 認可サーバーはそのURLにアクセスし、クライアント情報を確認する



- クライアント情報はクライアント側で管理
- 身元の根拠はドメイン所有権

MCPの認可フロー: 2025-06-18

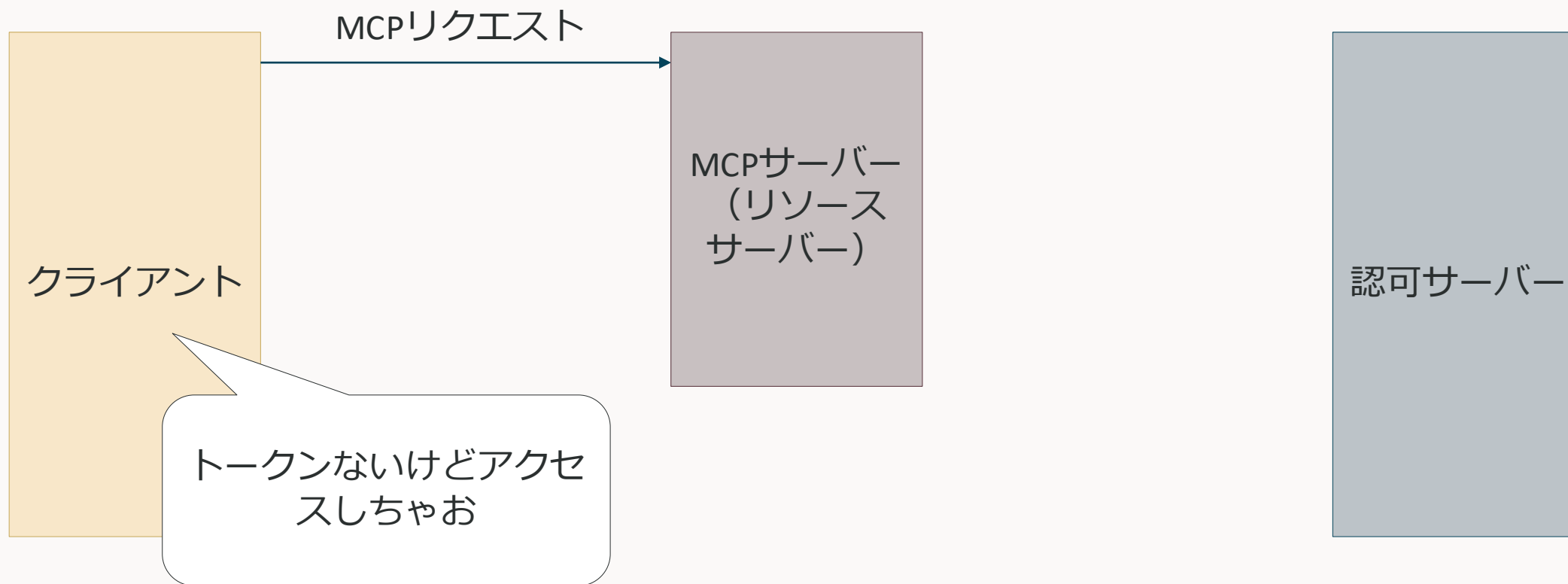
Anthropicの出している代表的なMCPクライアント（ホスト）であるClaude Desktopが2025-11-25に対応していないため、今回は2025-06-18の認可フローを説明

大きくフローを分けると、3つのフェーズからなる

1. 認可サーバー検出フェーズ
2. クライアント登録フェーズ（静的登録によりスキップ可）
3. OAuthのフロー

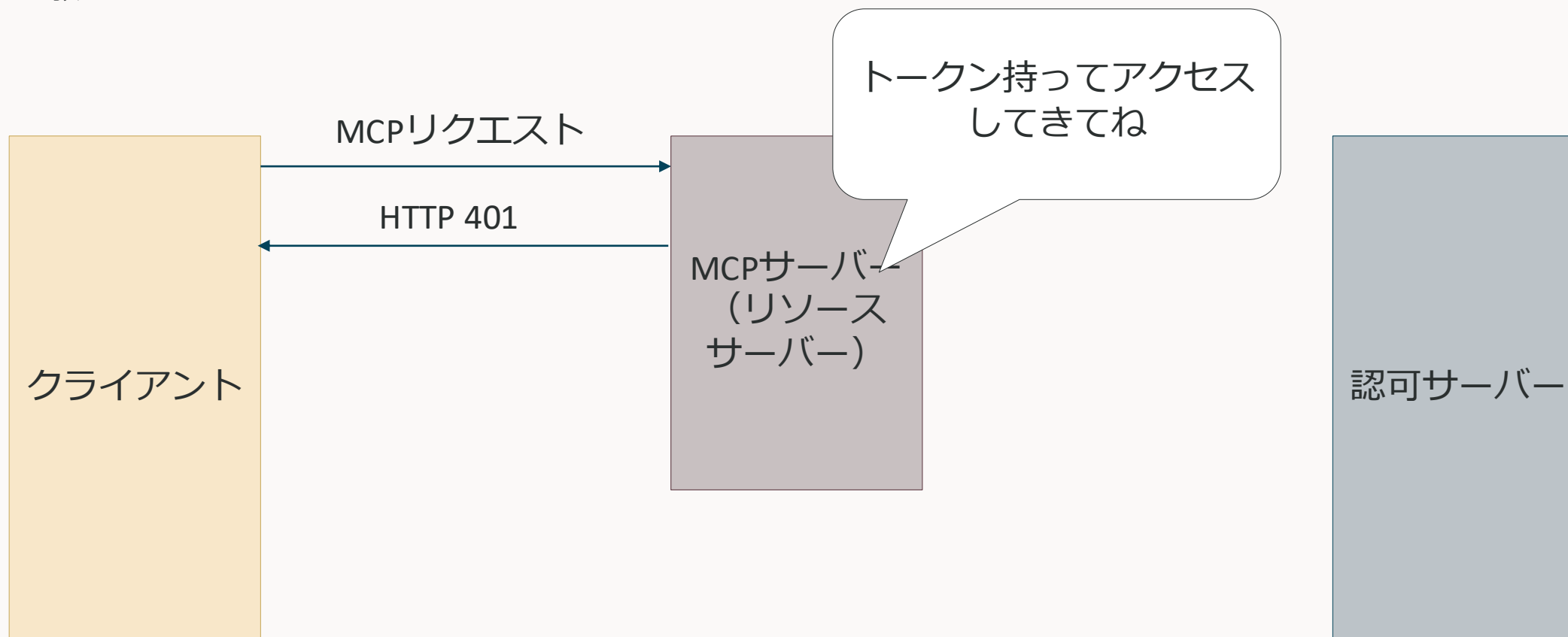
認可サーバー検出フェーズ

MCPクライアントは、接続したいMCPサーバーが要求する認可サーバーの情報を知らないので、それを教えてもらうフェーズ



認可サーバー検出フェーズ

MCPクライアントは、接続したいMCPサーバーが要求する認可サーバーの情報を知らないので、それを教えてもらうフェーズ



認可サーバー検出フェーズ

MCPクライアントは、接続したいMCPサーバーが要求する認可サーバーの情報を知らないので、それを教えてもらうフェーズ

※RFC9728: 保護リソースメタデータ

RFC9728にこのエンドポイントに聞けばわかるって書いてあるな

クライアント

MCPリクエスト

HTTP 401

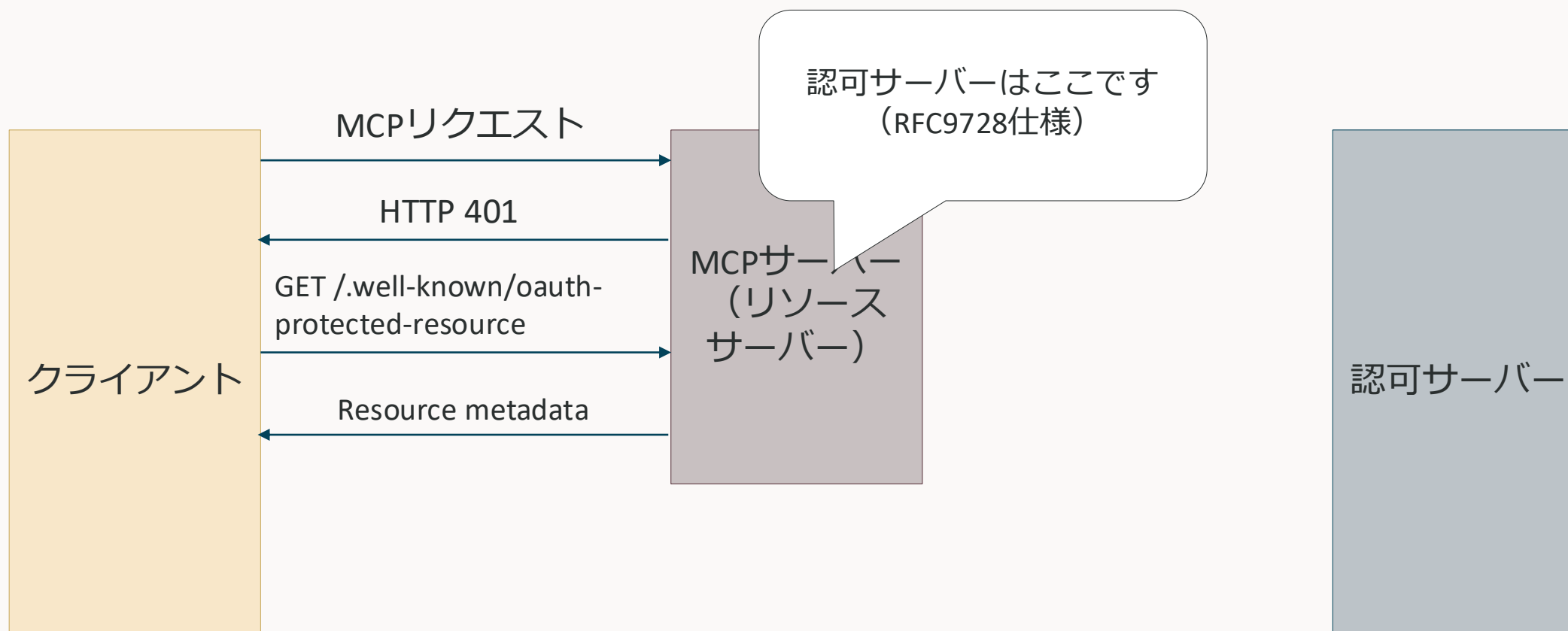
GET /.well-known/oauth-protected-resource

MCPサーバー
(リソースサーバー)

認可サーバー

認可サーバー検出フェーズ

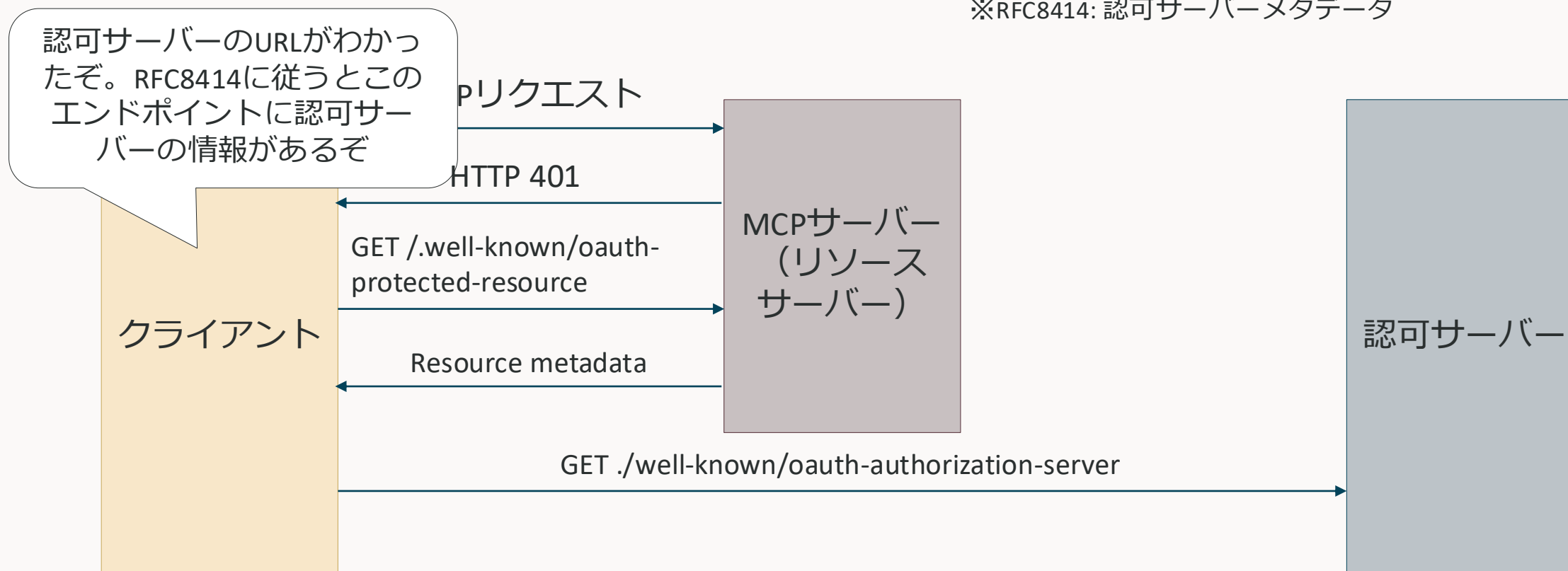
MCPクライアントは、接続したいMCPサーバーが要求する認可サーバーの情報を知らないので、それを教えてもらうフェーズ



認可サーバー検出フェーズ

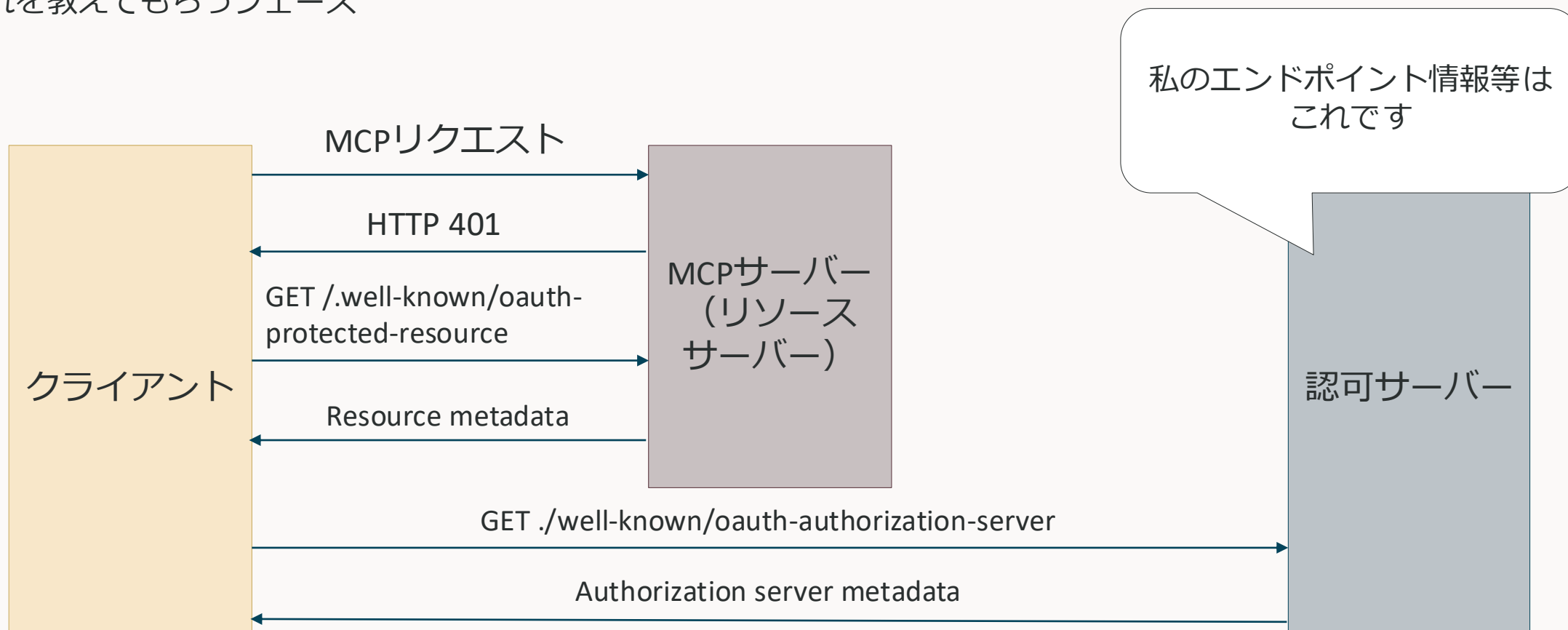
MCPクライアントは、接続したいMCPサーバーが要求する認可サーバーの情報を知らないので、それを教えてもらうフェーズ

※RFC8414: 認可サーバーメタデータ



認可サーバー検出フェーズ

MCPクライアントは、接続したいMCPサーバーが要求する認可サーバーの情報を知らないので、それを教えてもらうフェーズ



認可サーバー検出フェーズ

MCPクライアントは、接続したいMCPサーバーが要求する認可サーバーの情報を知らないので、それを教えてもらうフェーズ



クライアント登録フェーズ

仕様では、RFC7591（動的クライアント登録プロトコル）をサポートすべき（SHOULD）と書いてある

理由

- クライアントはMCPサーバーとその認可サーバーを事前に把握していない可能性がある
- 手動で全ての認可サーバーに登録するのは煩わしい
- 新しいMCPサーバーとシームレスに接続できる
- 認可サーバーは独自の登録ポリシーを実装できる

ただし、必ずしもRFC7591をサポートする必要はなく、その場合は

- MCPクライアントがクライアントID（コンフィデンシャルクライアントの場合はクライアントシークレットも）をハードコードする
- ユーザーが自分でOAuthクライアントに登録した後、これらの詳細を入力できるUIをユーザーに提示する

なお、Claude Desktopは動的・静的のどちらのクライアント登録も対応している



OAuthフロー

クライアント登録まで終われば、あとはOAuthのフローを行う

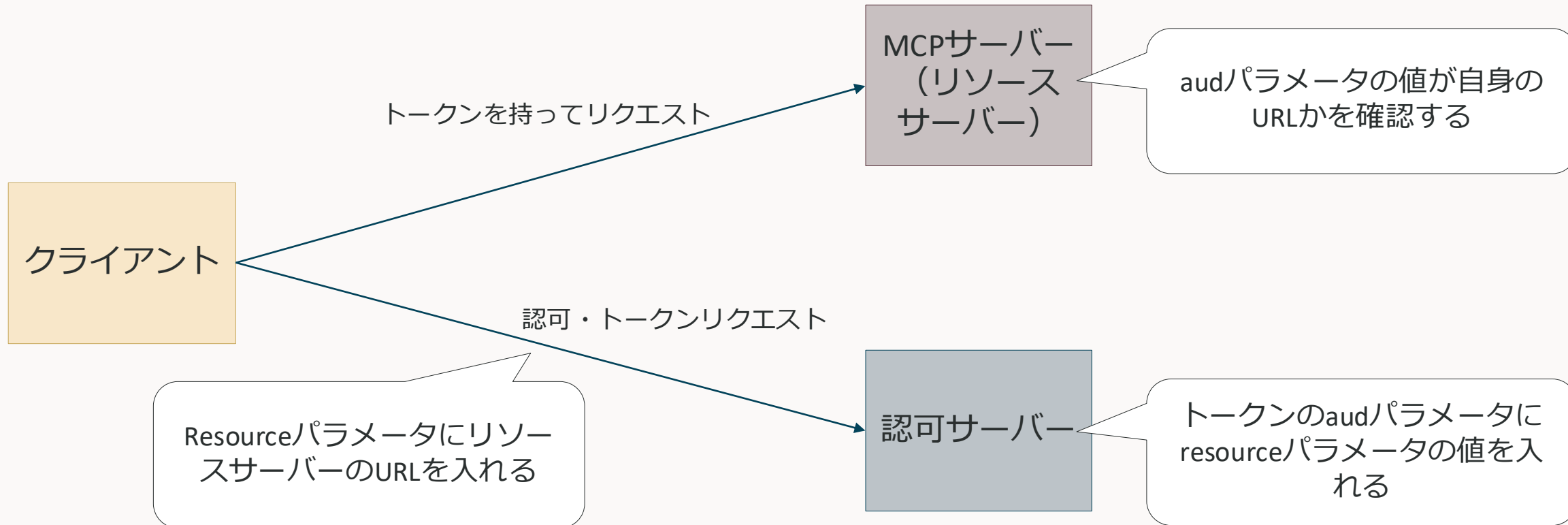
OAuth2.1 に従うので、グラントタイプは

- Authorization Code（PKCE必須）※PKCEに対応していないIdPもあるので注意
- Client Credentials
- Refresh Token（アクセストークン更新時のみ）

ただし、RFC8707で定義されているResource Indicatorsを実装する必要がある

Resource Indicators for OAuth2.0 (RFC8707)

- クライアントがトークンを取得する際に、トークンの使い道（リソースサーバー）を明示的に伝える
- 認可サーバーは、そのリソースサーバーをaudienceに含めるようにする（申告されたリソースサーバーでしか使えないようにする）



Resource Indicators for OAuth2.0 (RFC8707)採用の問題点

MCPのドキュメントには以下のように書かれている

*MCP clients **MUST** implement Resource Indicators for OAuth 2.0 as defined in [RFC 8707](#) to explicitly specify the target resource for which the token is being requested.*

つまり、認可サーバーがサポートしていなくてもresourceパラメータをつけて送れと言っているがこれが問題

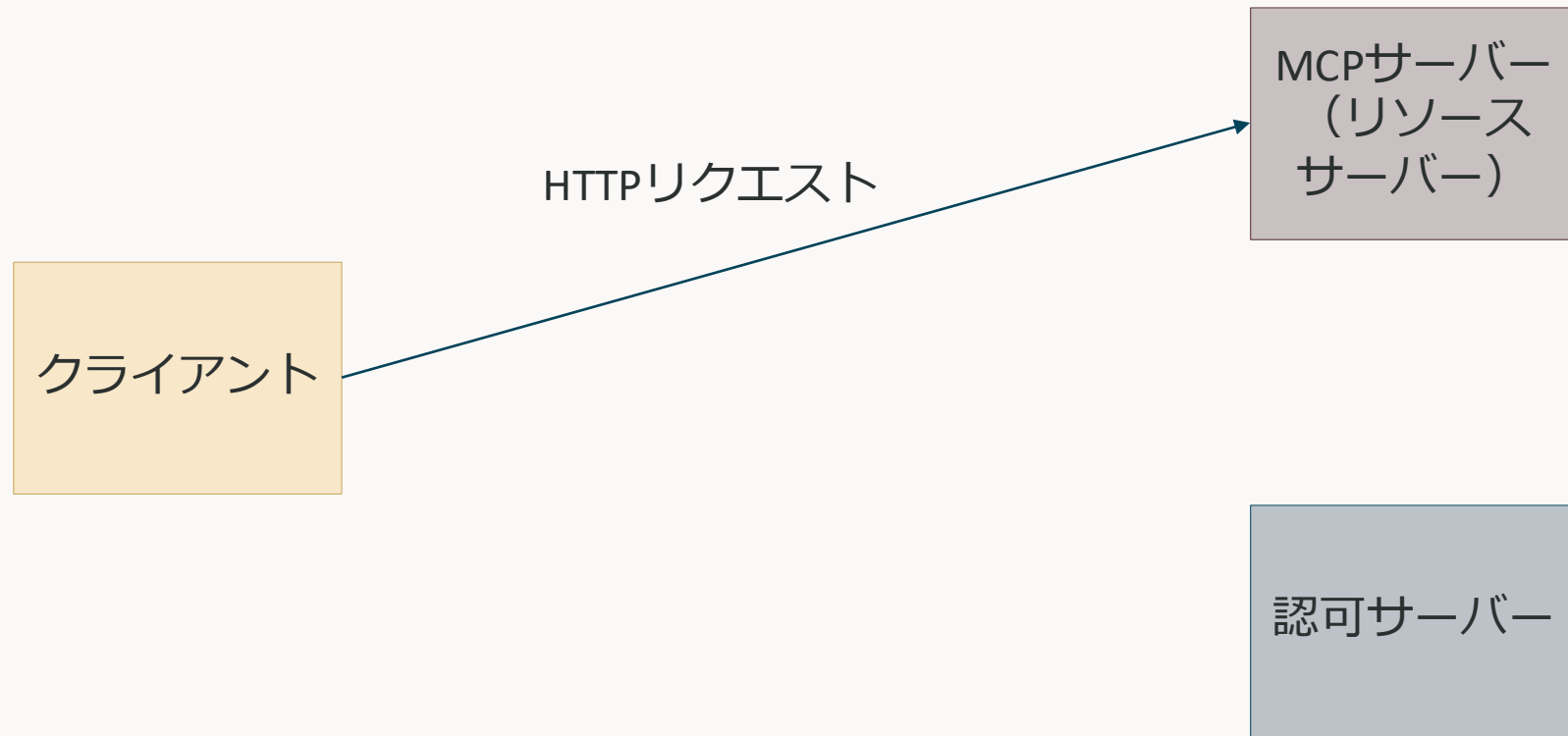
- IdPによっては、想定しないパラメータが送られてくるとエラーになる → OAuthフローが失敗する
- resourcesパラメータを無視するIdPでは、動作はするが意図したものではない（トークンが他のリソースサーバーでも使える）



- RFC8707に対応しているIdPを使う
- RFC8707は使わないようにする

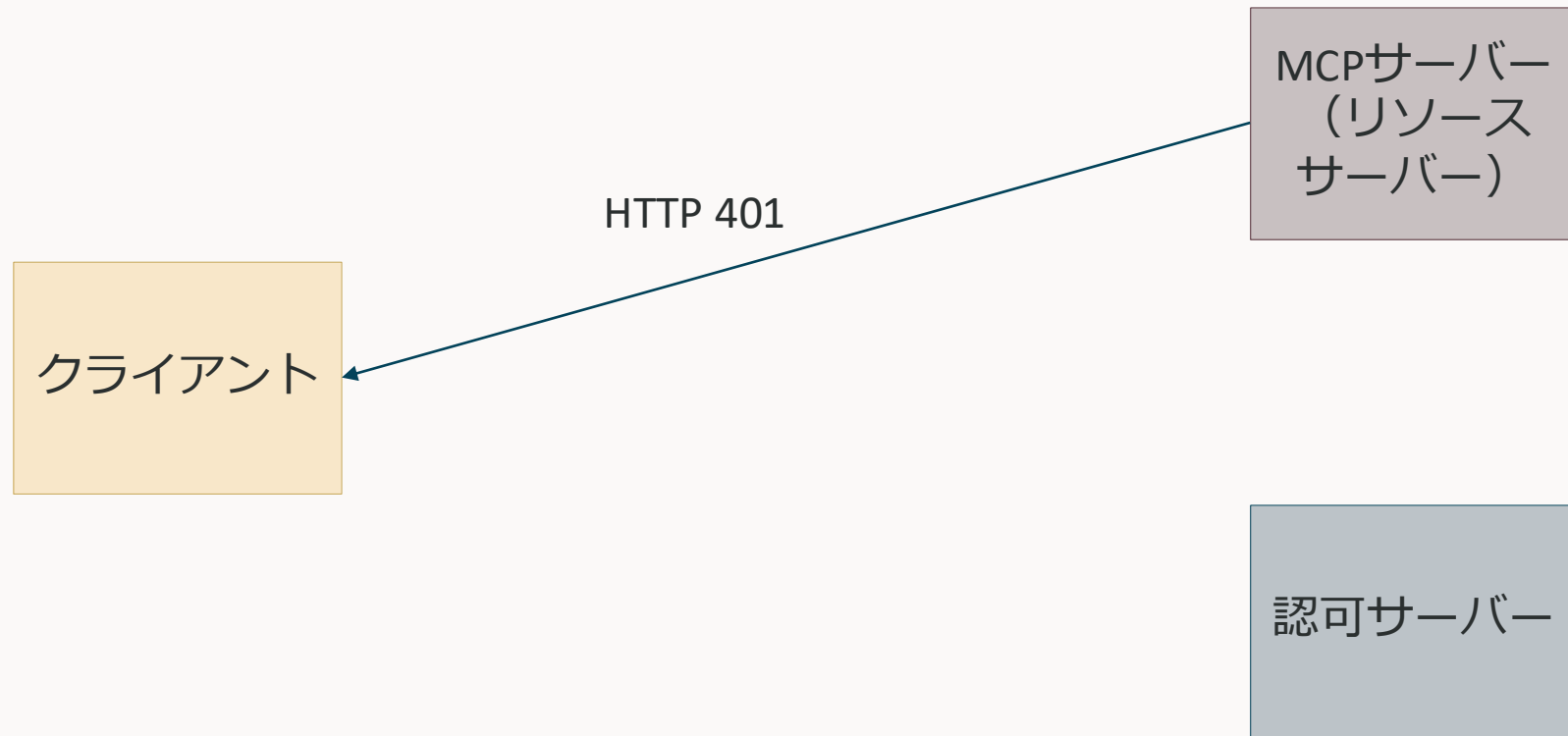
MCPの認可で現実的に辛いところを全体フローから眺める（2025-06-18）

1. 認可サーバーの検出（辛いところはない）



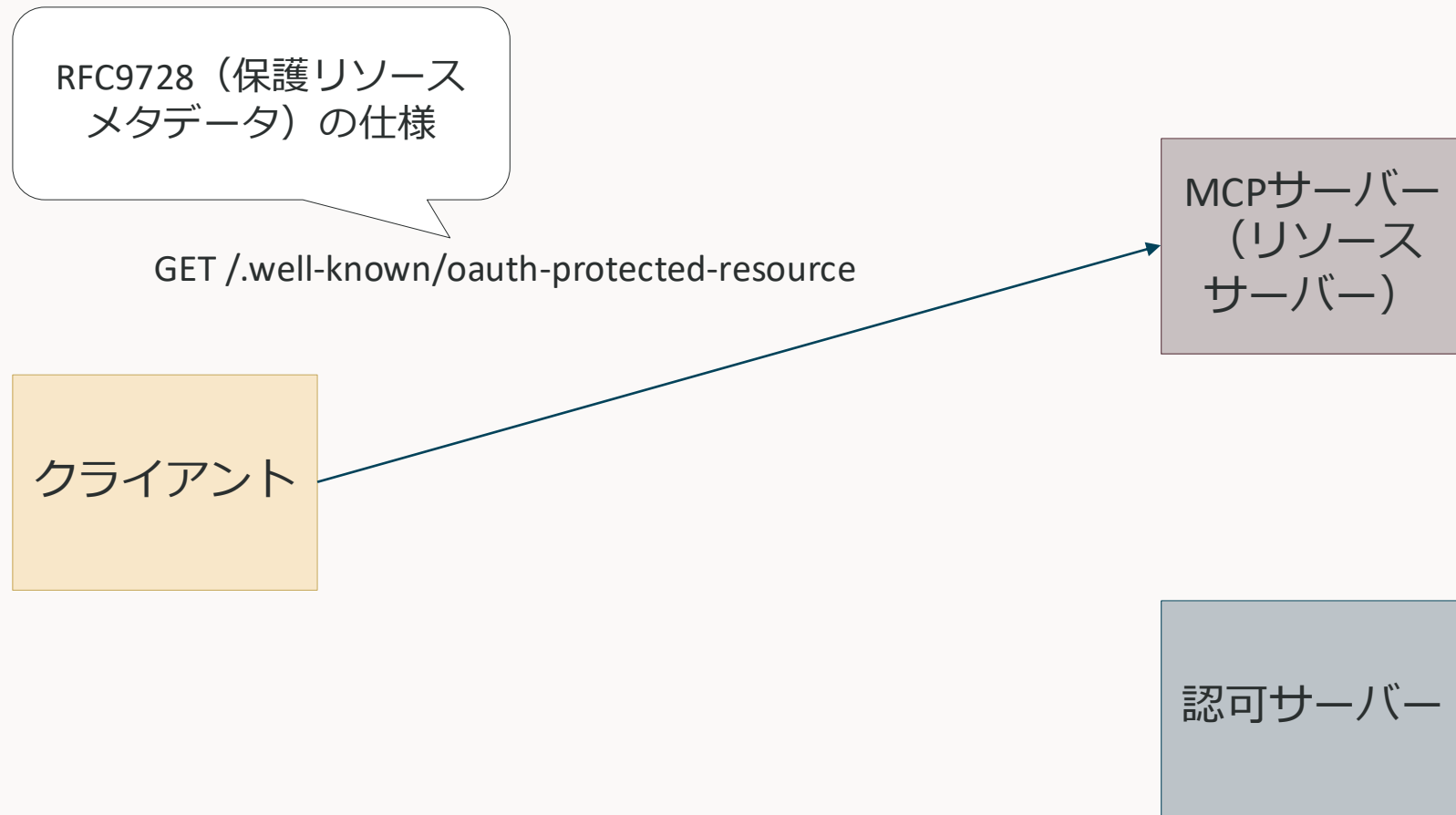
MCPの認可で現実的に辛いところを全体フローから眺める（2025-06-18）

1. 認可サーバーの検出（辛いところはない）



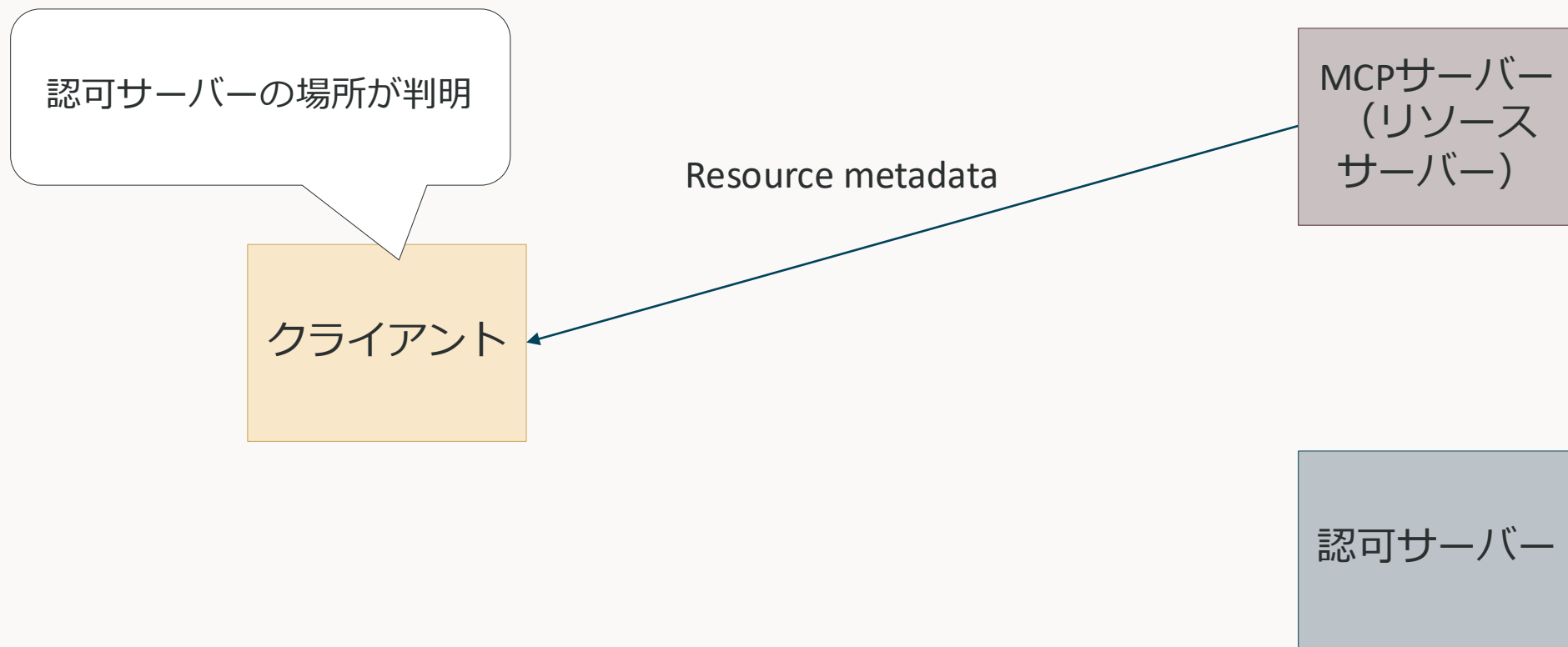
MCPの認可で現実的に辛いところを全体フローから眺める（2025-06-18）

1. 認可サーバーの検出（辛いところはない）



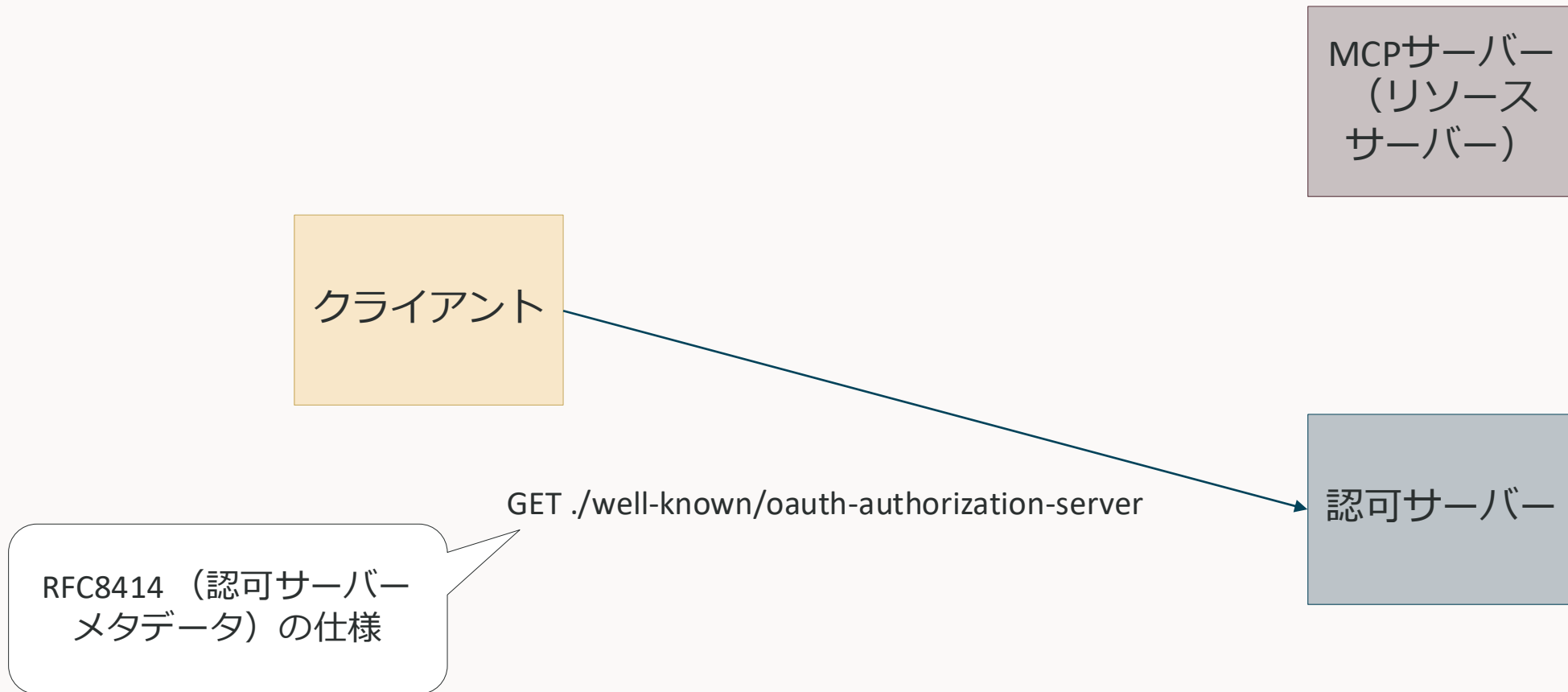
MCPの認可で現実的に辛いところを全体フローから眺める（2025-06-18）

1. 認可サーバーの検出（辛いところはない）



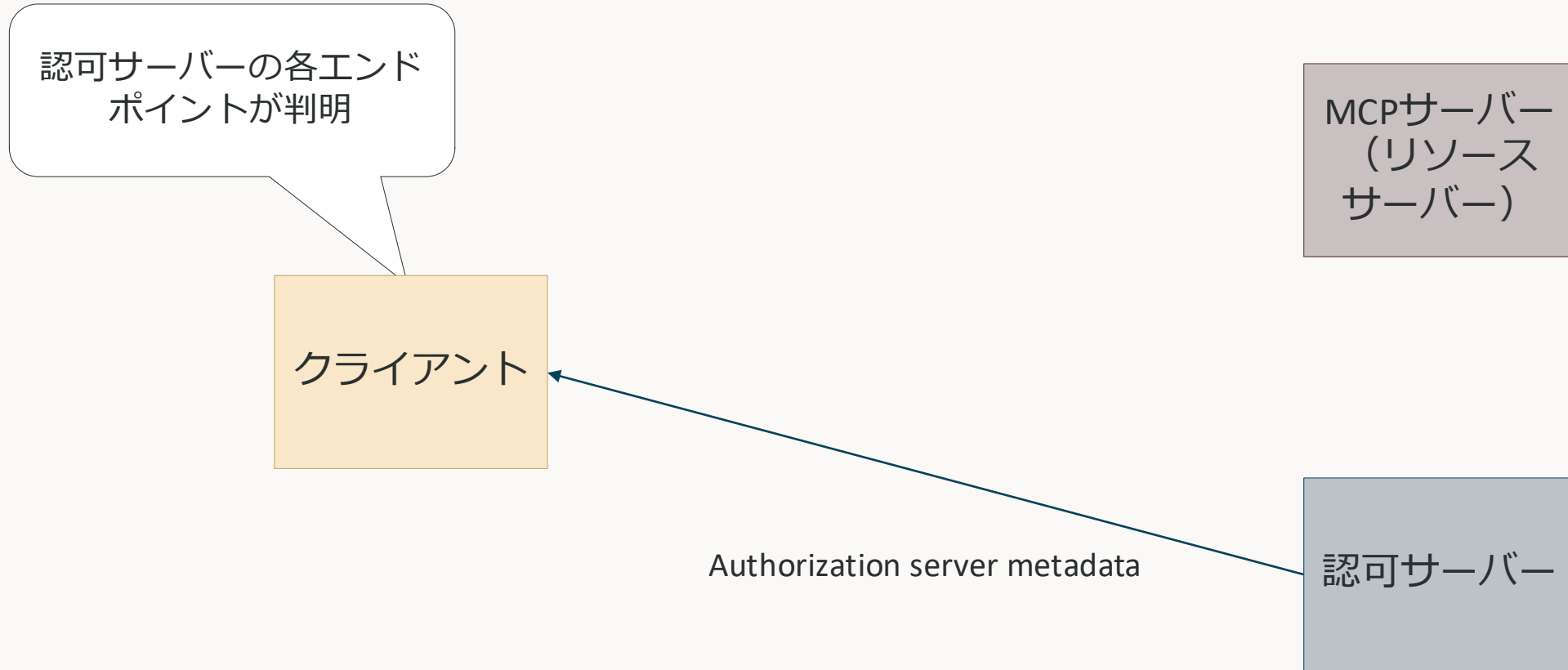
MCPの認可で現実的に辛いところを全体フローから眺める（2025-06-18）

1. 認可サーバーの検出（辛いところはない）



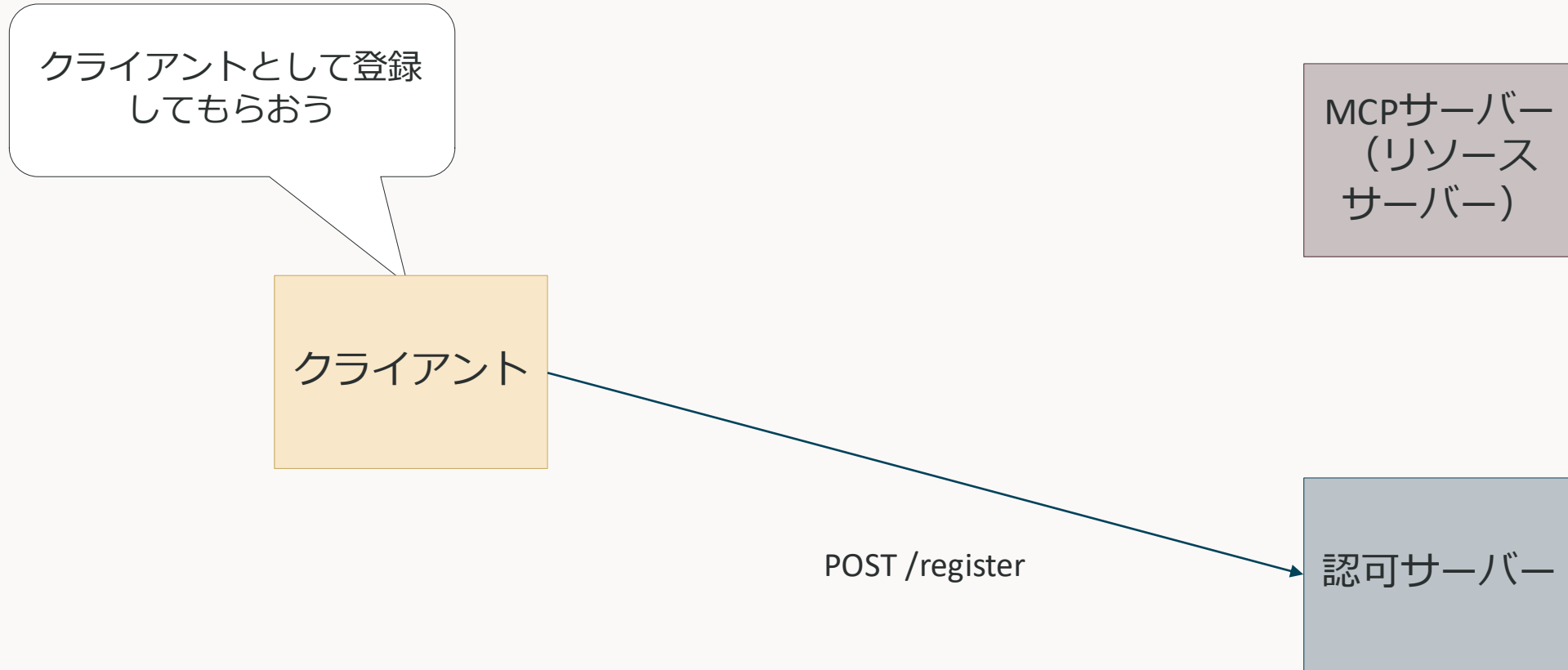
MCPの認可で現実的に辛いところを全体フローから眺める（2025-06-18）

1. 認可サーバーの検出（辛いところはない）



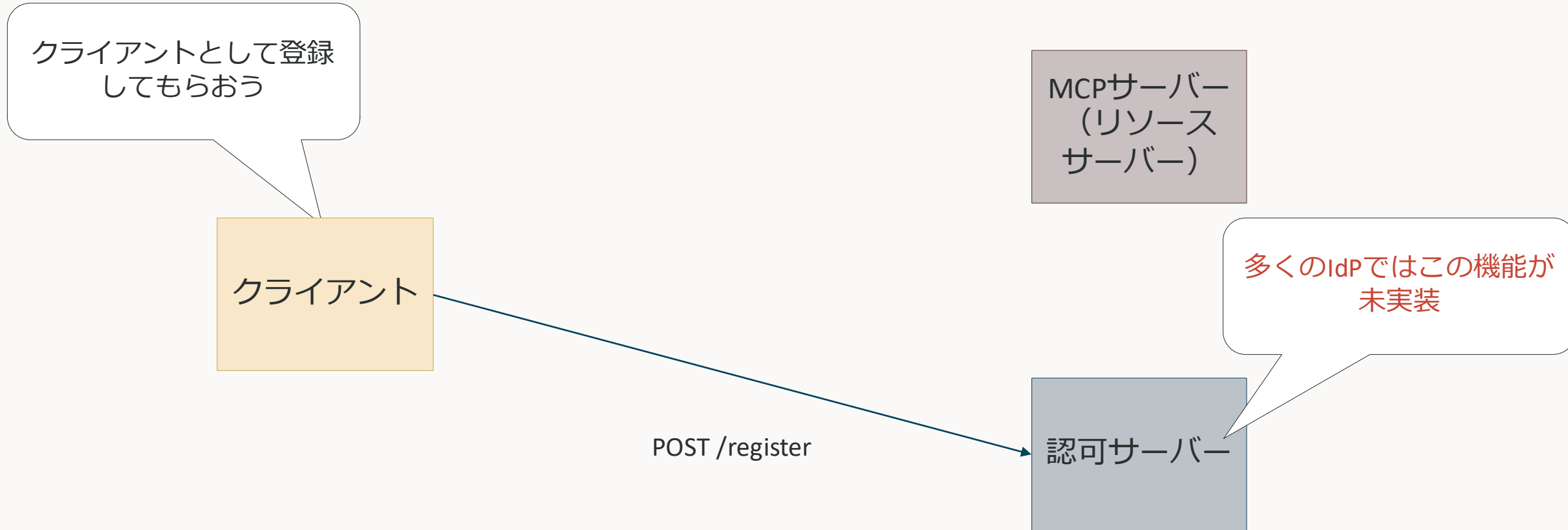
MCPの認可で現実的に辛いところを全体フローから眺める（2025-06-18）

2. 動的クライアント登録



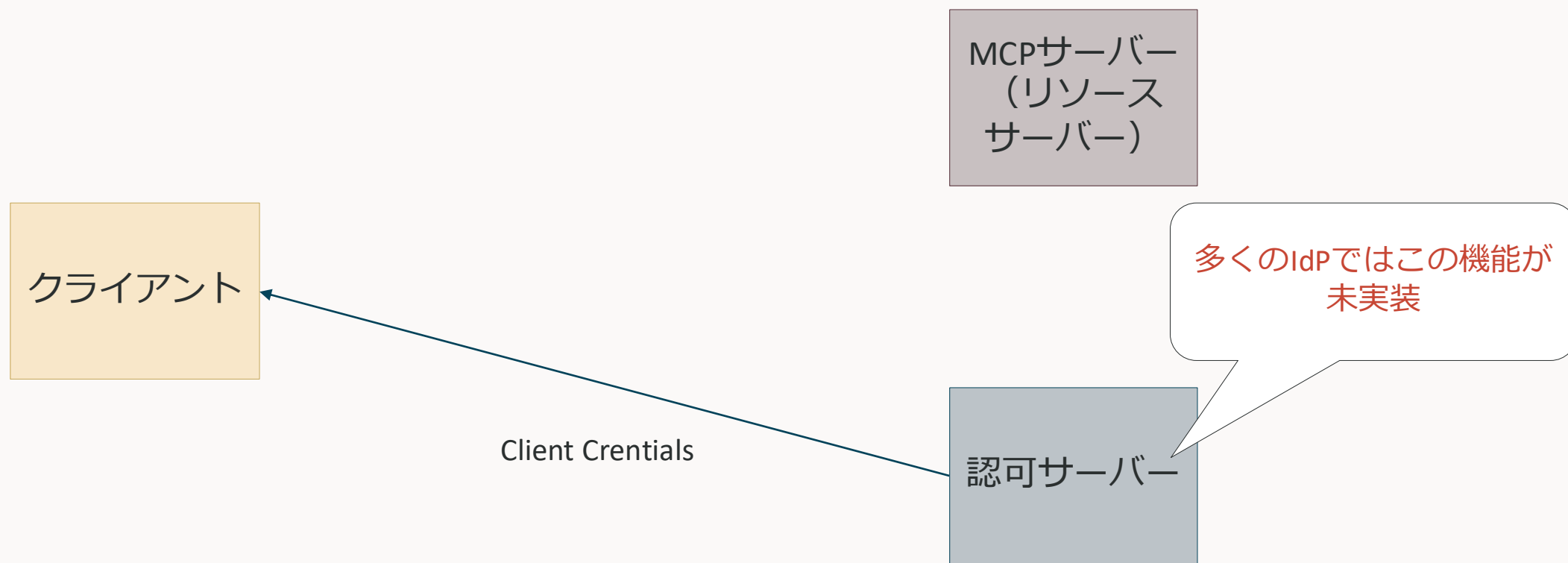
MCPの認可で現実的に辛いところを全体フローから眺める (2025-06-18)

2. 動的クライアント登録



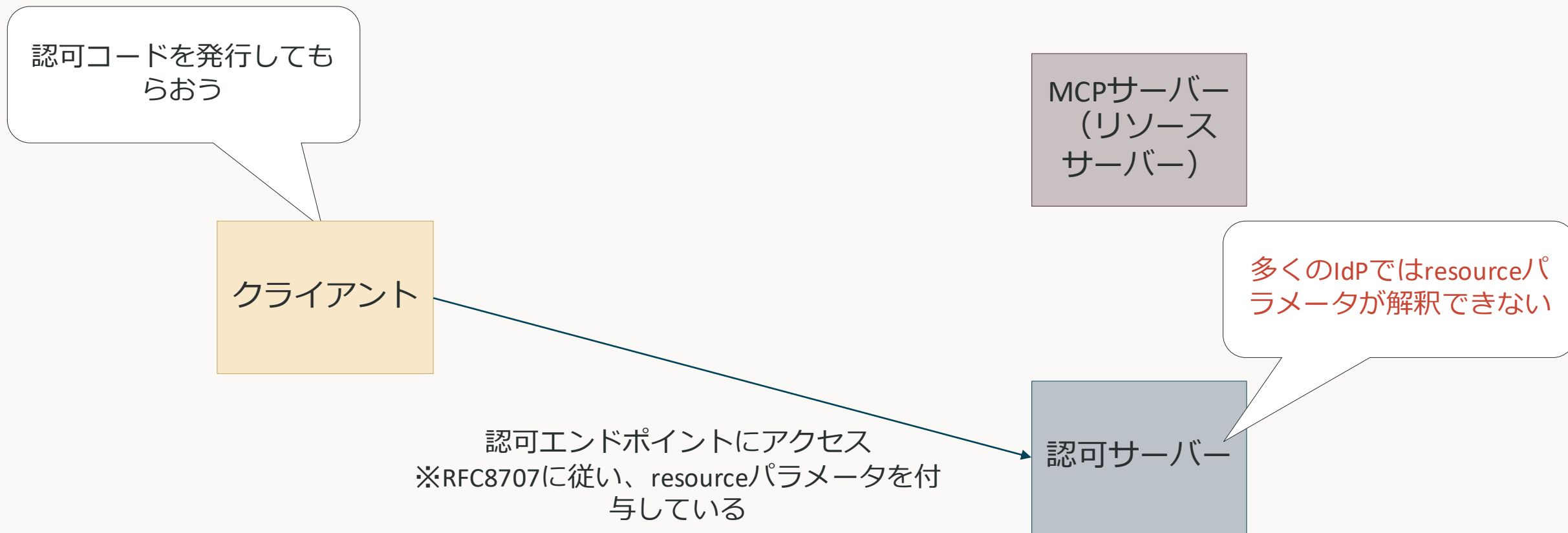
MCPの認可で現実的に辛いところを全体フローから眺める（2025-06-18）

2. 動的クライアント登録



MCPの認可で現実的に辛いところを全体フローから眺める（2025-06-18）

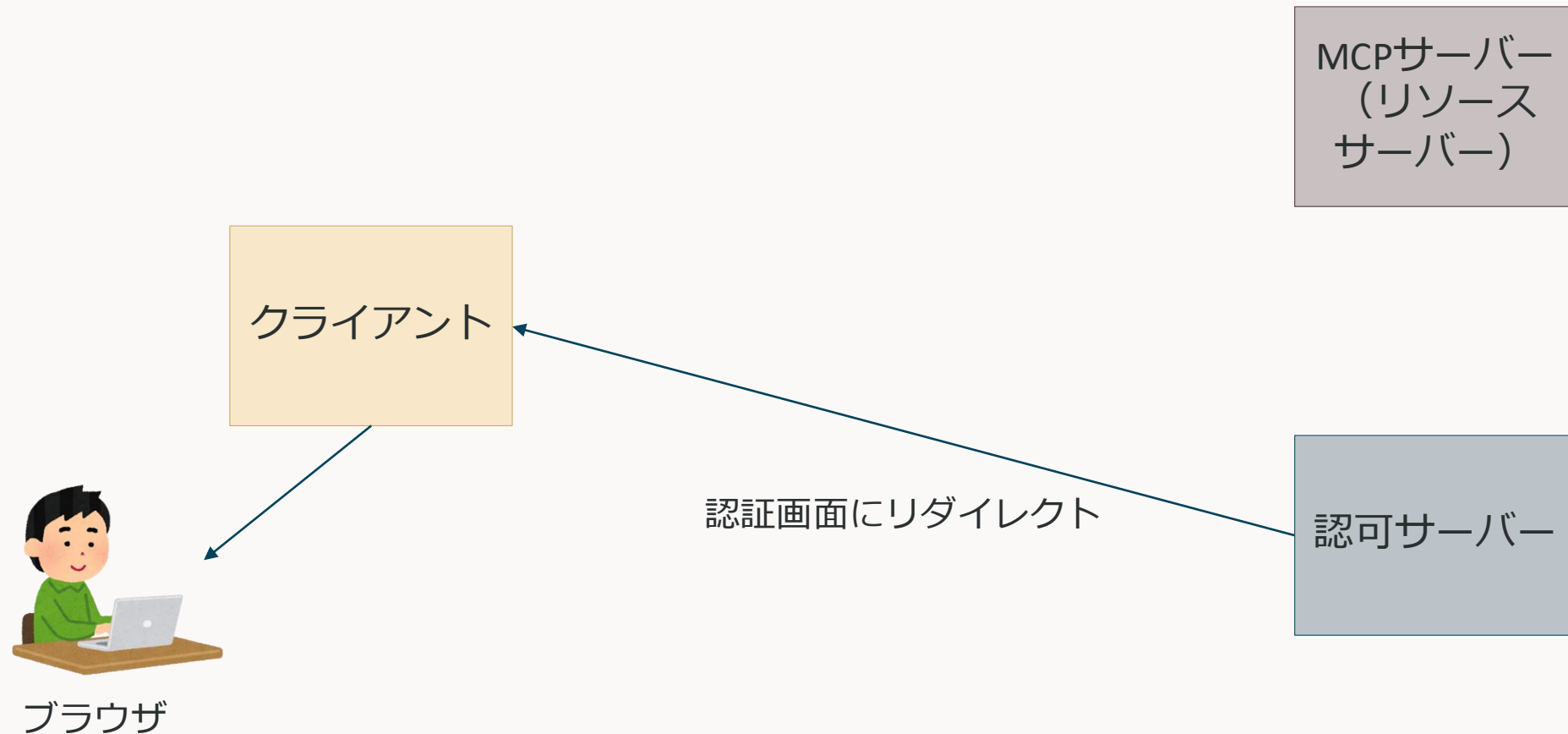
3. 認可フロー



※IdPの実装によっては、ここでOAuthフローが失敗する場合がある

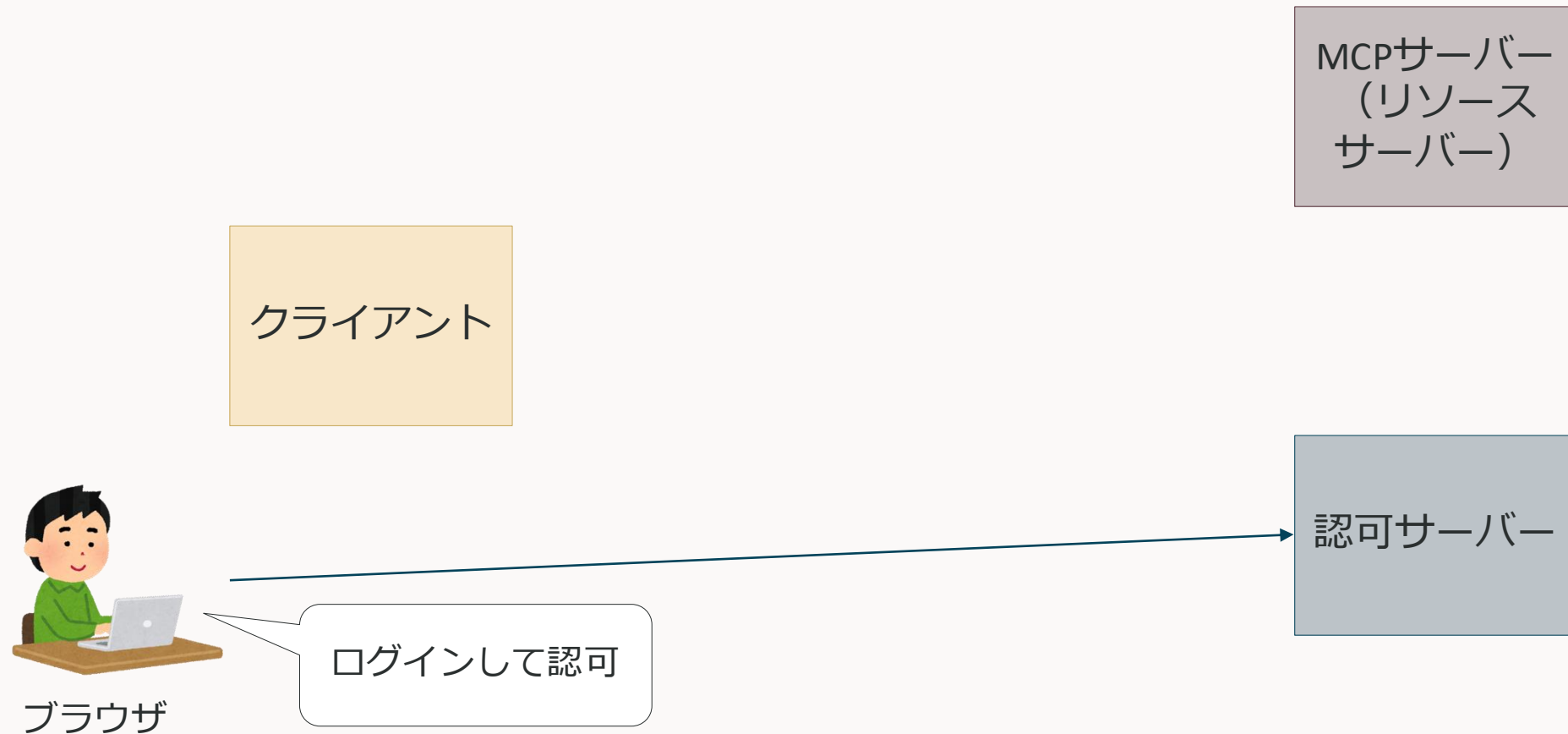
MCPの認可で現実的に辛いところを全体フローから眺める (2025-06-18)

3. 認可フロー



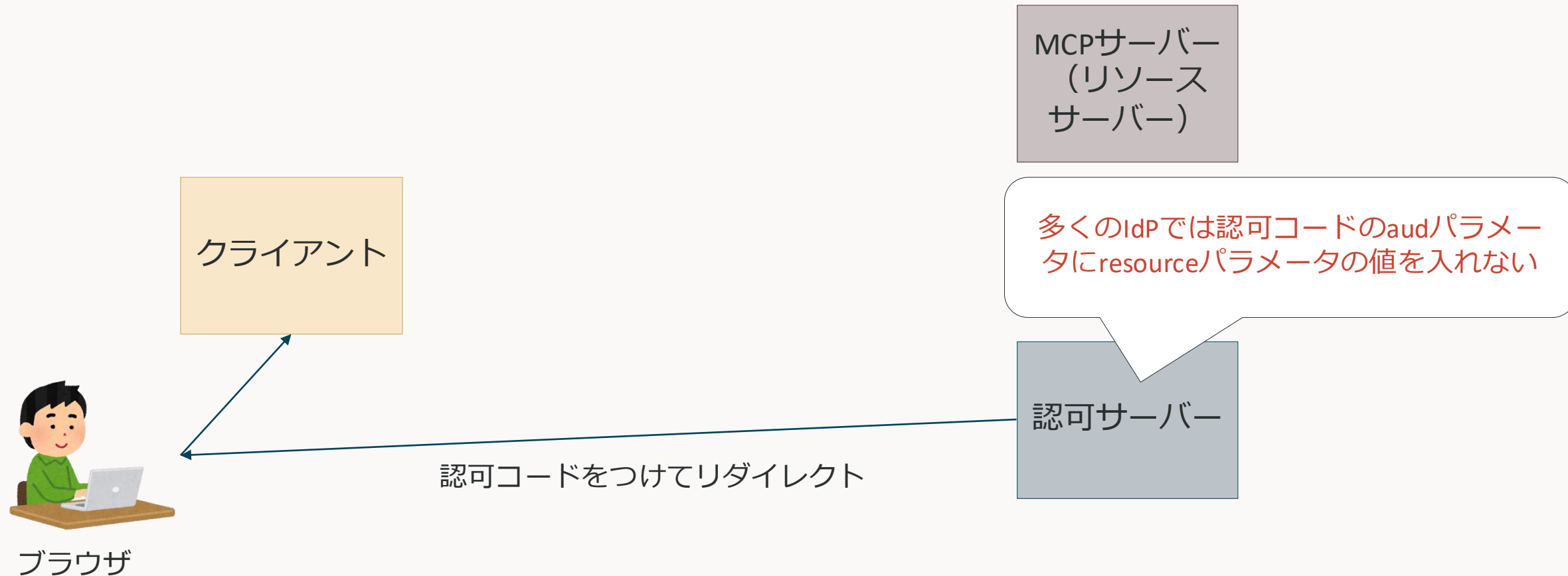
MCPの認可で現実的に辛いところを全体フローから眺める (2025-06-18)

3. 認可フロー



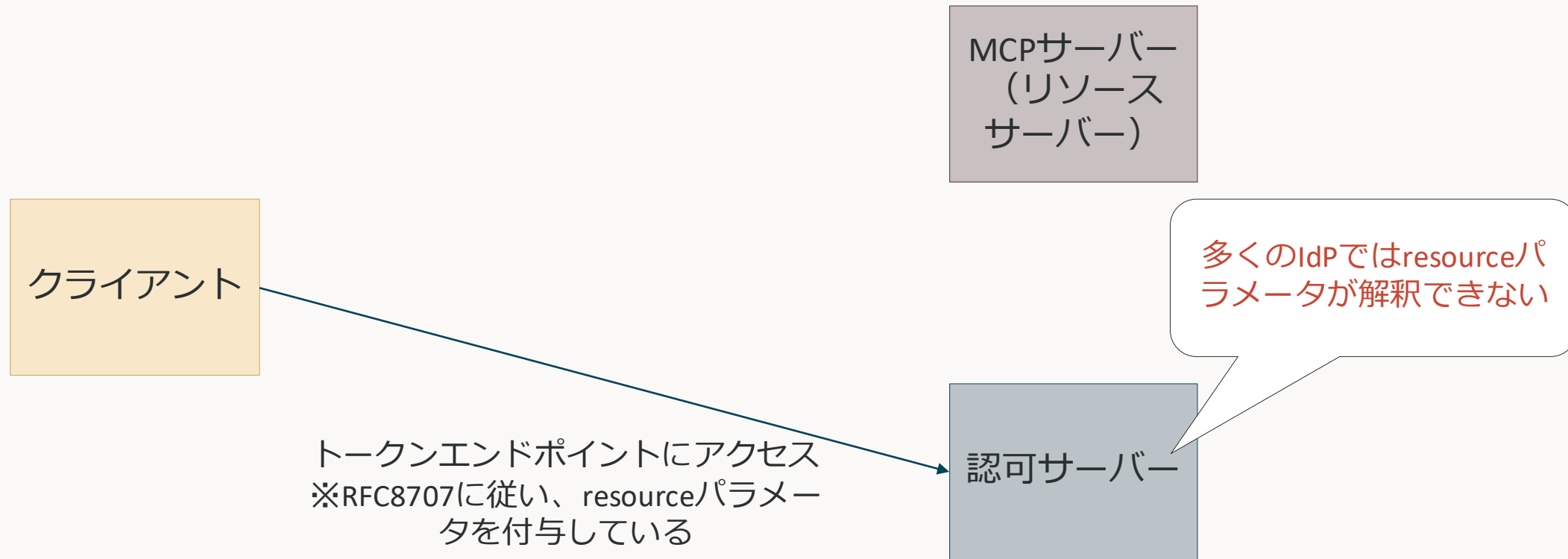
MCPの認可で現実的に辛いところを全体フローから眺める (2025-06-18)

3. 認可フロー



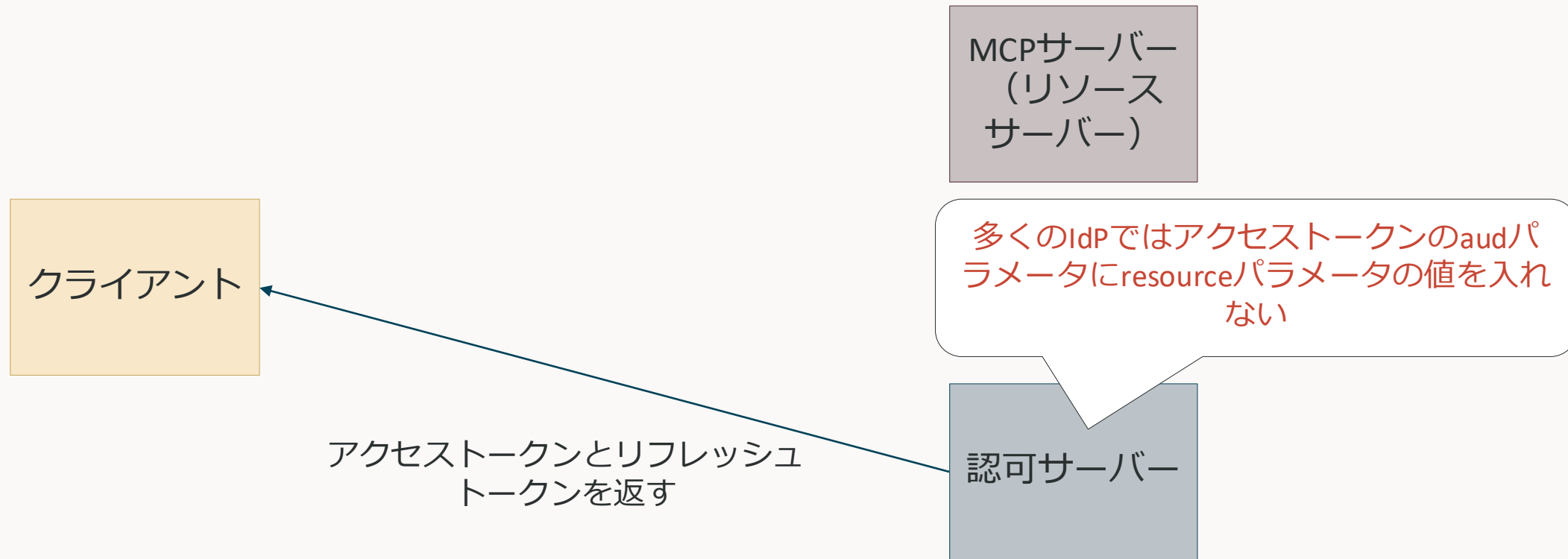
MCPの認可で現実的に辛いところを全体フローから眺める（2025-06-18）

3. 認可フロー



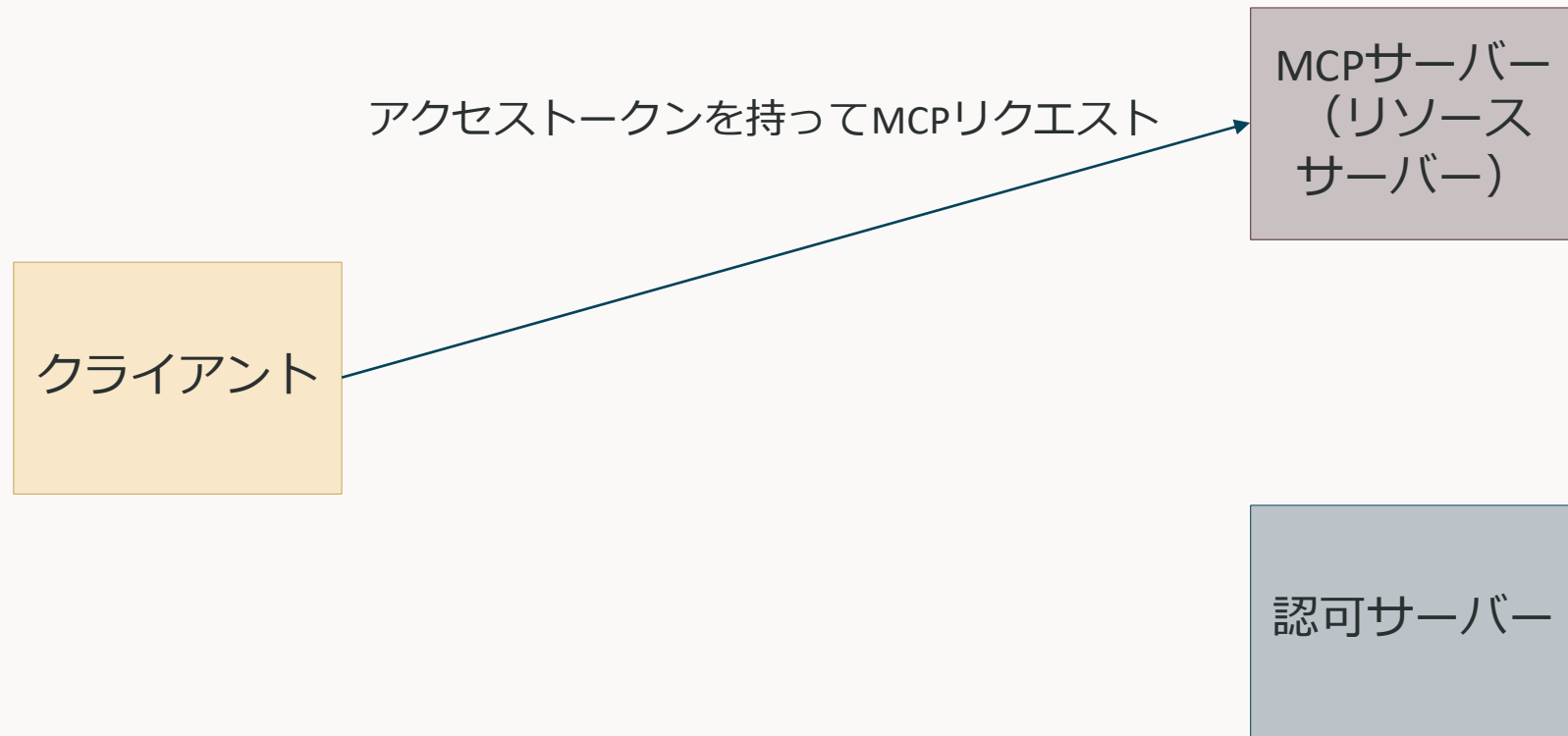
MCPの認可で現実的に辛いところを全体フローから眺める（2025-06-18）

3. 認可フロー



MCPの認可で現実的に辛いところを全体フローから眺める (2025-06-18)

3. 認可フロー



MCPの認可で辛いところまとめ

ドラフト版を標準としてるため、多くのIdPで対応していないものがある

- 動的クライアント登録
 - RFCだが対応しているIdPは限定的（Keycloakは対応済み）
- Resourcesパラメータ
 - RFCだが対応しているIdPは限定的（Keycloakは未対応）
- PKCE
 - 対応していないIdPも存在する（Keycloakは対応済み）

なお、2025-11-25では以下の問題もある

- クライアントIDメタデータドキュメント
 - ドラフトなので対応していない（Keycloakは未対応）

現状楽にMCPサーバーの認可を行うには？

- 動的クライアント登録・クライアントIDメタデータドキュメントを使わない
 - Anthropicが出しているClaude Desktopは静的クライアント登録にも対応している
 - 有名なMCP開発フレームワークのFastMCPでは、DCRを受けて静的クライアント登録に変換できる
- Resourcesパラメータ
 - 対応していない&エラーになるIdPを使う場合は指定しない
 - 対応している or エラーにならないIdPの場合は指定する
- PKCE
 - 対応している場合はAuthorization Code Grant with PKCE
 - 対応していない場合はClient Credentials Grantを使うしかないが注意が必要

MCPサーバーのObservability



MCPにおけるObservability

基本的にはWeb APIと変わらない

ログ・メトリクス・トレースの基本的な収集方法はWeb APIと大きくは変わらない

- ログ
 - リモートMCPサーバー（Streamable HTTP）の場合は、Web APIと同じ
- メトリクス
 - 基本的にはWeb APIと同じ（リクエスト数、レイテンシ、エラー率など）
 - MCPはエンドポイントが一つ（/mcp）なので、gRPCと同じ感じでJSON-RPCのmethodフィールドとツール名を組み合わせて識別する
 - エラーに関しては後述する違いがある
- トレース
 - Web APIと同様の実装パターンが使える（FastMCPではネイティブでOpenTelemetryをサポートしている）
 - MCP専用のOpenTelemetryセマンティックルールはまだドラフト段階



MCPのエラー

MCPのエラーは以下の3パターンある

- HTTPレベルのエラー
 - HTTPのエラーコードで把握できる
 - MCPサーバーが起動していないなどの場合に発生する（ある意味MCPの外側のエラー）
- JSON-RPCレベルのエラー
 - HTTP 200で返されるため、HTTPのエラーコードでは把握できない
 - 呼び出そうとしたメソッドが存在しない、パラメータの型が違うなどの場合に発生する
- isErrorフラグレベルのエラー
 - HTTP 200かつJSON-RPCレベルのエラーではない
 - プロトコルとしては正常に通信できたが、実行した中身（ツールなど）が失敗した場合に使われる

JSON-RPCレベルのエラー

HTTP200で返されるので、errorフィールドを確認する

発生するケース

- パースエラー (-32700)
 - 不正なJSON文法の時
- 不正なリクエスト (-32600)
 - JSON-RPCの型に沿っていないJSONの場合
- メソッド不存在 (-32601)
 - 存在しないJSON-RPCメソッドの呼び出し
 - tools/callをtool/callと呼び出した等
- 不正なパラメータ (-32602)
 - 必須フィールドがない場合など
- 内部エラー (-32603)
 - サーバー内部の予期しないエラー
- カスタムエラー (-32000~-32099)
 - サーバー実装固有のエラー用

```
{
  "jsonrpc": "2.0",
  "id": "request-123",
  "error": {
    "code": -32601,
    "message": "Method not found",
    "data": {
      "method": "tools/cal",
      "hint": "Did you mean 'tools/call'?"
    }
  }
}
```



isErrorフラグレベルのエラー

HTTP200, JSON-RPCのエラーコードも返さないなので、isErrorフラグをみる

発生するケース

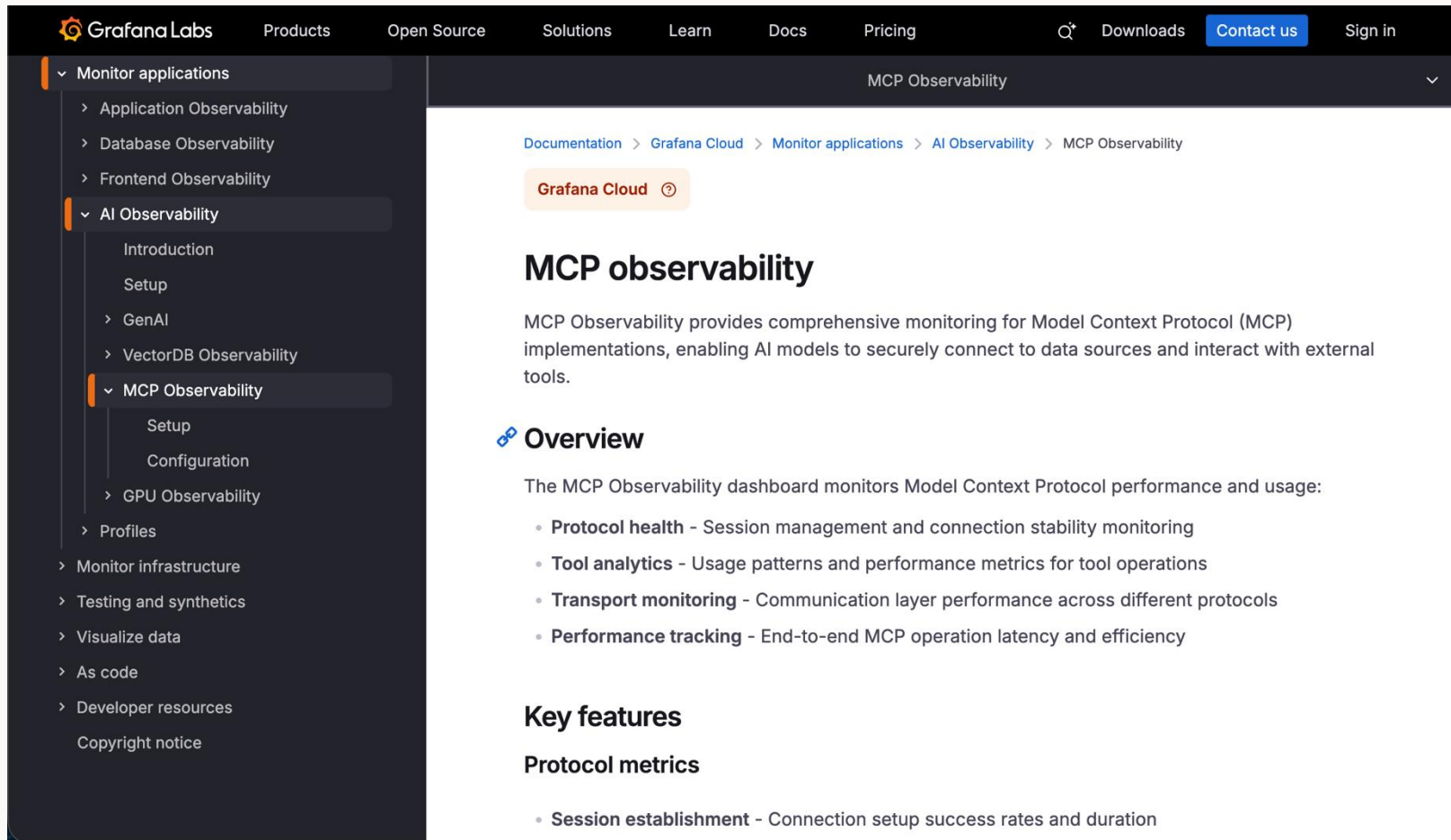
ツールは正常に呼び出されたが、ツール内部の処理が失敗した場合

- 外部APIの失敗
 - データベース接続タイムアウト
 - 外部APIのレート制限超過など
- 入力検証エラー
 - 予約日付が過去の日付、利用可能な席がない、など
- その他

```
{
  "jsonrpc": "2.0",
  "id": "request-789",
  "result": {
    "content": [
      {
        "type": "text",
        "text": "Failed to book flight: No available seats for the selecte
      ]
    ],
    "isError": true
  }
}
```



Grafana CloudではMCP用のダッシュボードも利用可能らしい



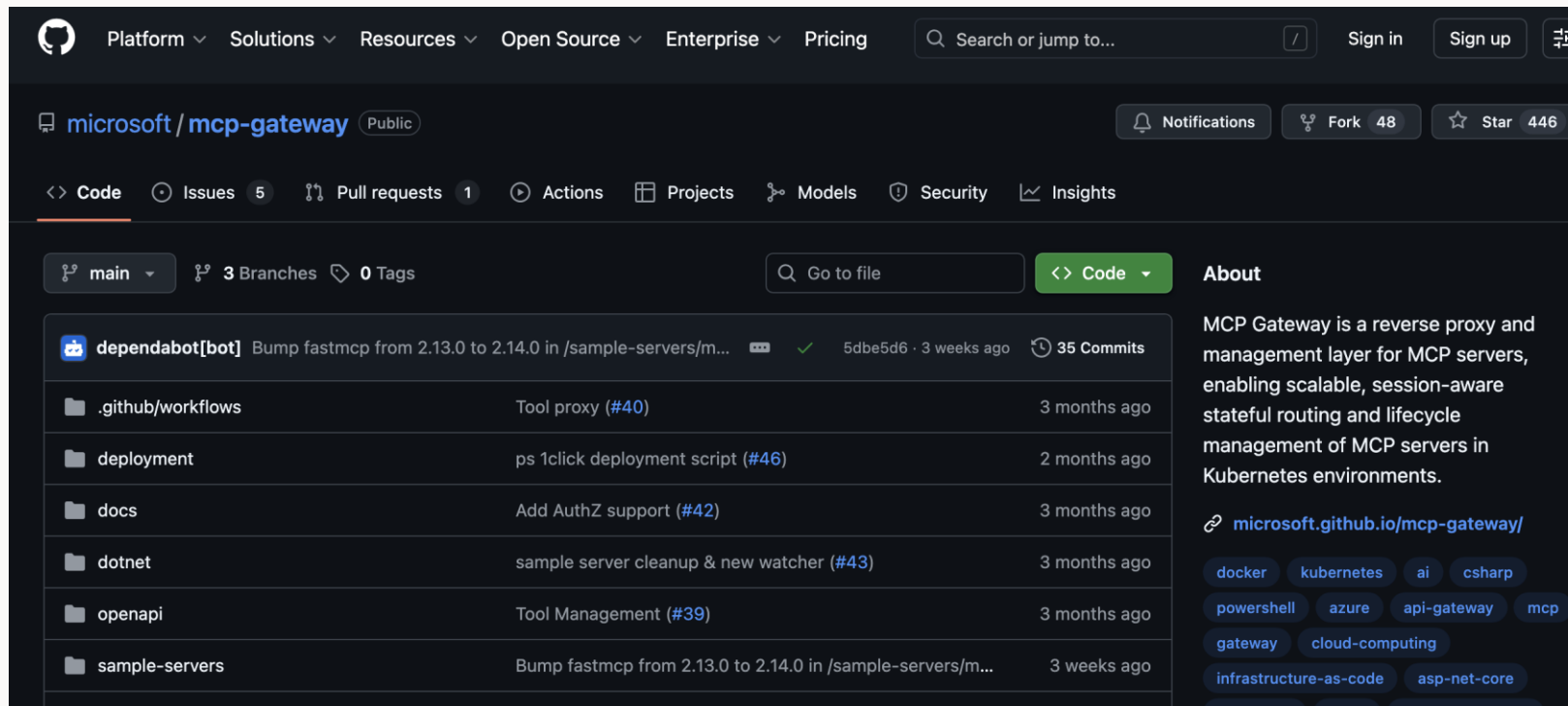
<https://grafana.com/docs/grafana-cloud/monitor-applications/ai-observability/mcp-observability/>

再掲: MCPゲートウェイという選択肢もある

MCPリクエストを適切なPodにルーティングしてくれる

Kubernetes上にMcp-Session-Idヘッダーを解釈できるLBを立てるイメージ

他にもいろいろできそう（MCPサーバーのライフサイクル管理、アクセス制御、Observabilityなど）



<https://github.com/microsoft/mcp-gateway>

MCP仕様へのネイティブOpenTelemetryサポート追加が議論されている

そのうち追加されるかも...?

The screenshot shows a GitHub discussion page for the repository `modelcontextprotocol/modelcontextprotocol`. The discussion is titled "[Proposal] Adding OpenTelemetry Trace Support to MCP #269" and was started by user `altryne` on April 3, 2025, in the "Ideas - General" category. The discussion includes a "Pre-submission Checklist" with two checked items: "I have verified this would not be more appropriate as a feature request in a specific repository" and "I have searched existing discussions to avoid duplicates". The "Your Idea" section contains an "Abstract" and a "Motivation". The abstract describes a proposal to integrate OpenTelemetry (OTel) tracing capabilities into the Model Context Protocol (MCP) to address the "black box" nature of MCP server interactions. The motivation states that as MCP gains traction, the complexity of applications built upon it will increase, and agent developers will need a standardized way for MCP servers to emit OTel trace spans back to the calling MCP client.

Category: Ideas - General

Labels: None yet

19 participants

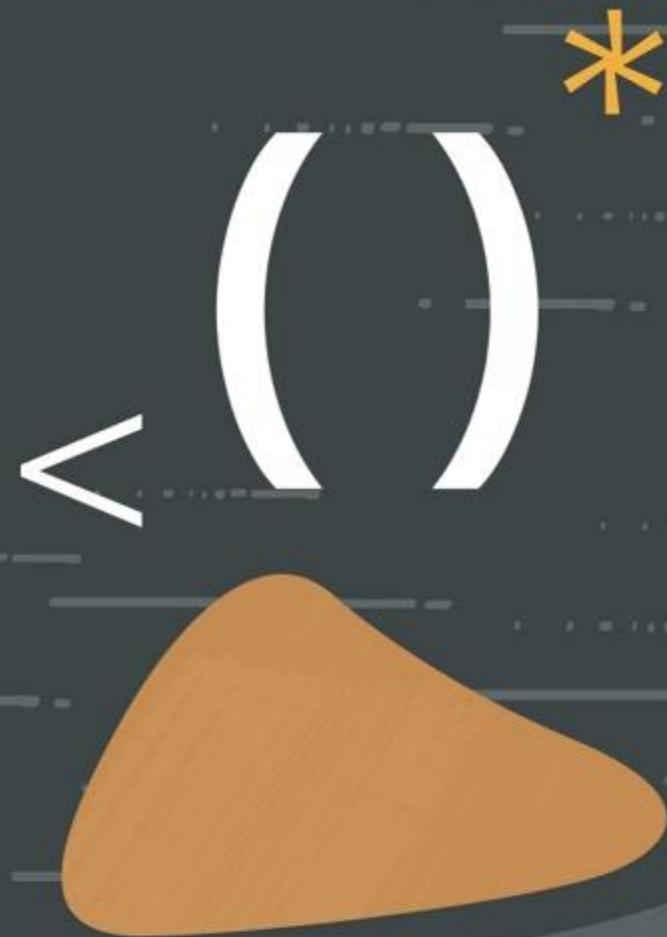
<https://github.com/modelcontextprotocol/modelcontextprotocol/discussions/269>

まとめ

- 基本的にはWeb API (REST API)やgRPCと同じ
- ログ
 - リモートMCPサーバー（Streamable HTTP）の場合は、Web APIと同じ
- メトリクス
 - 基本的にはWeb APIと同じ（リクエスト数、レイテンシ、エラー率など）
 - MCPはエンドポイントが一つ（/mcp）なので、gRPCと同じ感じでJSON-RPCのmethodフィールドとツール名を組み合わせて識別する
 - **エラーに関しては違いがある**
- トレース
 - Web APIと同様の実装パターンが使える（FastMCPではネイティブでOpenTelemetryをサポートしている）
 - MCP専用のOpenTelemetryセマンティックルールはまだドラフト段階
- まだあまり多くは情報が出ていない気がする...

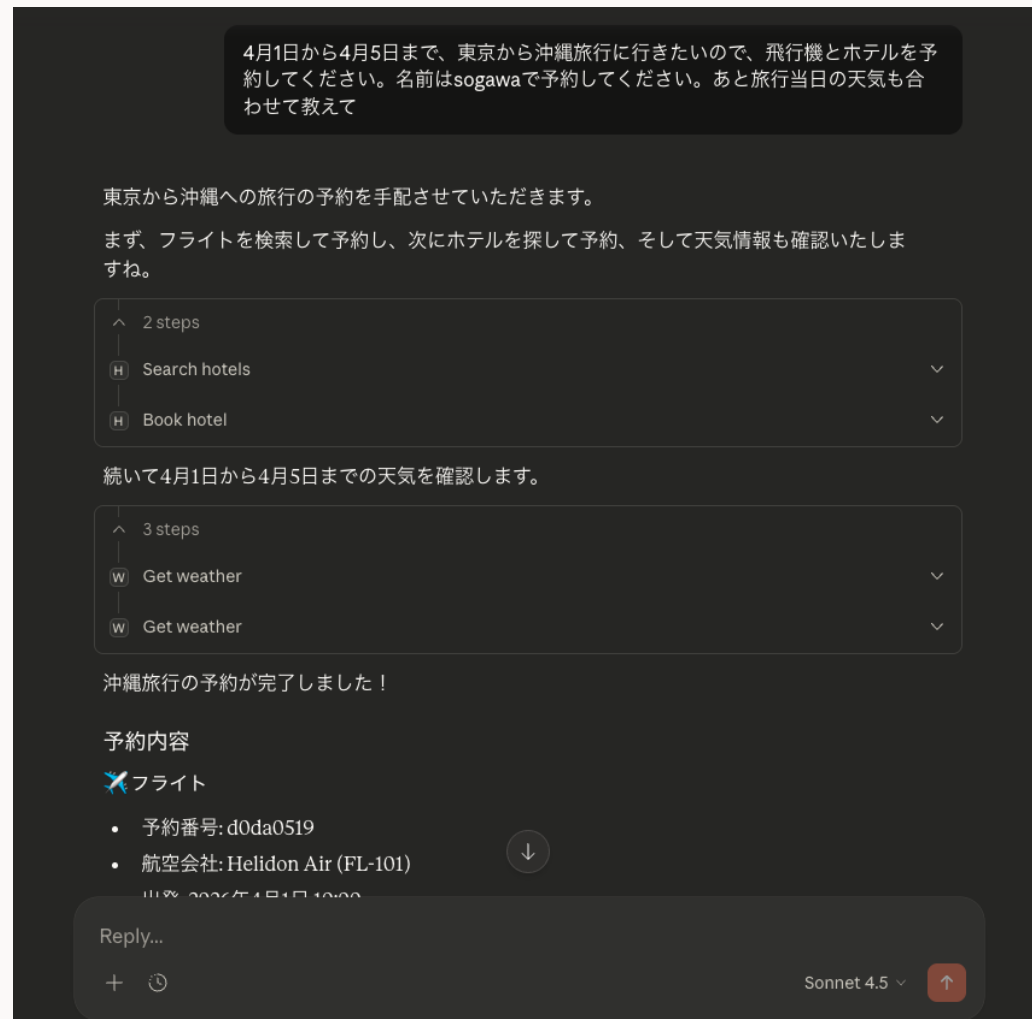


デモ



内容

Claude Desktopから旅行計画・予約を行うMCPサーバーをKubernetes上に構築



Helidon MCP

Helidonを使ってMCPサーバーを簡単に建てられる

アノテーションを使って簡単にツール・リソース・プロンプトを定義できる（アノテーションを使わない書き方も可能）

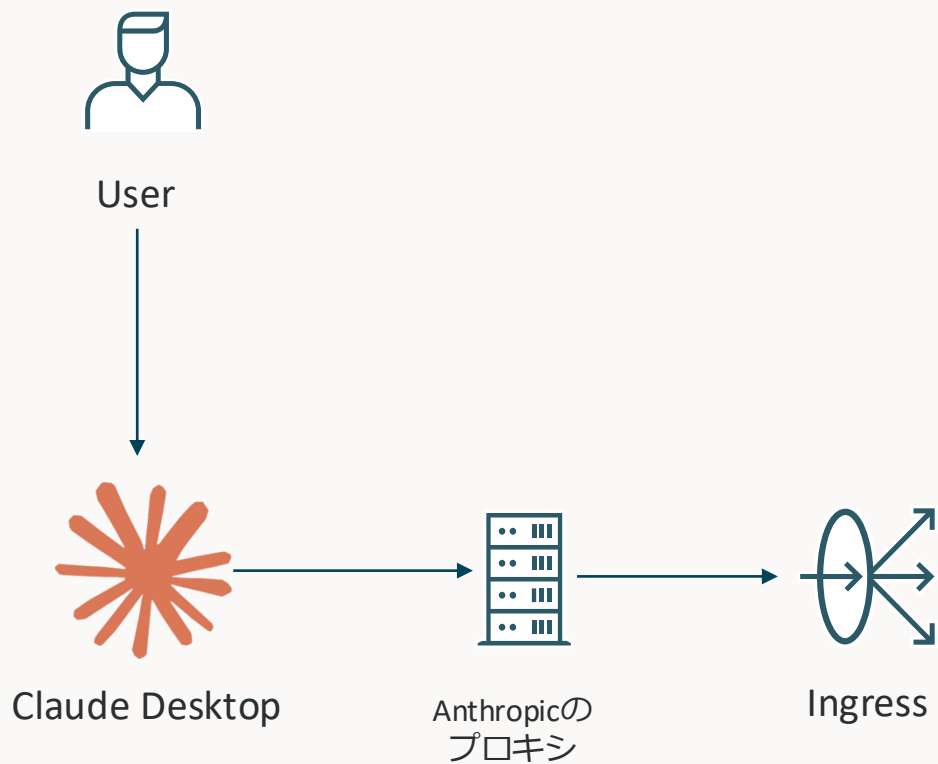
```
@Mcp.Tool("search_flights")
@RequiredScope("flight:read")
public List<McpToolContent> search_flights(String from, String to, String date) {
    JSONArray response = http1.get("/flights")
        .queryParams("from", from)
        .queryParams("to", to)
        .queryParams("date", date)
        .request(JsonArray.class)
        .entity();
    return List.of(McpToolContents.textContent(response.toString()));
}
```

ほとんどのMCPサーバーの機能をサポートしている（Elicitationは未実装）

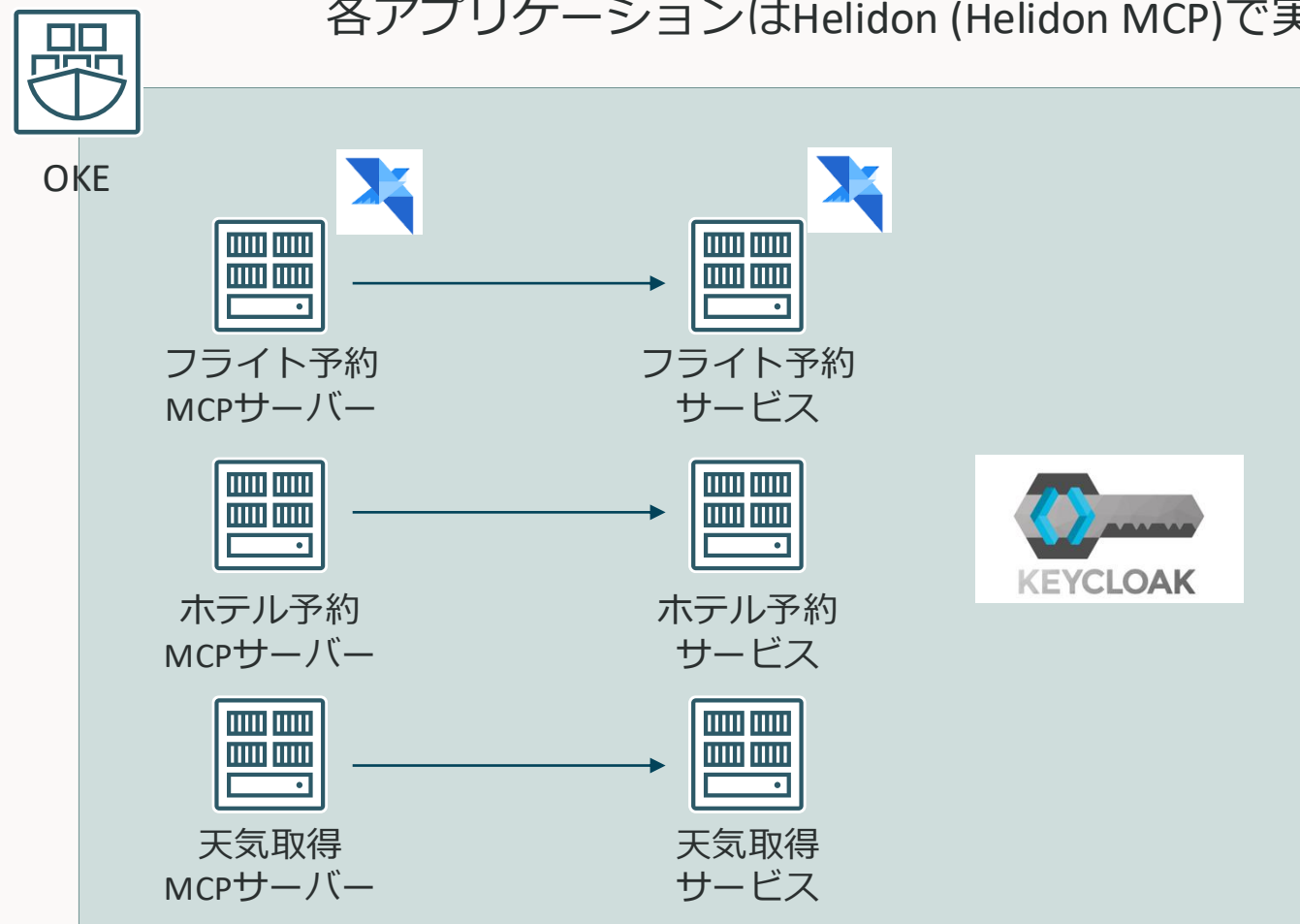
すでにHelidonを利用している or Javaを利用している場合は非常に便利

<https://github.com/helidon-io/helidon-mcp/tree/main>

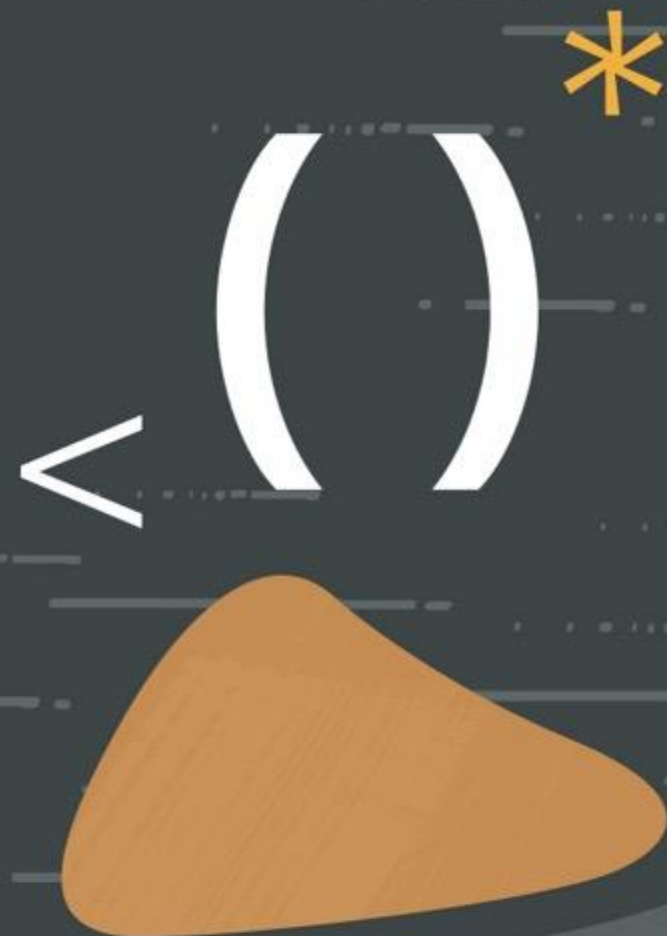
構成



各アプリケーションはHelidon (Helidon MCP)で実装



まとめ



まとめ

- MCPは、AIアプリケーションを外部システムに接続するための標準規格
- リモートMCPサーバーでは、ローカルMCPサーバーとは異なり以下のようなことを考慮する必要がある
 - スケーラビリティ
 - 利用できる機能は絞られるが、現実的にはステートレスにMCPサーバーを構築する方が楽
 - 認可
 - 利用するIdPに応じて方法を選ぶ必要がある
 - 現状は静的クライアント登録で、IdPに応じてresourcesパラメータを使うか使わないか選ぶのが楽
 - クライアントが動的クライアント登録しか使えない場合はプロキシを使うのが楽
 - 可観測性
 - 基本的にはREST APIと変わらない
 - エラーの扱いには注意する

参考

MCP公式ドキュメント

<https://modelcontextprotocol.io/docs/>

FastMCP公式ドキュメント

<https://gofastmcp.com/getting-started/welcome>

(Claude Desktop) Building Custom Connectors

<https://support.claude.com/en/articles/1150383-4-building-custom-connectors-via-remote-mcp-servers>

Helidon MCP

<https://github.com/helidon-io/helidon-mcp>

いまさらOAuth2.0から2.1での変更点

<https://zenn.dev/hashi8084/articles/0cf69e15bf793a>

MCP認可フローを仕様から読み解く（2025-06-18）

<https://qiita.com/yokawasa/items/74717789465ef2aeedfe>

MCPのOAuth認証で仕様通りに作るのが難しいこと3選

<https://zenn.dev/ncdc/articles/40a7a51a2a016d>

マイクロサービスの認証・認可とJWT (OCHaCafe #S4)

<https://speakerdeck.com/oracle4engineer/authentication-and-authorization-in-microservices-and-jwt>

参考

OAuth2.1 IETF Draft

<https://datatracker.ietf.org/doc/html/draft-ietf-oauth-v2-1-13>

RFC8414

<https://datatracker.ietf.org/doc/html/rfc8414>

RFC7591

<https://datatracker.ietf.org/doc/html/rfc7591>

RFC9728

<https://datatracker.ietf.org/doc/html/rfc9728>

Integrating with Model Context Protocol (MCP)

<https://www.keycloak.org/securing-apps/mcp-authz-server>

RFC8707

<https://www.rfc-editor.org/rfc/rfc8707.html>

OAuth クライアントIDメタデータドキュメント IETF Draft

<https://datatracker.ietf.org/doc/html/draft-ietf-oauth-client-id-metadata-document-00>

MCP Gateway

<https://github.com/microsoft/mcp-gateway>

[Proposal] Adding OpenTelemetry Trace Support to MCP

<https://github.com/modelcontextprotocol/modelcontextprotocol/discussions/269>

ORACLE